

recount — reproduction and analysis of the Csűrös 2026 GLD framework

Contents

recount	1
Install	1
Quick start — the recount CLI	2
The canonical bounded-fit campaign	3
Constraint regime	7
Does GLD give a sound foundation for ancestral genome inference?	8
Supporting data	29
Performance	40
Validation against Csurös' Java	41
Repository tour	42
Per-family per-node presence tables	42
Low-level Python API	43
Model	44
Indexing convention	44
Citing	44
L(0)-corrected trajectories	45
License	53

recount

Faithful re-implementation of Miklós Csűrös' [Count](#) — the phylogenetic gain-loss-duplication (GLD) likelihood, its analytical gradient, the arbitrary- Ω min unobserved-profile correction (Csűrös 2026 PNAS, SI Theorems 3–5), and the per-branch posterior expectations — in optimised CPU C with thin Python bindings. Four backends are available:

- **Native C** (`recount.native_backend`) — the production path. Apple Accelerate vForce + libdispatch threading; matches Java to machine precision at the LCA, **20–80× faster on M4 Max 16-core vs Csurös' Java** on the same hardware ([direct measurement](#)). Builds automatically on first import (`make -C native`) if the dylib isn't present (opt out with `RECOUNT_NO_AUTOBUILD=1`).
- **PyTorch** (`recount.torch_fast`) — vectorised reference + MPS path.
- **NumPy** (`recount.gld`) — pure-Python reference for testing.
- **Triton** (`recount.destructive_combine`, stubbed) — $O(W^2)$ destructive combine path for CUDA.

recount reproduces the Csűrös 2026 PNAS GLD analysis via a single 4-mode canonical campaign ([validation/Q UEUE_INSTRUCTIONS.md](#)), and then uses that reproduction to test the paper's closing claim — that GLD is a sound foundation for ancestral gene-content inference across a kingdom. See [Demonstration of reproduction](#) and [Does GLD give a sound foundation for ancestral genome inference?](#) below.

Install

```
git clone https://github.com/ssolo/recount.git && cd recount
pip install -e '.[torch,test]'           # NumPy + PyTorch + pytest
```

Requires Python 3.10. The native C backend builds on first import (`make -C native` runs if `librecount.dylib` / `.so` is missing); set `RECOUNT_NO_AUTOBUILD=1` to skip auto-build.

Apple Silicon Mac (production target)

No extra setup. The Makefile picks up `-mcpu=apple-m1` + Accelerate + libdispatch automatically. (`apple-m1` is the portable Apple Silicon baseline ISA target — the same binary runs and tunes well on M1/M2/M3/M4. For a marginal extra ~3% on the local chip only, use `make native` → `-mcpu=native`. Benchmarks throughout this README were measured on an M4 Max 12P+4E.)

```
cd native && make && cd ..           # optional: pre-build
python3 -c "from recount.native_backend import native_version; print(native_version())"
```

Intel Mac

Same `make` recipe; the Makefile detects `arm64` vs `x86_64` and falls back to `-march=native`.

Linux (x86_64 or aarch64)

```
sudo apt install build-essential libomp-dev      # or your distro's equivalent
cd native && make && cd ..
```

OpenMP replaces libdispatch on Linux; numerics are identical, threading is per-family (libgomp). All pytest checks pass on both platforms.

Skip native, run on NumPy only

```
pip install -e .                               # no PyTorch, no pytest extras
RECOUNT_NO_AUTOBUILD=1 python3 -c "import recount"
```

The NumPy backend is ~150× slower than native on arc269 but is the reference implementation everything else is tested against.

Quick start — the recount CLI

`pip install -e` (above) puts a `recount` command on your PATH; run `recount -h` for its subcommands. From a checkout you have not installed, use `python3 -m recount.cli` wherever `recount` appears below.

Data reaches the tool in one of two forms:

- an existing Count session XML, the format Csűrös' Java Count reads and writes (`.countxml`, `.countxml.gz`, or the same XML content under a local name such as `count.xml`);
- a plain Newick tree plus a family-by-taxon count table (`.csv`, `.tsv`, `.txt`, with optional `.gz` compression).

The examples below use the Williams 2017 archaeal dataset bundled with the repo, but the commands are intended to be literal templates for new jobs.

Existing Count users: start from CountXML

If you already have a Count session file, no conversion is needed. The session usually contains a tree, a fitted model, and one or more count tables. Use `--table` when the XML carries multiple tables.

Run likelihood checks on a previous Count job:

```
recount ll validation/Williams2017.countxml.gz \
  --table wsz60-aletrim-min4.txt \
  --min-copies 4 \
  --num-threads 8
```

Report native per-branch copy, gain, and loss posterior expectations from the same fitted CountXML:

```
recount events validation/Williams2017.countxml.gz \
  --table wsz60-aletrim-min4.txt \
  --num-threads 8 \
  --out-json /tmp/williams_events.json
```

Refit GLD rates from a CountXML table:

```
recount fit validation/Williams2017.countxml.gz \
  --table wsz60-alefamilies-min1.txt
```

The Williams bundle carries two count tables; `--table` selects the complete 31,293-family set (the table actually used is echoed on stderr; `recount fit -h` lists `--session`, `--table` and the fit options). `fit` writes the fitted per-node rate table — length, gain, loss and dup for every node — to stdout; add `--out-json rates.json` to also save it as JSON.

Tree plus table users: produce a CountXML

If you have a species tree and a count matrix but no CountXML yet, use `recount analyze`. It is the one-shot path: read the Newick and table, fit rates with the native backend, compute posterior summaries, and write a Java-readable CountXML alongside CSV reports.

Input tree:

- Newick file (`.nwk`, `.tre`, `.newick`) or `-` for stdin.
- Leaf labels must match the table's taxon column headers.
- Internal node labels are preserved when present.

Input table:

- First column: family identifier.
- Remaining columns: taxon names with integer copy counts.
- Comma- or tab-separated files are detected automatically.
- `.gz` files are read directly.
- Leaf columns may be in any order; `recount` reorders them to the tree.
- Columns whose headers are not tree leaves are ignored.
- Missing tree taxa default to zero; blank cells and `?` are treated as zero.

Minimal generic example:

```
Family,Species_A,Species_B,Species_C
fam0001,1,0,2
fam0002,0,3,0
```

Run the one-shot pipeline:

```
recount analyze examples/data/williams_tree.nwk \
  examples/data/williams_table.csv.gz \
  --out-prefix /tmp/williams
```

`analyze` reads the observation filter off the table: every Williams family here has at least 4 total copies, so it conditions the likelihood at `--min-copies 4` automatically (pass `--min-copies` to override). It writes three files:

- `/tmp/williams.countxml.gz` — Java-compatible session XML, carrying the fitted rates;
- `/tmp/williams.branches.csv` — one row per node: rates, gain/loss events, ancestral copy counts, and observed + unobserved-corrected family presence;
- `/tmp/williams.families.csv` — per-family, per-node `E_copies` (expected copy count) and `P_present` (presence probability).

If the CountXML is the only output you need, use the `<out-prefix>.countxml.gz` file and ignore the two CSVs. That file can be loaded again by `recount ll`, `recount events`, or Java Count.

Common table and filtering cases

Use `--min-copies` to match the observation filter in the input table. For unfiltered family tables this is usually `--min-copies 1`, meaning empty families are unobserved. For tables already filtered to families with at least four total observed copies, use `--min-copies 4`. In `recount analyze`, the default auto-detects this from the table's minimum profile sum; pass the flag explicitly when documenting or reproducing a published job.

For quick tests on a large existing CountXML, keep the model fixed and cap the family table:

```
recount ll previous.countxml.gz \
  --table families.tsv \
  --min-copies 4 \
  --max-families 1000
```

For parallel batches, either pass `--num-threads N` to native-backed CLI commands or set `RECOUNT_NUM_THREADS=N` in the environment.

The canonical bounded-fit campaign

All headline numbers in this README come from a **single canonical recipe** applied uniformly to every focal dataset:

```
optimizer      native_bfgs
cycle iters    100
max cycles     12      (5 for the Brownian-extend ML pass - see below)
seed           2025
clip           |log_rate| 33  (Csurös' MAX_GAIN_RATE)
sub-critical   dup_v    MAX_PROB_NOT1 = 1 - 23  (logit transform; see $[constraint regime](#constraint-r
```

Four fit modes per dataset:

Table 1. The four canonical fit modes, with driver script and output directory for each.

mode	description	script	output dir
1 cold ML BOUNDED	bare-bones likelihood, 1 random init	validation/ml.py	bounded_csuros/
2 Brownian-extend ML (default)	warm-start from mode 4's Brownian-MAP rates, drop the prior , short pure-ML fine-tune (5 cycles)	validation/brownian_extend_ml.py	brownian_extend_<ds>/
3 MAP =1 cold	independent log-Normal(=1) prior on every per-node log-rate	validation/map.py	map_signal/
4 MAP Brownian =1 (recommended baseline)	tree-Brownian autocorrelated log-rate prior (per-edge Gaussian on log-rate increments)	validation/map.py --prior brownian	brownian_<ds>/

The full recipe + per-dataset wall estimates + the `MODES=...` / `NUM_THREADS=...` knobs are in [validation/QUEUE_INSTRUCTIONS.md](#). One-liner:

```
NUM_THREADS=16 bash validation/queue_full_subset_run.sh
```

(arc269's expensive modes 1, 3, 4 are skipped automatically when the canonical-settings outputs already exist; only the missing mode 2 runs.)

Default optimization recommendation

For new datasets, the default recommendation is the **two-step Brownian-then-ML pipeline** (modes 4 → 2):

1. **Mode 4 — MAP Brownian =1.** The tree-structured prior on log-rate increments anchors the optimizer in a biologically tight basin (subsets land at $\text{cpf} \ 1.1\text{--}1.3$, vs $1.7+$ for cold MAP). The prior trades a small amount of data-fit for orders-of-magnitude better identifiability on the per-node rates: boundary blow-ups are absent (max dup < 1, max gain 10^2 across all subsets vs Csurös' published 5×10^1).
2. **Mode 2 — Brownian-extend ML.** Once the Brownian-MAP basin is located, drop the prior and run a short pure-ML pass (5 cycles). The rates relax toward the data-likelihood peak without diverging from the basin — typical improvement is a few hundred nat (see the **Brown-extend ML** (Δ) column in the LL table, Table 4) with cp/fam staying close to the Brownian-MAP value.

This pipeline is faster than the historical 3-start multistart-polish recipe (~25 min for the 5-mode-minus-15-start variant vs ~2 h across 6 datasets at 16 threads) and lands in a more biologically interpretable basin.

Demonstration of reproduction

Everything downstream rests on first reproducing Csurös’ published GLD analysis. The four tables below (Tables 2–5: root families, copies-per-family, log-likelihood, boundary diagnostics) report what each fit produces at the subset root, compared with Csurös’ bundled rates from Csurös 2026 PNAS. The two recommended modes (MAP Brownian =1 and Brown-extend ML, bold column headers) match Csurös’ log-likelihood to within a few nat on every dataset — so the analysis in **Does GLD give a sound foundation for ancestral genome inference?** is an analysis of the published fits, not of a different one.

Root families across 4 fit modes Posterior count of families present at the root. Each cell shows raw / L(0)-corrected: raw = posterior over observed families (*_observed in branches.csv); corrected adds the Csurös $F^* = F/(1-L(0))$ amplification for families that went entirely extinct before sampling. For $\Omega_{min}=4$ fits the corrected number is Csurös’ Pn-tensor amplification bit-for-bit ($L(0) = 0.03-0.07$, so the gap is small); for arc269 mc=1 the corrected code path is gated off, so raw=corrected here — the proper $L(0)=0.79$ amplification is in the **filter sweep table** (Table 11) of section 3.

Bold column headers flag the two recommended modes (MAP Brownian, Brown-extend ML). The Csurös-ref column is Csurös’ published rates, not a recount fit.

Table 2. Posterior root family count (raw / L(0)-corrected) across the four fit modes and Csurös’ reference rates.

Dataset (F, leaves, Ω_{min})	Cold ML (raw / corr)	MAP =1 cold (raw / corr)	MAP Brownian =1 (raw / corr)	Brown-extend ML (raw / corr)	Csurös ref (raw / corr)
williams (5,378, 60, 4)	1,341 / 1,361	1,260 / 1,275	1,047 / 1,054	1,164 / 1,175	1,092 / 1,092
dpann80 (3,034, 80, 4)	925 / 935	865 / 870	743 / 745	833 / 839	1,192 / 1,210
proteo75 (5,179, 75, 4)	1,459 / 1,461	1,575 / 1,579	1,742 / 1,750	1,742 / 1,750	2,062 / 2,096
eury114 (7,335, 114, 4)	1,530 / 1,533	1,441 / 1,459	1,402 / 1,413	1,249 / 1,255	1,441 / 1,453
ed194 (8,855, 194, 4)	1,709 / 1,710	1,480 / 1,497	1,509 / 1,522	1,554 / 1,576	1,399 / 1,419
arc269 (90,243, 269, 1)	4,055 / 4,055	3,693 / 3,693	3,847 / 3,847	3,849 / 3,849	— not fit by Csurös —

Copies-per-family at the root (cp/fm) raw / corr. L(0) appears in both numerator and denominator and almost cancels in the ratio: at $L(0) = 0.03-0.07$ raw and corrected agree to within ~ 0.01 (rounded together at 2 dp when identical).

Table 3. Copies-per-family at the root (raw / corrected) across the four fit modes and Csurös’ reference.

Dataset	Cold ML	MAP =1 cold	MAP Brownian =1	Brown-extend ML	Csurös ref
williams	1.28 / 1.28	1.30 / 1.29	1.25 / 1.25	1.28 / 1.28	1.25 / 1.25
dpann80	1.16 / 1.16	1.12 / 1.12	1.12 / 1.12	1.14 / 1.14	1.16 / 1.15
proteo75	1.31 / 1.31	1.33 / 1.33	1.35 / 1.35	1.35 / 1.35	1.62 / 1.61
eury114	1.16 / 1.16	1.15 / 1.15	1.15 / 1.15	1.14 / 1.14	1.20 / 1.20
ed194	1.26 / 1.26	1.22 / 1.22	1.22 / 1.22	1.24 / 1.24	1.20 / 1.19
arc269	1.74 / 1.74	1.71 / 1.71	1.70 / 1.70	1.69 / 1.69	—

LL across fit modes (Δ = ours — Csurös bundled rates) **Table 4.** Log-likelihood per fit mode, with Δ from Csurös’ bundled-rate reference.

Dataset	Csurös ref LL	Cold ML (Δ)	MAP =1 cold (Δ)	MAP Brownian =1 (Δ)	Brown-extend ML (Δ)
williams	−117,694.84	−117,103 (+592)	−117,439 (+256)	−117,872 (−178)	−117,695 (−0)
dpann80	−94,147	−94,178 (−31)	−94,339 (−192)	−94,258 (−111)	−94,147 (+0)
proteo75	−154,159	−154,692 (−533)	−155,487 (−1,328)	−154,494 (−335)	−154,494 (−335)
eury114	−274,643	−274,817 (−174)	−274,871 (−228)	−274,896 (−253)	−274,647 (−4)
ed194	−395,601	−397,028 (−1,427)	−395,970 (−369)	−395,978 (−377)	−395,604 (−3)
arc269	— not fit —	−1,131,918	−1,128,902	−1,129,009	−1,128,439

Boundary diagnostics — max dup, max gain per fit max dup / max gain across all non-root nodes. dup asymptotes to MAX_PROB_NOT1 = 1 but never exceeds it; cells reading 1.0000 / ... mean the optimizer parked at the cap on at least one node.

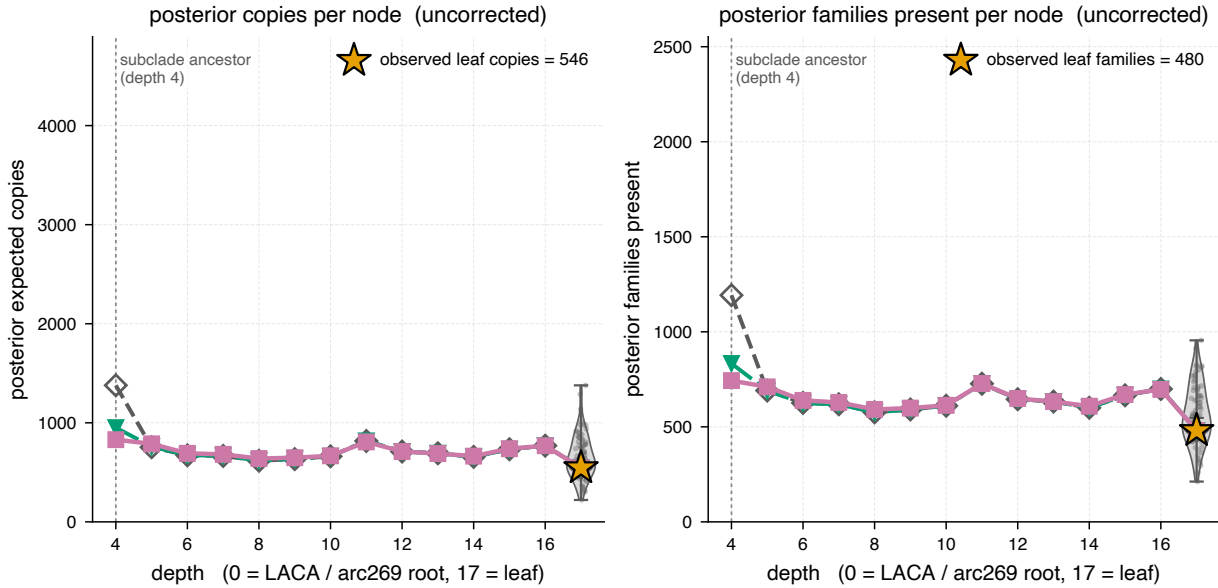
Table 5. Boundary diagnostics — max duplication and max gain across non-root nodes, per fit mode.

Dataset	Csurös ref	Cold ML	MAP =1 cold	MAP Brownian	Brown-extend ML
williams	1.0000 / 1.2×10^1	1.0000 / 104	0.999 / 4.60	0.981 / 0.99	1.0000 / 162
dpann80	1.0000 / 4.9×10^{13}	1.0000 / 3.1×10^3	0.669 / 2.74	0.673 / 3.83	0.999 / 786
proteo75	1.0000 / 5.2×10^1	1.0000 / 2.8×10^1	1.0000 / 17	1.0000 / 222	1.0000 / 162
eury114	1.0000 / 5.8×10^{12}	1.0000 / 1.2×10	0.974 / 0.97	0.995 / 1.02	1.0000 / 158
ed194	1.0000 / 1.3×10^1	1.0000 / 703	0.975 / 0.89	0.995 / 1.42	0.9999 / 99
arc269	— <i>not fit</i> —	1.0000 / 1.1×10^{11}	0.968 / 0.14	0.986 / 0.12	0.9998 / 2.21

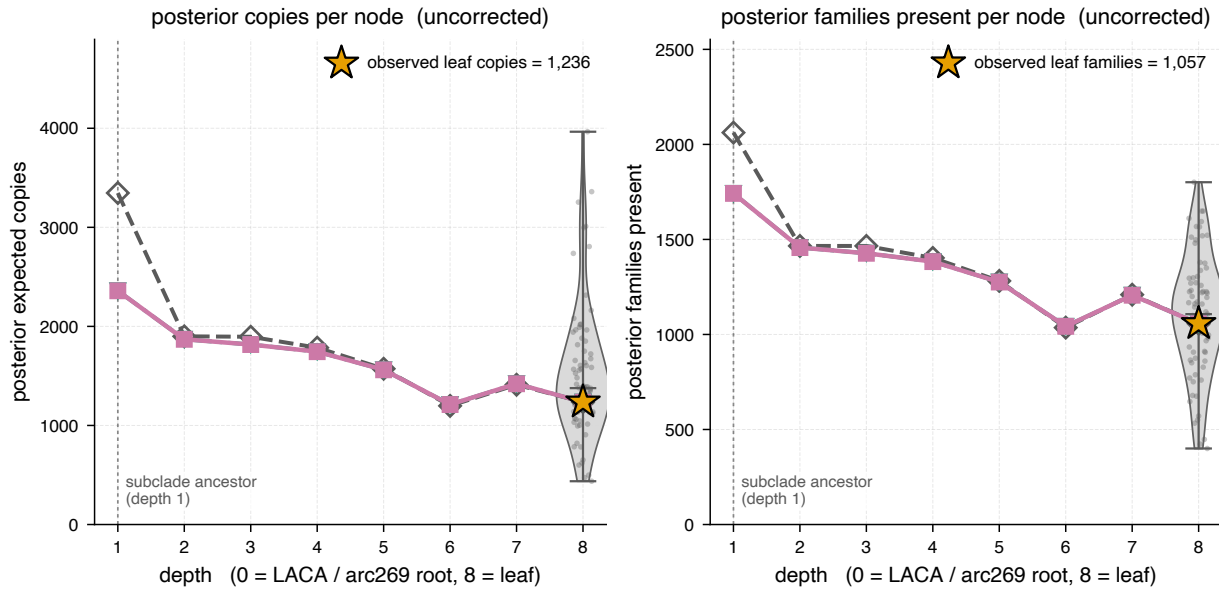
The recommended modes (MAP Brownian, Brown-extend ML) keep max gain 10^3 everywhere; Csurös' subset-only ML hits 5×10^1 on proteo75 (a known consequence of unconstrained ML on copy-count data with sparse outlier families on long branches). Section **Why a prior is more general** discusses why the Brownian prior tames these blow-ups without needing the hard sub-critical cap to be active.

Per-subset trajectory plot: arc269-root → median-fm leaf The path plots below (Figure 1) trace each fit's posterior cp and fm from the arc269 root (LACA, leftmost) to the subset's *median-fm* leaf (rightmost), for the three reproduction-relevant fits — Csurös' subset-only ML, our MAP Brownian $\sigma = 1$, and our Brown-extend ML. As with the diagnostic Sets, the line styles and the gold-star / khaki-violin convention are carried by the shared legend below; the grey vertical line in each panel marks the subclade's LCA depth, at which the subset fits begin.

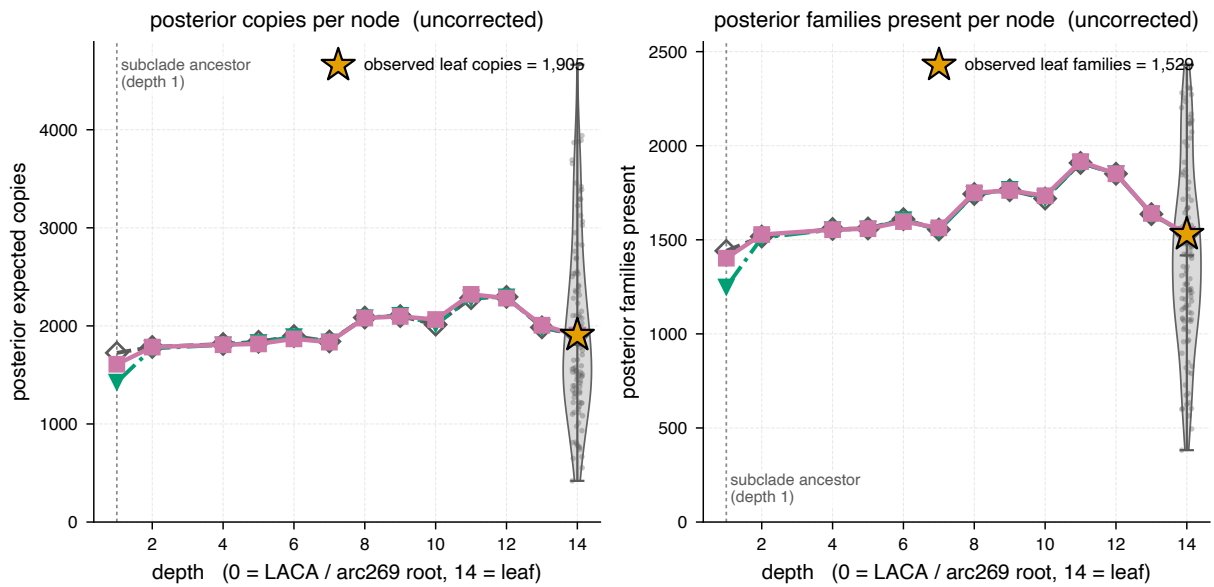
DPANN (D80) · reproduction of Csurös' min = 4 fit · arc269 root to median-fm leaf '1974445' (F = 3,034)



Proteoarchaea (P75) · reproduction of Csurös' min = 4 fit · arc269 root to median-fm leaf '2058097' (F = 5,179)



Methanobacteriati (E114) · reproduction of Csurös' min = 4 fit · arc269 root to median-fm leaf '679901' (F = 7,335)



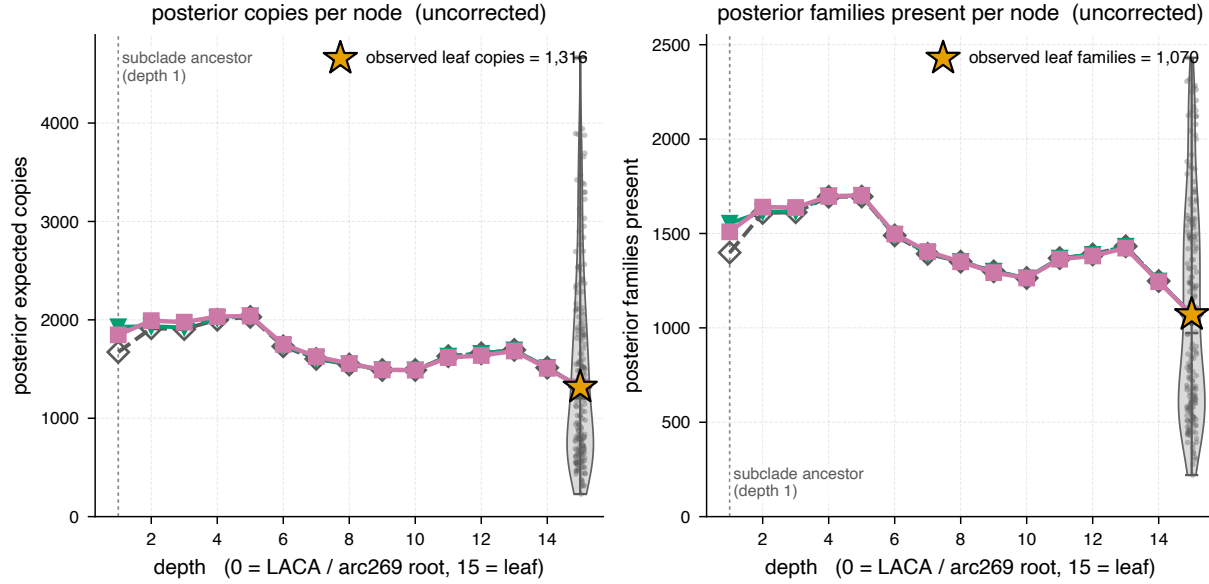


Figure 1. Per-subclade posterior reproduction trajectories from the arc269 root to each subset's median-fm leaf, for Csurös' subset-only ML and recount's two recommended fits.

Reading each panel of Figure 1: the three lines run from the arc269-root depth (left) to the median-fm leaf depth (right), and all three meet the OBSERVED gold-star value at the leaf tip on every subset. They diverge at the deeper internal nodes, where the recommended modes (purple, olive) sit close to Csurös' reference (gray dashed). The L(0)-amplified counterparts are in **L(0)-corrected trajectories**; both the panels and the shared legend are regenerated by `validation/reproduction_path_plots.py`.

Constraint regime

Two complementary regularisers shape the fit; both default to *on*.

Sub-critical Yule cap on duplication

Per-branch duplication is bounded *just below* the loss rate via the logit transform $\text{dup}_v = \text{cap} \cdot \text{sigmoid}(_v)$, where the cap $\text{MAX_PROB_NOT1} = 1 - 2^{-3} \cdot 0.99999999907$. Equivalently the Pólya shape parameter $_v \cdot (1 - q_v) > 0$ everywhere, keeping the branching process sub-critical. This matches the `is_duprate_bounded = true` default in Csurös' Java ([MLDistribution.java:157](#)) and the convention documented in **Csürös 2026 PNAS** (full citation in [Citing](#)). Without the cap the optimizer can let $\text{dup} \rightarrow \infty$ on individual branches and the likelihood runs away along a ridge of equally-good but biologically meaningless solutions; see [NO_DUPPLICATION_CONSTRAINT.md](#) for the unconstrained snapshot.

Why a prior is a more general (and arguably more biological) choice

Csurös' sub-critical cap is a **hard constraint** on a single quantity ($\text{dup} \leq 1$ per branch). It rules out the literal $\text{dup} \rightarrow \infty$ pathology but leaves the rates on neighbouring branches **statistically independent** — adjacent edges of the same clade can still take wildly different values, gain can still blow up to $\sim 10^1$ at isolated nodes ([GAIN_CONSTRAINT.md](#) walks the published-rate forensics), and the optimizer keeps finding multiple local basins.

A **prior** is a more general regulariser and is arguably better motivated biologically. The Brownian prior (mode 4 in the canonical queue) places a Gaussian on the **log-rate increment** between parent and child branches:

$$\log r_v - \log r_{\text{pa}(v)} \sim \text{Normal}(0, \sigma^2 \cdot t_v) \text{ for each rate axis } r \in \{q, t\}, \text{ each non-root } v$$

This is the **Thorne–Kishino–Painter** autocorrelated-rate model (Thorne, Kishino, Painter 1998, *Estimating the rate of evolution of the rate of molecular evolution*, MBE 15(12):1647–1657), originally proposed for molecular-clock relaxation and reused here for GLD rate variation. It captures a substantive biological assumption: **rates of gene-content evolution are auto-correlated along the tree** — sister branches that share recent ancestry

should have similar rates a priori, because the biological mechanisms governing gain/loss/duplication (genome size, lifestyle, mutational regime) themselves vary smoothly along the species tree.

The mathematical specification and the bit-perfect FD-validated gradient are in [docs/brownian_prior.pdf](#). The 2026-05-19 update for arc269 + subsets data:

- **All 5 subset Brownian fits sit inside the sub-critical region** without needing the dup-cap to be active — the prior dominates and the cap is never reached. Max gain stays bounded at 10^2 (vs Csurös’ published 5×10^1).
- **The Brownian prior subsumes the sub-critical cap as a special case:** in the $\rightarrow \infty$ limit the prior is uniform and the cap is the only thing keeping dup bounded; for finite the prior softly penalises any large rate excursion long before it reaches the cap.

The cap is kept *on by default* as a hard fallback for ML-only fits (modes 1 and 2) where no prior is active. For MAP fits (modes 3, 4) the cap is rarely binding.

Does GLD give a sound foundation for ancestral genome inference?

Csurös 2026 PNAS closes its abstract with a claim: “*The GLD framework provides a statistically sound foundation for hypothesizing about gene content evolution across the diversity of entire kingdoms.*” The **reproduction section** above confirms the first half — the GLD likelihood, its gradient and the $\Omega_{\min}$ sampling-bias correction are implemented correctly, and the recount fits recover Csurös’ published rates to within a few nat of log-likelihood. This section tests the second half: when GLD is used to reconstruct an *ancestral genome* from gene-copy-number data, is the result a sound foundation for hypotheses about how gene content evolved across a whole kingdom?

Tested on 269 archaeal genomes (arc269 — a superset that contains Csurös’ four focal subset clades as slices), the answer is no. The argument runs in three steps:

1. **GLD cannot separate horizontal transfer from inheritance.** It reads only per-genome copy counts; two families with the same counts get the same likelihood whether they were inherited from a common ancestor or spread between lineages by transfer.
2. **Unfiltered, GLD fits the family-size data well — and infers an ancestor larger than any living archaeon.** The inflation is the expected behaviour of a copy-number-only method on a kingdom with pervasive transfer; it is not a fitting failure, because the unfiltered fit is the *best* fit to the family-size distribution.
3. **The family-size filter that suppresses the inflation has no GLD-internal justification, and it makes the fit to the family-size distribution worse.**

Scope. We do not dispute the GLD likelihood or its algorithms — the reproduction section affirms both. The claim under test is the narrower, downstream one: that a GLD fit to copy-number data is a sound *foundation for ancestral gene-content inference* across a kingdom.

1 — Copy numbers cannot separate transfer from inheritance

The GLD likelihood factors over gene families. For one family it reads a single integer per genome — how many copies of that family the genome carries — and nothing else. Call the family’s vector of per-genome copy counts its *profile*, $\Xi = (\Xi_1, \dots, \Xi_N)$ over the N leaves of the tree. The likelihood $L(\Xi \mid \text{rates})$ is a function of that integer vector and the tree; no other feature of the family enters.

GLD is a likelihood model. For each family it does not choose a single history; it sums over every history its model permits — every arrangement of gains, losses and duplications along the species-tree branches — weighting each by its probability under the fitted rates. The posterior it reports for “family present at node X” is the rate-weighted share of those histories in which the family is present at X.

Two consequences follow. First, that posterior is a function of the copy-number profile, the tree and the fitted rates, and of nothing else: two families with identical profiles receive identical posteriors, necessarily. Second, GLD’s space of histories contains only vertical events — gain, loss, duplication along the tree. It has no horizontal-transfer edge.

Now take a family found in clade A and clade C but not in the clade B that lies between them. Its true history might be vertical — present in the A–B–C common ancestor, then lost on the lineage to B — or horizontal — arisen within A, carried into C by transfer, never in the A–B–C ancestor. The two produce the same copy-number

profile. GLD, scoring that profile, has no transfer history available. It accounts for the profile with the gain-loss-duplication histories its model does permit: a single gain in the A–B–C ancestor followed by a loss in B, or two independent gains — one in A, one in C — with no copy ever in the A–B–C ancestor. Only the first is a vertical history; the second is not, but neither is it a transfer — GLD’s model has no edge that carries a copy from one lineage to another. The relative weight of these histories is set by the fitted rates: a rare gain rate with a high loss rate favours the single-ancestral-gain history, the reverse favours the independent-gain history. This is a likelihood, not a parsimony reconstruction: GLD does not pick the ancestral history, it integrates over all of them. But whatever ancestral posterior the integration yields, GLD returns the *same* value for the genuinely ancestral family and for the transferred one — their profiles are identical, and the profile is the whole input.

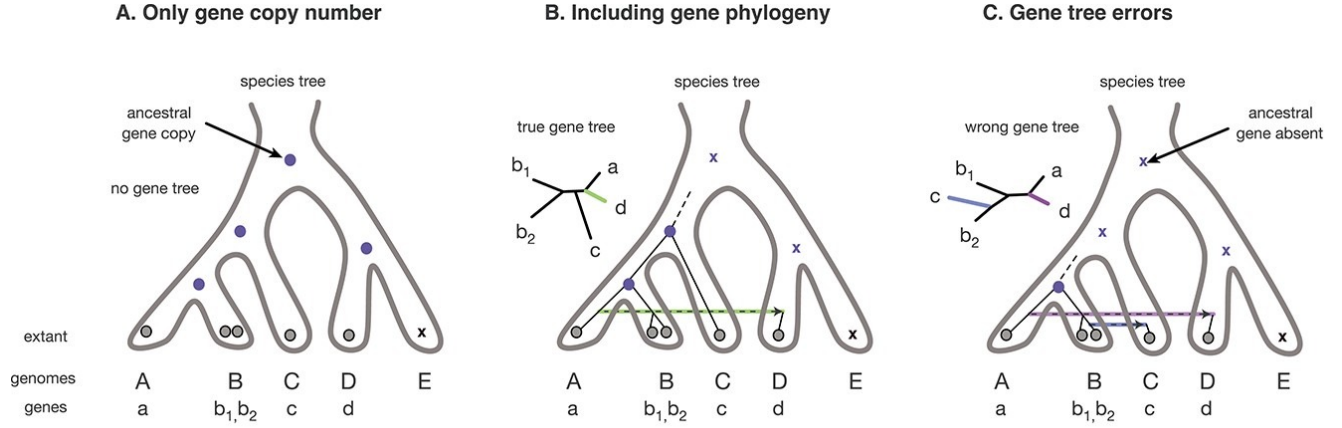


Figure 2. A sampled gene-tree topology distinguishes a horizontally transferred gene family from a vertically inherited one — the distinction a copy-number profile alone cannot make. Reproduced under fair use from T. A. Williams, A. A. Davin, L. L. Szánthó et al., “Phylogenetic reconciliation: making the most of genomes to understand microbial ecology and evolution,” *The ISME Journal* 18:wrae129 (2024).

This is a structural property of copy-number-only inference, and it is the reason gene-tree-reconciliation methods exist: a sampled gene-tree topology *does* carry information that separates a transferred family from an inherited one, and reconciliation methods use it (Williams, Davin, Szánthó et al. 2024, Fig 2, reproduced above as Figure 2). GLD does not consume gene trees, so it cannot make that separation.

A worked example A small tree that shows the problem has five genomes, related as (((A,B),C),(D,E)). One gene family is present in A (one copy), B (two copies), C (one) and D (one), and absent in E. The unrooted gene tree of the five gene copies is ((a,d),(b₁,b₂),c): the copy in A and the copy in D are each other’s closest relatives. A and D are distant in the species tree — A inside the ABC clade, D in the DE clade — so the a–d pairing is discordant with the species tree: the transfer signature Figure 2 illustrates. Reading that gene tree, reconciliation recovers the family’s true history — it originates on the branch leading to the ABC ancestor, one copy is transferred from the A lineage to D, and one copy duplicates on the terminal branch to B. The family is genuinely present at A, AB, ABC and D; the root (the ABCDE ancestor), the DE ancestor and E are empty of it.

GLD sees none of the gene tree; it has only the copy-number profile [A=1, B=2, C=1, D=1, E=0]. Reconstructed in recount with global rates and unit branch lengths, GLD’s posterior probability that the family is present (1 copy) at each node along the two root-ward trajectories — D → DE → root and A → AB → ABC → root — is a function of the global gain rate. The plots below (Figure 3) trace it at two fixed loss rates (loss is the GLD time unit; the duplication rate is held at 0.2). The dotted black line is the true history — present at A, AB, ABC and D, absent at DE and the root.

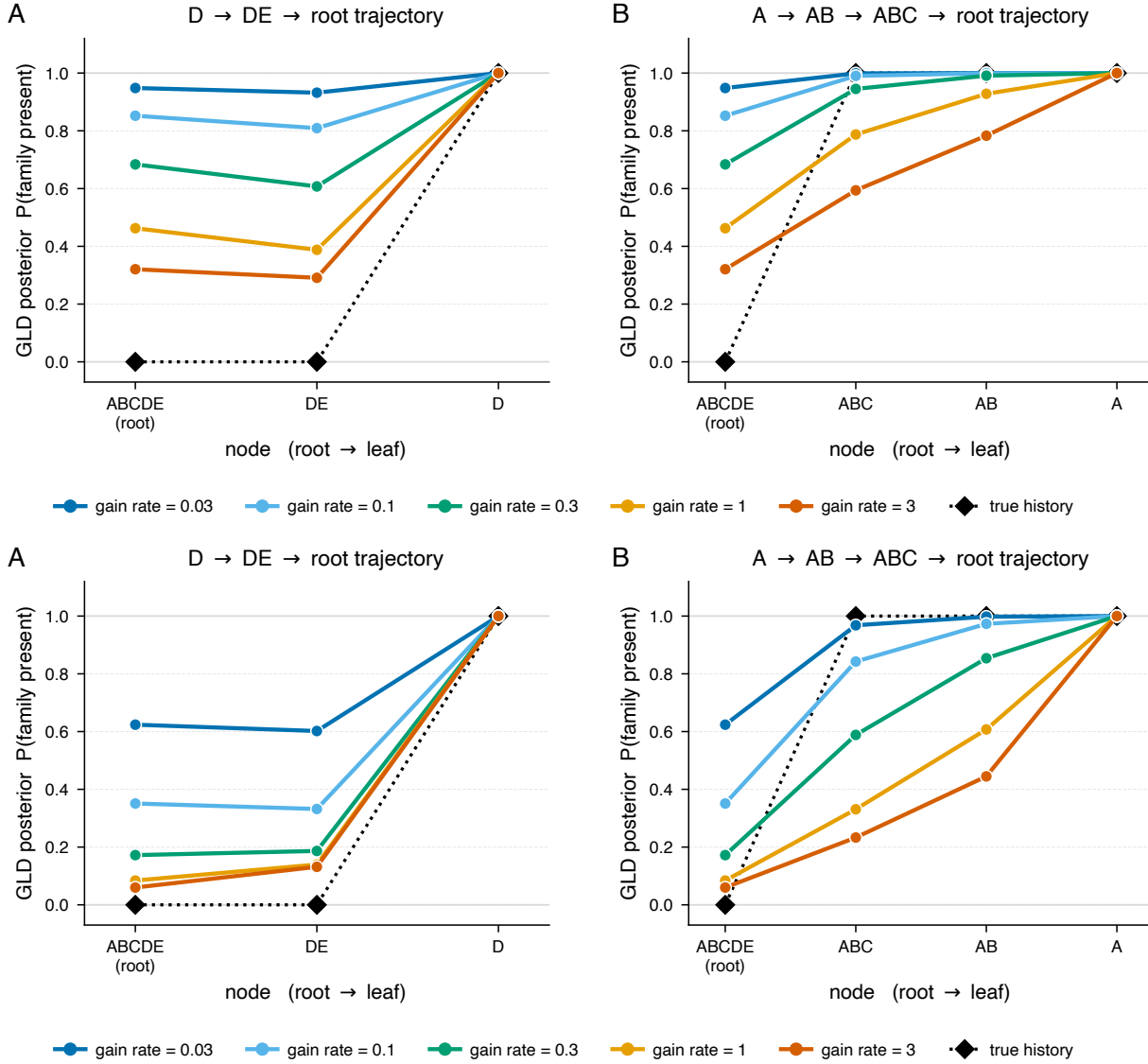


Figure 3. GLD presence posterior along the two root-ward trajectories of the worked example, at loss rates 1 and 2, one curve per global gain rate, with the true history dotted.

Read each trajectory from the root (left) to the leaf (right). On D → DE → root the true history places the family only at the leaf D, yet GLD lifts DE and the root well above the dotted zero line — at loss rate 1, DE to 0.29–0.93 and the root to 0.32–0.95 over the gain rates shown, lower at loss rate 2 but still positive. D’s copy, a transfer, is read by the copy-number likelihood as evidence for the family deep in the tree. On A → AB → ABC → root GLD keeps the family through ABC, the clade it truly arose in, then carries it on to the root — empty in the true history — by the same amount. The spurious ancestral mass shrinks as the gain rate rises (independent gains become cheap) and as the loss rate rises, but it never reaches zero, the true value.

Which positive number GLD reports is set by the rates — and the rates are not a per-family quantity. GLD estimates one set of rates jointly from the entire dataset, and those dataset-level rates then shape every individual family’s reconstructed history. That much it shares with phylogenetic reconciliation methods, which likewise estimate their parameters across the whole dataset and apply them family by family. The difference is the information each family contributes: a reconciliation method also reads that family’s gene tree, recovered from its sequence alignment, and the alignment carries the signal that separates a transfer from an inheritance. GLD reads only the family’s copy numbers, which do not carry that signal. The trajectory computations are reproduced by [validation/transfer_id entifiability_example.py](#).

This biases ancestral inference in a definite direction. A copy acquired by transfer sits in a clade the family never reached by descent; GLD, having no transfer edge, can join it to the rest of the family only through a common

ancestor in the species tree — a node deeper than the family’s true origin. In the worked example the family arose in the ABC clade, but its transferred copy in D forces GLD to seat the family at the root, the common ancestor of the ABC clade and D, where the family never was. Its correct contribution to that root is zero; GLD gives it a positive one — the posterior its profile earns, the same a genuinely root-ancestral family with that profile would get. Each cross-clade transferred family therefore adds spurious mass to the ancestral count, and the bias has a sign: transfer can only push the inferred ancestor *upward*. Archaea, like other prokaryotes, exchange genes by horizontal transfer extensively — resolving transfer is the problem the reconciliation literature is built around — so on archaeal data this upward bias is not an edge case. Its magnitude cannot be read off the copy numbers, because the copy numbers cannot say which families were transferred; section 2 measures the inferred ancestor that results.

2 — Unfiltered GLD: a good fit to family sizes, an ancestor larger than any living archaeon

This section makes two observations about the unfiltered fit (every family with at least one copy somewhere; no family-size filter). They pull in opposite directions, and that tension is the point:

- the unfiltered fit **reproduces the empirical distribution of family sizes** — it is the only fit in the whole filter sweep that does;
- the same fit **infers an ancestor with more gene families than the richest modern archaeal genome in the data** carries.

The unfiltered fit is not a failed fit that a better optimiser would cure: it already reproduces the family-size distribution (below). The inflation is therefore not an optimisation artefact — it is intrinsic to reconstructing ancestry from copy numbers (section 1).

What the GLD likelihood predicts about family sizes Under the fitted GLD birth-death process every family has a random profile Ξ ; let $\Omega(\Xi) = \sum_v \Xi_v$ be its *total copy count* summed over all leaves. The process induces a distribution over Ω . Define

$$L(0, N) = P(\Omega(\Xi) < N \mid \text{fitted } \lambda, \mu, \tau)$$

— the model’s own predicted fraction of families whose total copy count falls below N . (Csurős writes this $L(0)$; it is the sampling-bias term — the mass the rate process puts on families too small to be seen once a sum $\Omega_{\min}$ filter is applied.)

The numerical value of $L(0, N)$ is tied to the evaluation universe: the same fitted rates can give different $L(0, N)$ values when Ω is computed over the fit’s own training leaves/filter universe versus over the full arc269 leaf set used for the histogram check below.

Two quantities follow:

- the per-bin probability $P(\Omega = k) = L(0, k+1) - L(0, k)$;
- the implied total family count $F^* = F / (1 - L(0, \Omega_{\min}))$, where F is the number of families the fit actually sees — the Csurős extinction-amplification that scales observed ancestral counts back up for families that left no trace.

If the fitted rates describe the family-size distribution over the evaluation universe, the predicted counts $F^* \cdot P(\Omega = k)$ should match the empirical number of families with total copy count k — both above the filter, where the fit was trained, and below it, where the underlying profile still records the small families the filter discarded. The next subsection runs exactly that check.

This comparison is a posterior-predictive check, written in closed form. The identity $F^* \cdot (1 - L(0, \Omega_{\min})) = F$ is only the normalisation implied by the observation filter; it is not the test. The test is the bin-wise comparison $F^* \cdot P(\Omega = k)$ versus the empirical histogram. $F^* \cdot P(\Omega = k)$ is the expected count of families with total copy count k that one would obtain by simulating F^* families from the fitted GLD process and tallying them by size; the Pn-tensor recursion (Csurős SI Theorems 3–5) returns that expectation directly, so the test carries no Monte-Carlo noise. The question it asks is the standard one for any fitted generative model: draw gene families from it, and do you recover the family-size distribution that was actually observed? (It is a *plug-in* predictive check — it uses the point-estimate rates; for fits over tens of thousands of families the rate uncertainty is negligible next to the discrepancies seen below.)

As a sanity check on the $L(0)$ bin calculation itself, [validation/pnas_l0_family_size_sim.py](#) forward-simulates raw families from the two rate sets used in the PNAS letter figure and compares the simulated Ω bins with

$L(0,k+1)-L(0,k)$. With 250,000 raw simulated families per fit, the $\Omega = 2$ conditional simulated/predicted ratios are 0.972–1.025 for the unfiltered fit and 0.987–1.009 for the $\Omega_{\min}=4$ fit (JSON; ratio plot; log-log plot).

Goodness-of-fit: empirical vs model-predicted family-size histogram Empirical arc269 sum-bin histogram (full $F = 90,243$). Every family has total copy count $\Omega \geq 1$: a family with $\Omega = 0$ is present in no genome, leaves no trace, and so is unobservable in principle — the $\Omega = 0$ bin is empty in any dataset, not only this one. The rate process, by contrast, does assign $\Omega = 0$ a probability (the sampling-bias term $L(0, 1)$); that predicted-but-unobservable mass is what the goodness-of-fit test below has to handle separately from the observed bins:

Table 6. Empirical arc269 sum-bin family-size histogram over the full $F = 90,243$ families.

sum k	empirical count	% of arc269
1 (singleton)	65,339	72.40
2	9,491	10.52
3	3,859	4.28
4	2,062	2.28
5	1,176	1.30
6	851	0.94
7	644	0.71
8	6,821	7.57

Unconditional test: predicted $F^* \cdot P(\Omega = k)$ divided by empirical $E(k)$. 1 = perfect; < 1 = model under-predicts; > 1 = over-predicts. Rows are sequential arc269 fits at increasing filter strength.

Table 7. Unconditional goodness-of-fit — predicted/empirical family-size ratio per sum bin, one row per arc269 fit.

fit (filter, $L(0, \Omega_{\min})$)	k=1	k=2	k=3	k=4	k=5	k=6	k=7	sum	8
mc=1 no filter ($L(0,1) = 0.79$)	0.94	1.29	1.11	1.03	1.09	1.06	1.08	1.08	
sum 2 ($L(0,2) = 0.40$)	0.10	0.46	0.84	1.17	1.56	1.68	1.80	1.54	
sum 3 ($L(0,3) = 0.27$)	0.03	0.17	0.35	0.57	0.86	1.04	1.22	1.50	
sum 4 (Csurös) ($L(0,4) = 0.21$)	0.01	0.09	0.19	0.32	0.51	0.65	0.79	1.35	
sum 5 ($L(0,5) = 0.18$)	0.01	0.05	0.13	0.21	0.34	0.45	0.56	1.22	
sum 6 ($L(0,6) = 0.17$)	0.00	0.04	0.09	0.16	0.26	0.34	0.43	1.14	
sum 7 ($L(0,7) = 0.16$)	0.00	0.02	0.07	0.12	0.21	0.28	0.35	1.06	
sum 8 ($L(0,8) = 0.16$)	0.00	0.02	0.05	0.10	0.17	0.23	0.29	1.00	
sum 9 ($L(0,9) = 0.15$)	0.00	0.01	0.04	0.08	0.14	0.19	0.25	0.95	
sum 10 ($L(0,10) = 0.15$)	0.00	0.01	0.03	0.06	0.11	0.16	0.21	0.90	
union-min4 ($L(0,4) = 0.21$)	0.01	0.08	0.18	0.29	0.47	0.60	0.74	1.32	

(The $L(0, \Omega_{\min})$ values in this table are computed with `unobserved_logL0_native` over the full 269-leaf arc269 evaluation universe. They are therefore goodness-of-fit quantities for the full arc269 sum-bin histogram, not necessarily the same $L(0, \Omega_{\min})$ values used internally by a fit under its own training filter. For example, the union-min4 fit reports a much smaller training-filter correction for its own union family set, whereas its full-arc269 histogram check gives $L(0,4) = 0.21$.)

Two objections to the unconditional comparison, and what the conditional tests handle. The unconditional ratio mixes two issues. First, the unfiltered fit’s apparent advantage on the full histogram is partly tautological — the $mc = 1$ likelihood was trained on every bin including the singletons, so it has a built-in advantage there over any filtered fit that never saw $k < \Omega_{\min}$. Second, the singleton stratum itself ($\Omega = 1$, 72 % of arc269) is the bin a birth-death model is most open to doubt on: a gene called in a single genome may be a very recent arrival, or an annotation artefact, rather than a family with the kind of gain / loss / duplication history GLD describes. A fair reading might be that the filtered fits fail only because of one or both of these effects, and would be fine on the families that matter.

The two conditional tests below remove both objections at once. Each re-runs the comparison with the smallest families dropped from both the model and the data — singletons for the $\Omega = 2$ test, singletons plus 2-copy families for the $\Omega = 3$ test — and re-normalises the model’s predicted distribution to the same range, so predicted and

empirical mass are conserved on the bins both fits speak to. On those bins neither fit has a training-set advantage, and a shape divergence is a genuine fit-quality difference rather than an artefact of which strata each fit was trained on.

Conditioned on Ω_2 (singletons excluded):

Table 8. Conditional goodness-of-fit with singletons excluded — predicted/empirical ratio per sum bin, re-normalised over Ω_2 .

fit (filter, $L(0, \Omega_{\min})$)	k=2	k=3	k=4	k=5	k=6	k=7	sum 8
mc=1 no filter	1.11	0.96	0.89	0.94	0.91	0.93	0.93
sum 2	0.46	0.84	1.17	1.56	1.68	1.80	1.54
sum 3	0.25	0.51	0.83	1.26	1.52	1.78	2.19
sum 4 (Csurös)	0.17	0.37	0.61	0.97	1.23	1.51	2.56
sum 5	0.12	0.28	0.49	0.79	1.01	1.27	2.79
sum 6	0.09	0.23	0.41	0.68	0.88	1.11	2.94
sum 7	0.07	0.19	0.35	0.60	0.79	1.00	3.05
sum 8	0.05	0.15	0.30	0.53	0.71	0.91	3.13
sum 9	0.04	0.13	0.26	0.47	0.65	0.84	3.20
sum 10	0.03	0.10	0.22	0.41	0.58	0.76	3.27
union-min4	0.16	0.35	0.58	0.93	1.18	1.46	2.61

Conditioned on Ω_3 (1+2-copy families excluded as well):

Table 9. Conditional goodness-of-fit with 1- and 2-copy families excluded — predicted/empirical ratio per sum bin, re-normalised over Ω_3 .

fit (filter, $L(0, \Omega_{\min})$)	k=3	k=4	k=5	k=6	k=7	sum 8
mc=1 no filter	1.03	0.95	1.01	0.98	0.99	1.00
sum 2	0.63	0.88	1.17	1.26	1.35	1.15
sum 3	0.35	0.57	0.86	1.04	1.22	1.50
sum 4 (Csurös)	0.24	0.40	0.64	0.81	1.00	1.70
sum 5	0.18	0.31	0.51	0.66	0.82	1.81
sum 6	0.15	0.27	0.44	0.57	0.71	1.88
sum 7	0.12	0.22	0.38	0.50	0.63	1.94
sum 8	0.10	0.19	0.34	0.45	0.57	1.98
sum 9	0.08	0.16	0.30	0.41	0.52	2.01
sum 10	0.06	0.14	0.26	0.36	0.48	2.05
union-min4	0.23	0.38	0.61	0.78	0.96	1.72

Visual summary (five representative fits, three conditioning modes side-by-side; log-y; horizontal dashed line at ratio = 1; green band marks “within $2\times$ ” of perfect):

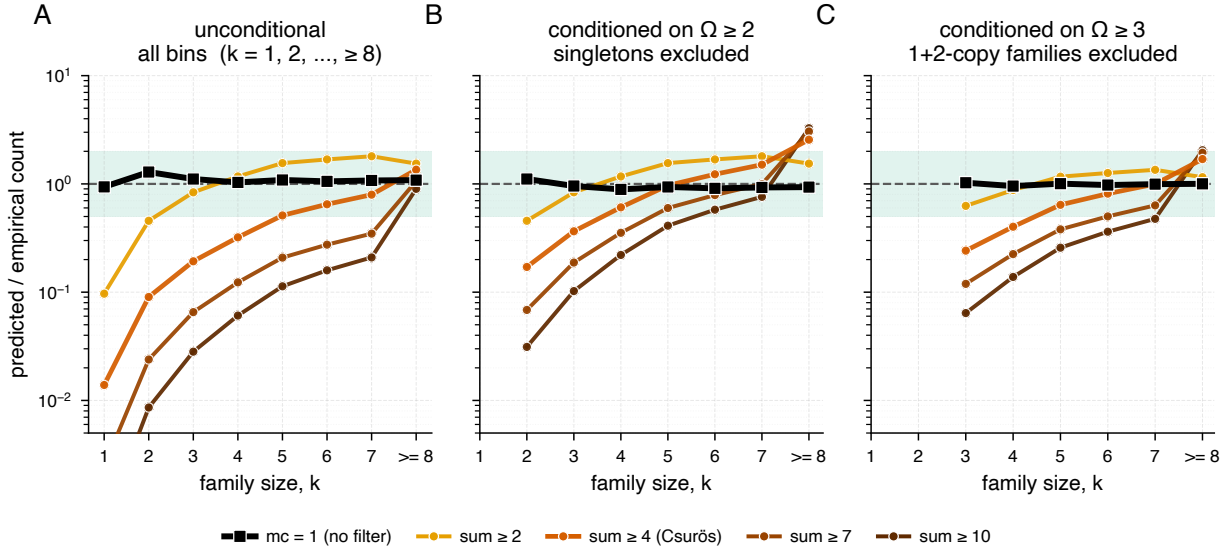


Figure 4. $L(0)$ goodness-of-fit — predicted/empirical family-size ratio vs sum bin for five representative fits across three conditioning modes.

Generated by `validation/10_gof_plot.py` from the same fits used by Tables 7–9. The $mc=1$ line (black squares) hugs the perfect-fit line in every panel; the filtered fits (warm-color gradient on filter strength) start far below 1 at low k and cross above 1 at high k — shape mismatch persists across all three conditioning modes.

Focused view: $mc = 1$ reference vs canonical $\min = 4$ fits — predicted $F^* \cdot P(\Omega = k)$ vs empirical $E(k)$ absolute counts at every sum bin. In Figure 5, panel A is the $mc = 1$ reference (the fit that matches the data); panels B and C are the canonical $\Omega_{\min} = 4$ fits (Csurös’ full-tree filter and the union-min4 filter):

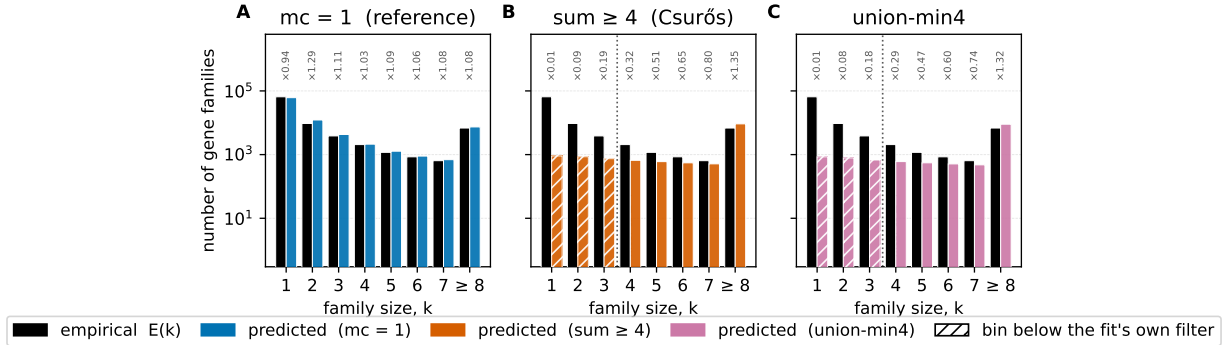


Figure 5. Focused $\min = 4$ goodness-of-fit — predicted vs empirical family counts per sum bin for the $mc = 1$ reference and the two canonical $\Omega_{\min} = 4$ fits.

The same three panels with the bin axis extended to $k = 30$, so the long-tail behaviour is visible family-size by family-size:

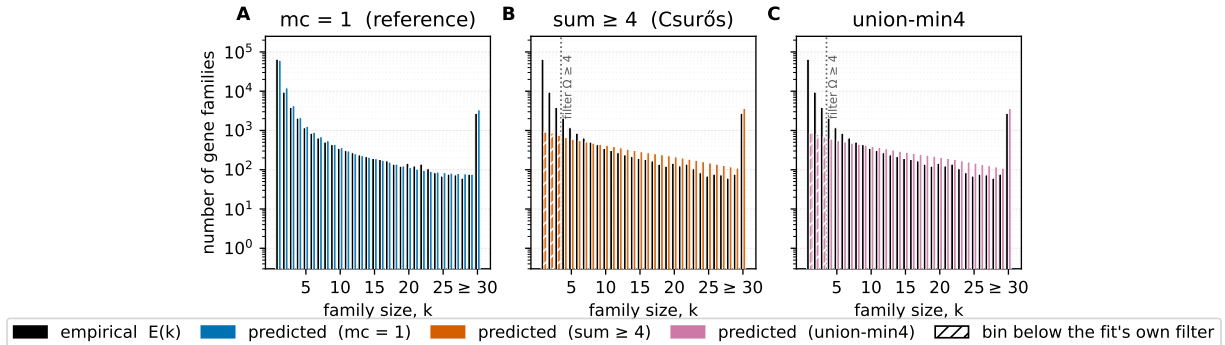


Figure 6. The Figure 5 comparison with the bin axis extended to $k = 30$, exposing the long-tail behaviour family-size by family-size.

Both generated by `validation/10_gof_min4_focus.py`; numeric table ($k = 1..7, 8$) in `validation/outputs/10_gof_min4_focus.txt`. Figure 5 panel A is the reference: at $mc = 1$ every bar pair has ratio ~ 1.0 (0.94, 1.29, 1.11, 1.03, 1.09, 1.06, 1.08, 1.08). Figure 5 panels B and C show the canonical filter-4 fits failing the same bins by $0.01 - 0.32$ at sub-filter $k < 4$ (hatched) and $0.32 - 0.80$ just above the filter. The contrast between “what a fit that matches the data looks like” (Figure 5 panel A) and “what the canonical $\Omega_{\min} = 4$ fits actually produce” (Figure 5 panels B and C) is the focal observation.

Log-log version — what the metric actually compares. The ratio test divides two count histograms; here are the two histograms themselves on log-log, side by side, for the two filter-4 fits + the $mc=1$ reference:

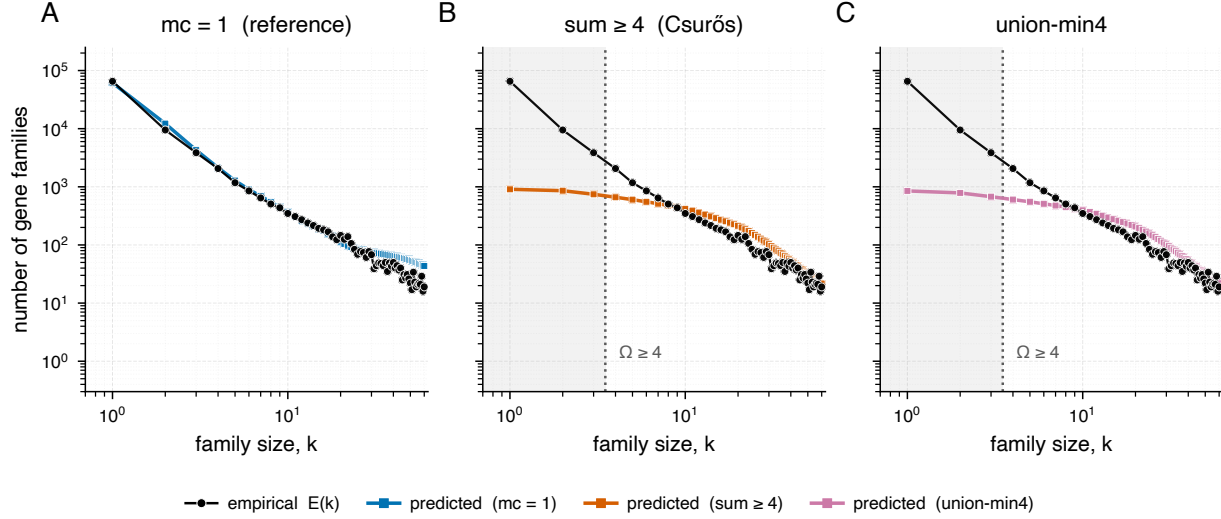


Figure 7. Log-log sum-bin count histograms — predicted $F^* \cdot P(\Omega = k)$ vs empirical $E(k)$ for the $mc = 1$ reference and the two filter-4 fits.

Generated by `validation/10_gof_loglog.py`. On log-log the empirical histogram traces a power-law-like decay from $\text{sum}=1$ (65k families) down to the tail at $\text{sum} = 60$ (~ 10 families). **Figure 7 panel A ($mc=1$, reference):** predicted curve (green squares) overlays empirical across the entire range — what a well-described rate process looks like. **Figure 7 panels B and C ($\text{sum} = 4$ Csurös + union-min4):** predicted curves match empirical only past $k = 5$ and visibly diverge in the sub-filter region ($k < 4$, shaded gray) by 1–2 orders of magnitude.

Reading Tables 7–9. One fit reproduces the empirical family-size histogram: the unfiltered $mc = 1$ fit. Its predicted/empirical ratio stays in **0.94–1.29** across every bin unconditionally, and the two conditional tests confirm this is not an artefact of the dominant singleton bin — re-normalised to $\Omega = 2$ (singletons dropped) the ratios are **0.89–1.11**, and to $\Omega = 3$ (1- and 2-copy families dropped) **0.95–1.03**. To within roughly 10–30 % per bin, the rate process the unfiltered fit recovers is a good generative description of the marginal arc269 family-size distribution.

No filtered fit does this. Every filtered $\text{sum} = N$ row of Tables 7–9 departs from the family-size histogram, and the departure grows with the threshold N — under-predicting the small families, over-predicting the large-family tail. The conditional tests show this is not a singleton artefact either: at $\text{sum} = 4$ the $\Omega = 2$ ratios already span 0.17–2.56. Section 3 returns to these rows; here they establish only that the *unfiltered* fit is the one fit in the sweep that reproduces the family-size distribution.

A caution on the word “fit”. That the unfiltered rate process reproduces the family-size distribution does not make its *ancestral* reconstruction correct. The size distribution is a marginal — one integer per family, pooled across families. The ancestral reconstruction is a per-family, per-node claim, and section 1 showed it is not identifiable from copy numbers when transfer is present. A perfectly calibrated generative model of family sizes can still place every transferred family on the root. The good fit established here matters for one reason: it removes “the optimiser failed” as an explanation for what the rest of section 2 shows.

(Implementation note: $L(0)$ and the Csurös $F \cdot L(0)/(1-L(0))$ extinction-amplification reproduce bit-for-bit against Csurös’ Java at $\Omega_{\min} > 1$. At $\Omega_{\min} = 1$ the Pn-tensor pipeline is not wired to the all-zeros branch, so the mul-

tiplicative $\times 1/(1-L(0))$ form is applied at internal nodes only. The three goodness-of-fit tables (Tables 7–9) are regenerated by `validation/l0_goodness_of_fit.py`; the formal derivation of the unconditional and conditional histogram checks is [docs/l0_conditional_gof.pdf](#).)

...and infers an ancestor larger than any living archaeon The same unfiltered $mc = 1$ fit — the one fit that reproduces the family-size distribution — places the arc269 root (LACA, the last archaeal common ancestor) at **3,847 gene families** (raw posterior). The richest single modern archaeal genome in the data carries about **3,425** families: the unfiltered fit’s ancestor is larger than any genome alive today. Csurős’ own extinction-amplification — multiplying by $1/(1 - L(0))$ to restore families that left no surviving copy — uses $L(0) = 0.79$ here and lifts the root estimate to **18,647 families**, roughly five times the richest modern archaeon and an order of magnitude above the median modern genome (890–1,642 families).

The same pattern holds clade by clade. Evaluated at each of the four focal subclades’ last common ancestors, the unfiltered arc269 fit infers more families than the richest modern member of that subclade:

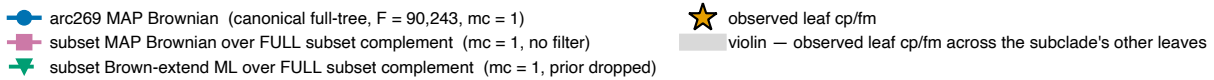
Table 10. Unfiltered arc269 fit at each subclade LCA, against the richest and median modern genome below it.

subclade	arc269 fit at the subclade LCA	richest modern genome below it	median modern genome	LCA / median
DPANN	2,822	1,731	890	3.2
Proteoarchaea	3,565	3,425	1,419	2.5
Methanobacteriati	3,777	3,000	1,642	2.3
Euryarchaeota	3,777	3,000	1,284	2.9

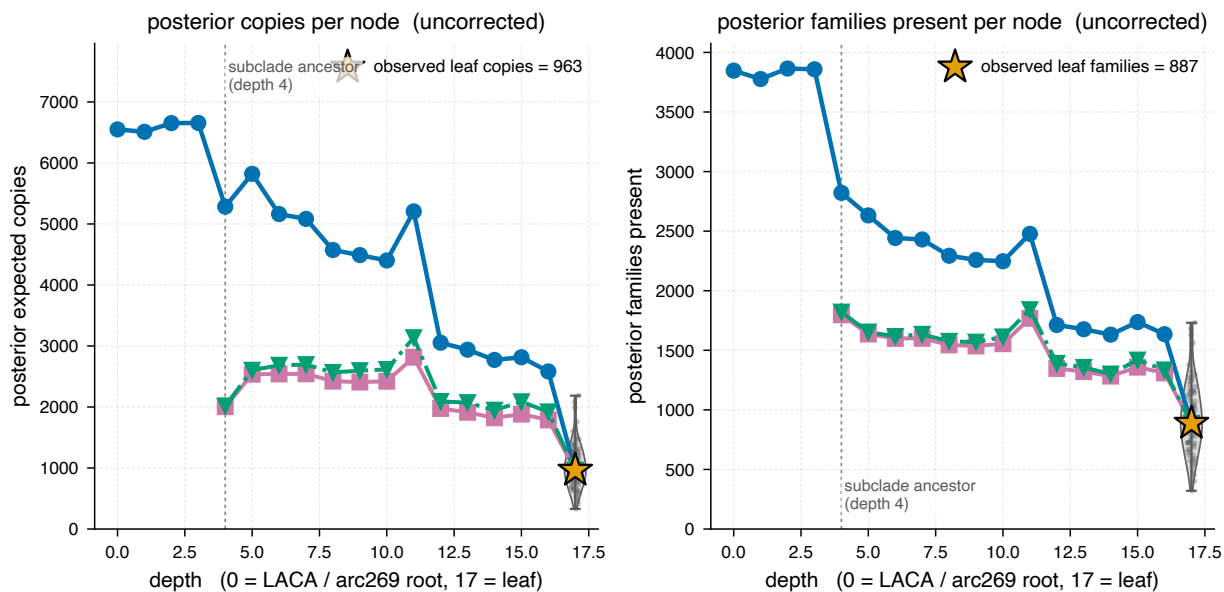
(Family counts over the full $F = 90,243$ family universe; the “arc269 fit” column is the no-filter $mc = 1$ posterior read at the subclade’s LCA node. Fitting each subclade on its own genomes instead gives the same picture — 1,732–3,666 families at the subclade root.)

This is the inflation. Section 1 gave the mechanism: every cross-clade transferred family contributes ancestral posterior it should not, and GLD has no way to discount it, so a copy-number fit on a transfer-rich kingdom carries a systematic upward bias on ancestral content. From copy numbers we cannot measure how much of the 3,847 is genuine ancestral content and how much is that bias — the identifiability gap of section 1 — so the figure cannot be read as an ancestral genome size. The next subsection asks whether the inflation is itself a modelling error; it is not, which is exactly why it cannot be waved away.

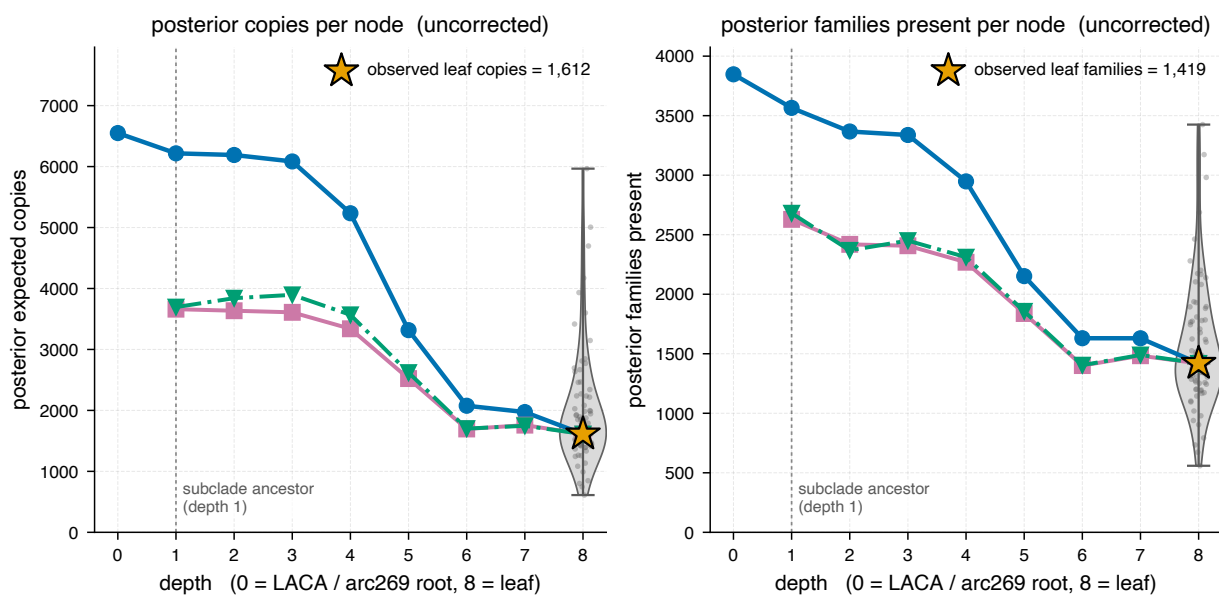
The trajectory plots below (Figure 8) trace this node by node. Each panel follows the path from the arc269 root to one subclade’s median-fm leaf; the red line is the unfiltered arc269 fit, the blue lines the subclade fitted on its own genomes (also unfiltered). The gold star marks the observed gene content of the leaf genome and the khaki violin the spread across the subclade’s other genomes — the modern values the reconstruction runs down to. In every subclade the fitted ancestor at the root (left edge) stands well above the cloud of observed modern genomes at the leaf (right edge).



DPANN (D80) · Set 2 · arc269 root to median-fm leaf '1974445'



Proteoarchaea (P75) · Set 2 · arc269 root to median-fm leaf '2058097'



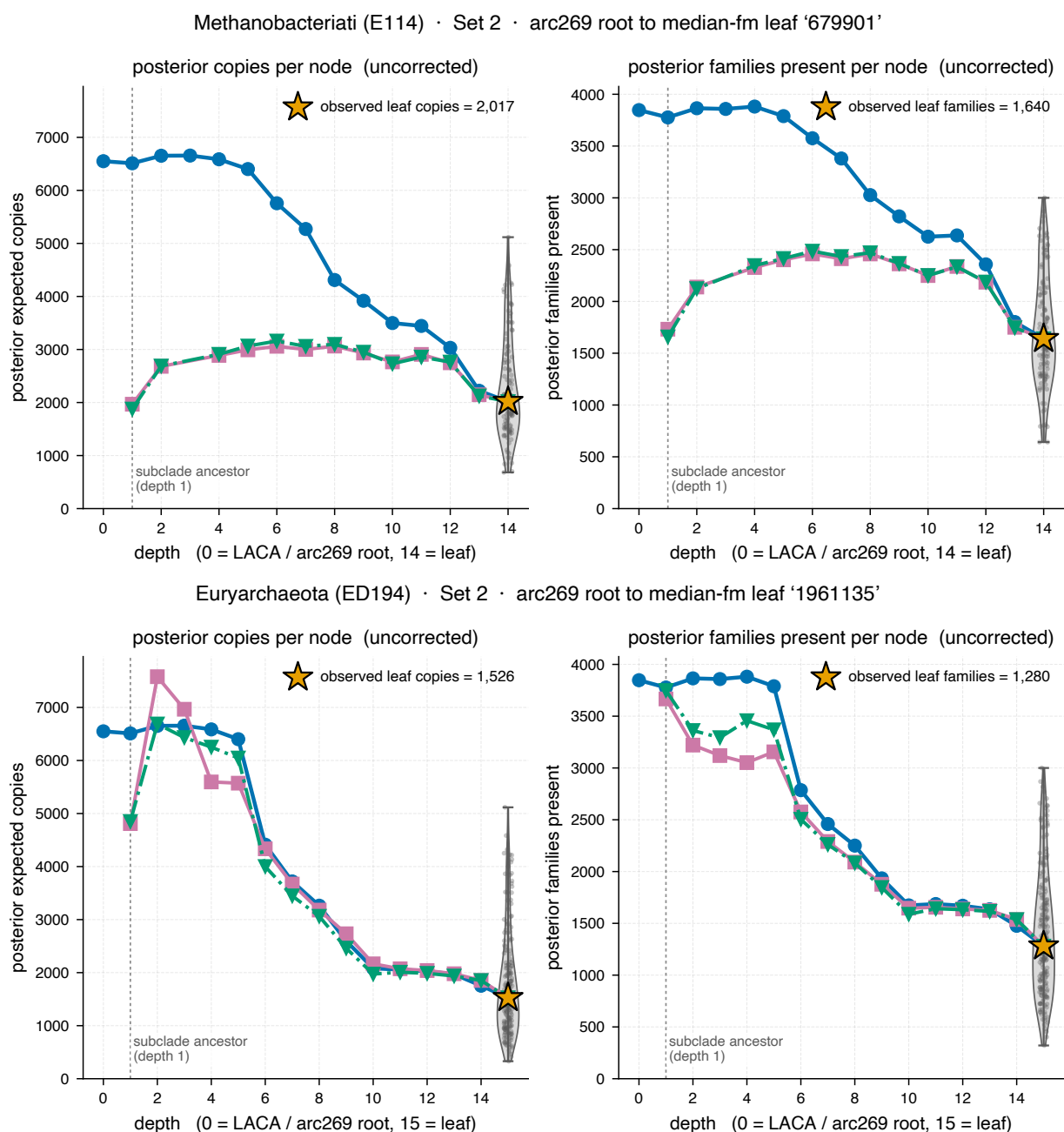


Figure 8. No-filter (Set 2) root-to-leaf trajectories — the unfiltered arc269 fit and each subclade’s own unfiltered fit, from the arc269 root to the subclade’s median-fm leaf.

Is the inflation a modelling error? No, and it is worth being precise about why not. Under any birth-death process with a non-zero loss rate the ancestor is *expected* to hold more families than any one descendant: if a fraction p of the ancestor’s families survive to a given genome, that genome shows about $p \cdot (\text{ancestor size})$ families, so the ancestor exceeds it by about $1/p$. An ancestor richer than its descendants is the generic signature of birth-death-with-loss, not a violation of the model.

The problem is not that GLD infers a large ancestor — it is that GLD on copy numbers **cannot apportion** that large ancestor between the two histories it cannot tell apart: families genuinely present at LACA and later lost, versus families that were never at LACA and reached their modern spread by transfer. The inflated count is the honest output of a correctly fitted model; it simply is not interpretable as an ancestral genome size. The next section examines the filter Csurós uses to bring the number down.

3 — The family-size filter, and why it does not rescue the inference

Csurös does not report the unfiltered ancestor. The published fits apply a family-size filter — keep only families with total copy count sum $\geq \Omega_{\min}$, with $\Omega_{\min} = 4$ — and the inferred ancestor that results is much smaller. This section shows the filter has two problems: its threshold is not derived from GLD, and applying it makes the fit to the family-size data worse.

The filter shrinks the inferred ancestor Apply each sum $\geq N$ filter ($N \in \{1..10\}$; $N = 1$ is no filter) to the arc269 profile and refit. The inferred root shrinks monotonically as N rises:

Table 11. arc269 filter sweep — F, L(0) and inferred root copies/families per sum $\geq N$ threshold.

arc269 fit	F	L(0)	root cp (raw / L(0)-amp)	root fm (raw / L(0)-amp)	cp/fm
no filter, mc=1	90,243	0.794	6,550 / 31,745	3,847 / 18,647	1.70
sum ≥ 2	24,904	0.250	3,750 / 4,998	2,460 / 3,279	1.52
sum ≥ 3	15,413	0.099	2,925 / 3,247	2,064 / 2,290	1.42
sum ≥ 4 (Csurös' threshold)	11,554	0.043	2,508 / 2,620	1,843 / 1,926	1.36
sum ≥ 5	9,492	0.021	2,342 / 2,393	1,741 / 1,779	1.34
sum ≥ 6	8,316	0.012	2,249 / 2,278	1,681 / 1,702	1.34
sum ≥ 7	7,465	0.008	2,178 / 2,195	1,634 / 1,647	1.33
sum ≥ 8	6,821	0.005	2,117 / 2,127	1,593 / 1,601	1.33
sum ≥ 9	6,315	0.003	2,051 / 2,058	1,550 / 1,555	1.32
sum ≥ 10	5,879	0.002	1,985 / 1,989	1,506 / 1,509	1.32

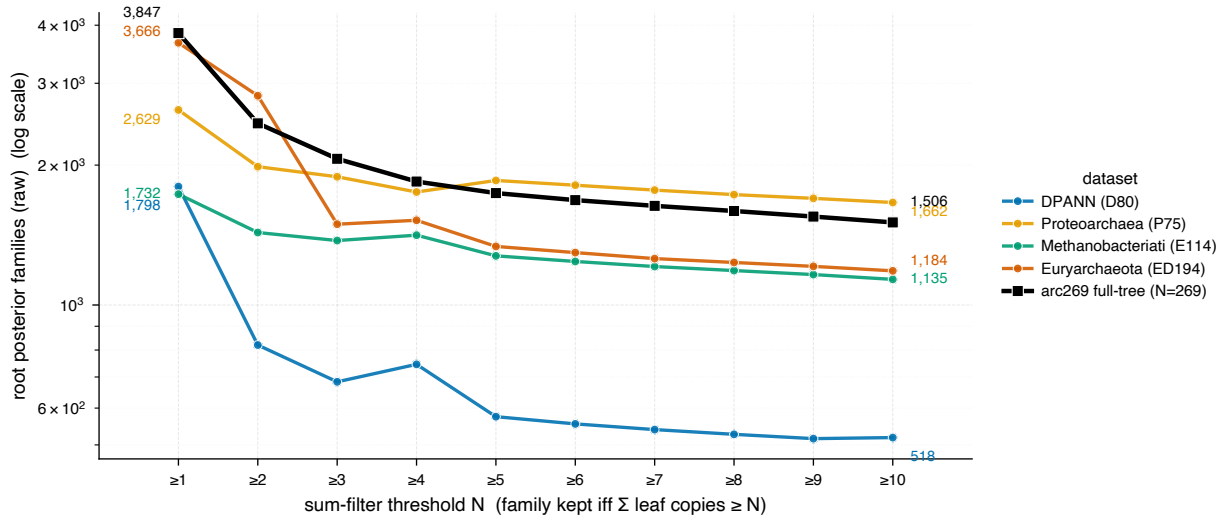
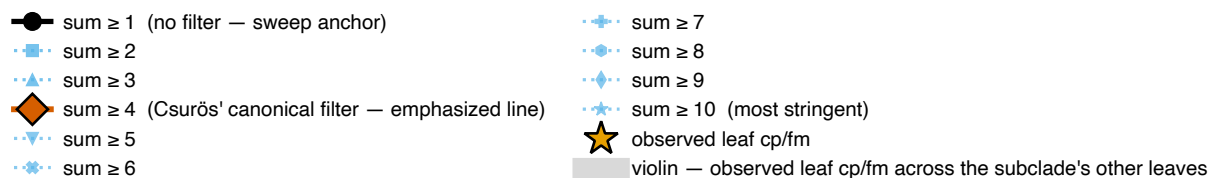


Figure 9. Inferred root family count vs the sum $\geq N$ filter threshold, for arc269 and each focal dataset. Generated by validation/sum_sweep_root_summary.py.

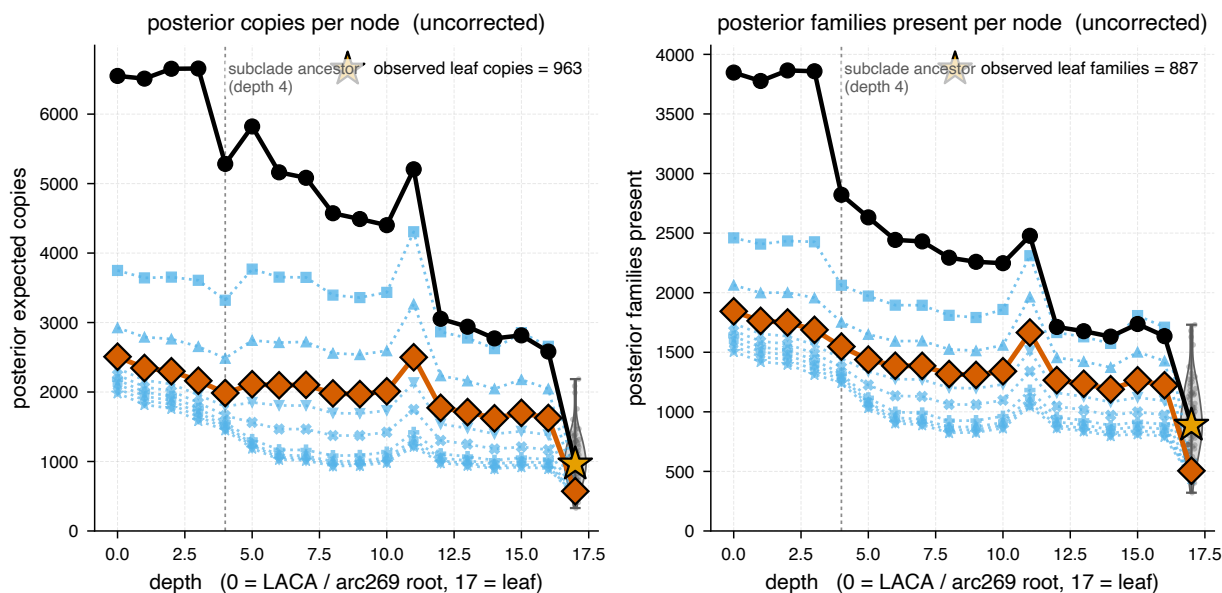
The root family count falls smoothly from 3,847 (no filter) to 1,506 (sum ≥ 10) — and the same decline holds for the four focal subclades fitted on their own genomes (Figure 9; the companion copy-count plot is validation/outputs/sum_sweep_root_copies.png). At Csurös' $\Omega_{\min} = 4$ the root is 1,843 families — less than half the unfiltered estimate, and now within the range of modern genome sizes. The filter does shrink the ancestor. The question is whether it shrinks it for a principled reason.

The sweep traced along the root-to-leaf path The shrinkage is not confined to the root node. Tracing each fit's posterior count along the path from the deepest node down to a subclade's median-fm leaf shows the whole ancestral half of the tree moving with the threshold. Two plot sets follow. **Set 1.5** sweeps the *single* arc269 full-tree fit across sum $\geq 1..10$ — one GLD fit to all 269 genomes at each threshold, its posterior then read along each subclade's path. **Set 1** runs the same sum $\geq 1..10$ sweep *per subclade*: each clade fitted on its own genomes alone, with no information from the rest of the tree. The two sets use the same colour and marker for the same threshold, so they can be read against each other; sum ≥ 4 , Csurös' canonical filter, is drawn bold with a black-edged marker.

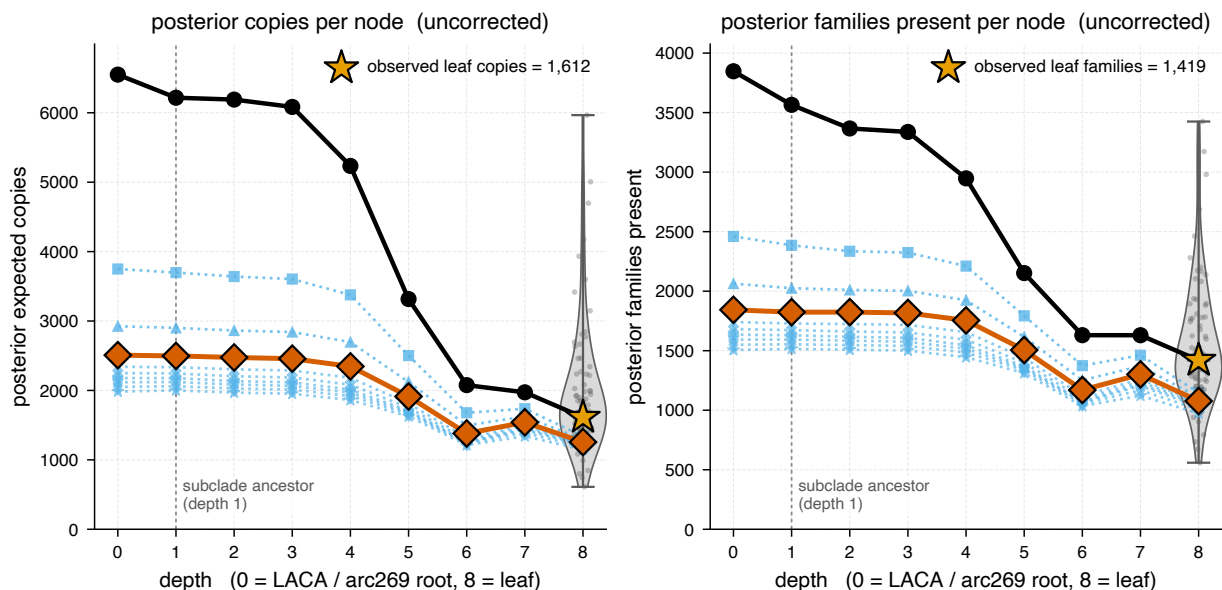
The Set 1 subset fits are recount's reproductions of Csurös' per-clade analysis — run on the same family profiles they recover his published per-clade fits (see [The fits analysed here are Csurös' fits below](#)).



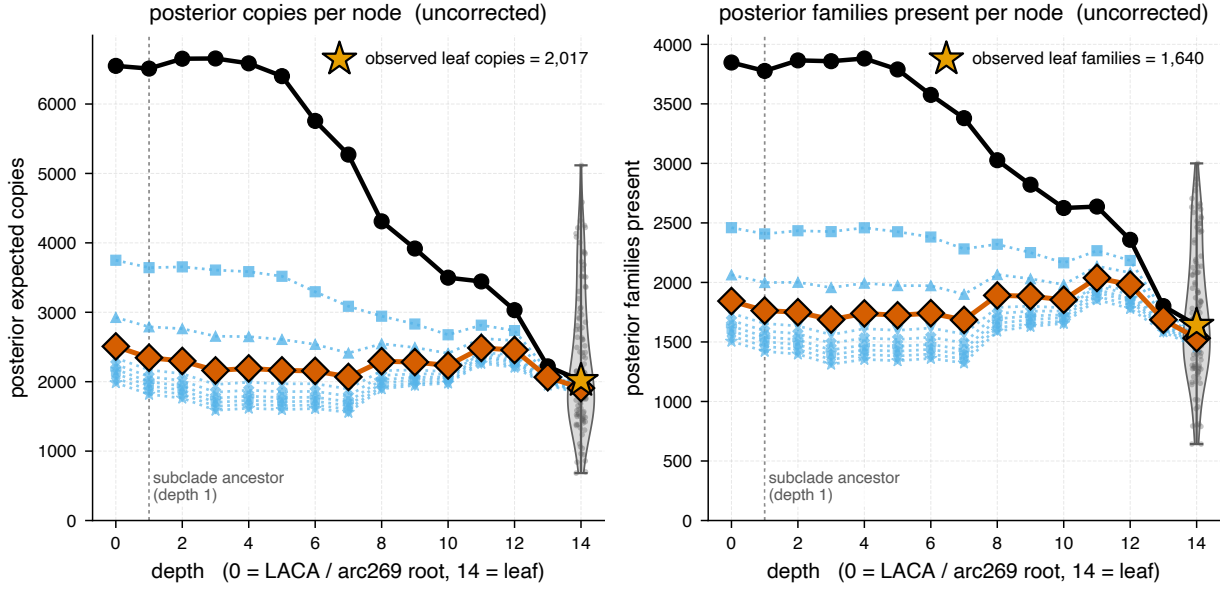
DPANN (D80) · Set 1.5 · arc269 root to median-fm leaf '1974445'



Proteoarchaea (P75) · Set 1.5 · arc269 root to median-fm leaf '2058097'



Methanobacteriati (E114) · Set 1.5 · arc269 root to median-fm leaf '679901'



Euryarchaeota (ED194) · Set 1.5 · arc269 root to median-fm leaf '1961135'

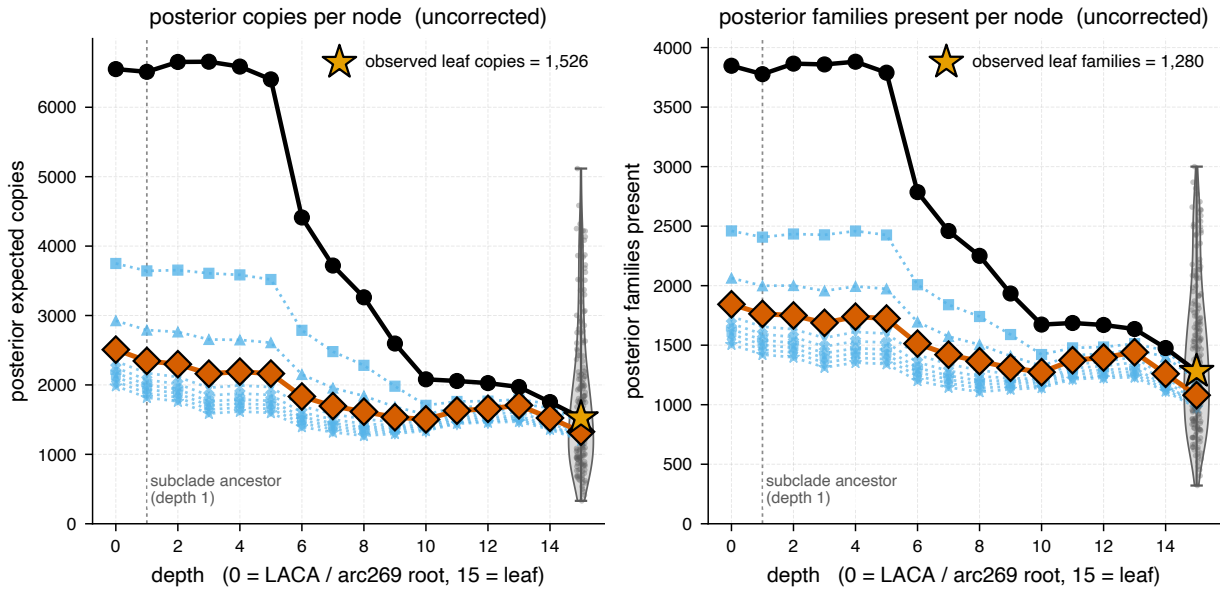
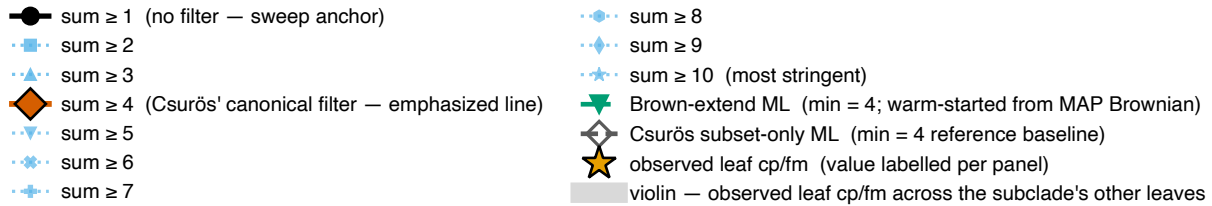
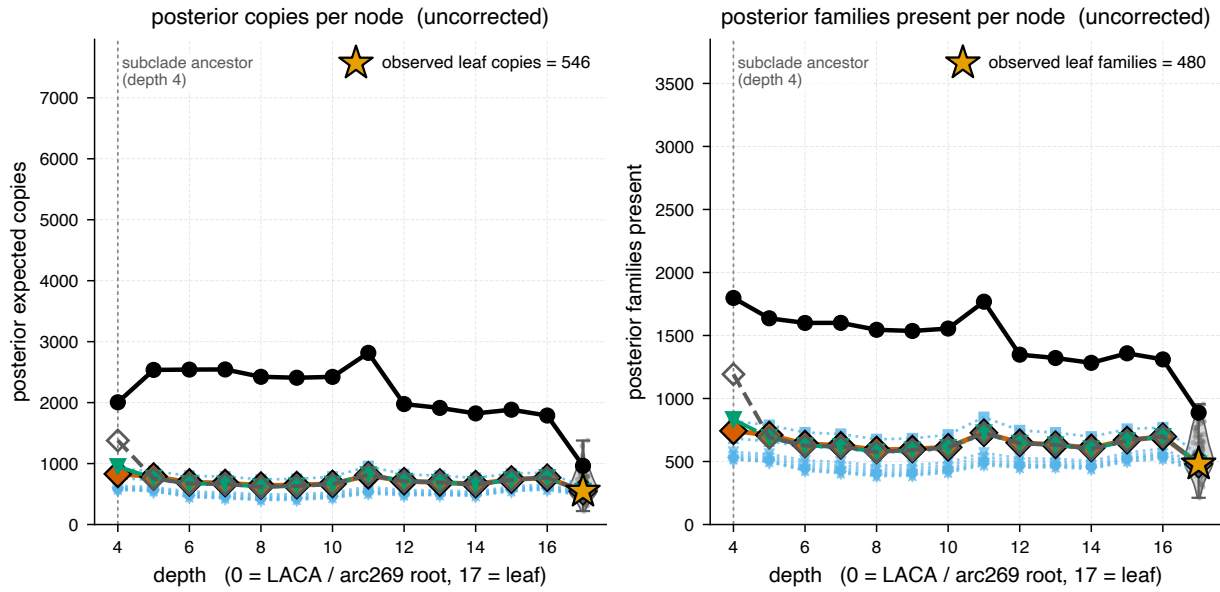


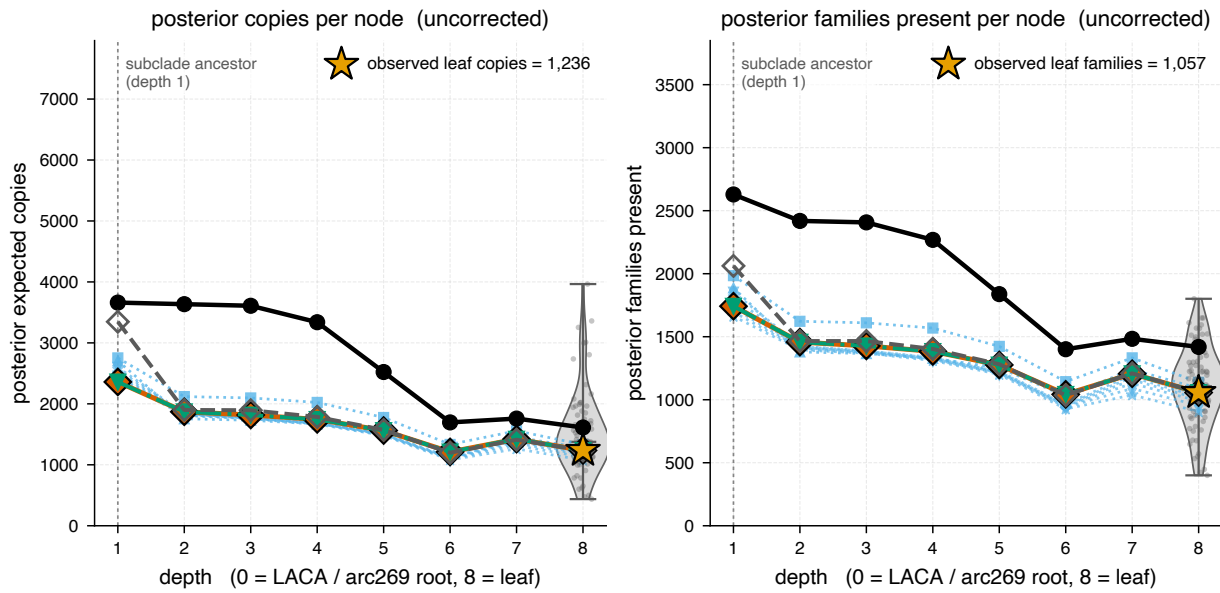
Figure 10. Set 1.5 — the single arc269 full-tree fit swept across sum 1..10, its posterior traced along each subclade's root-to-leaf path.



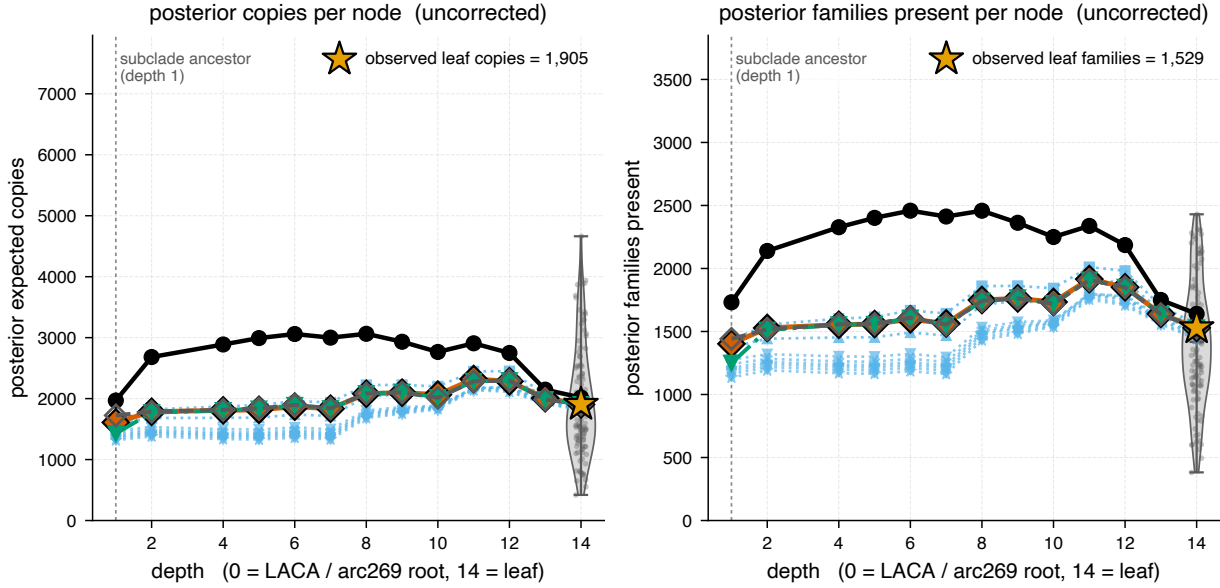
DPANN (D80) · Set 1 · arc269 root to median-fm leaf '1974445' (F = 3,034)



Proteoarchaea (P75) · Set 1 · arc269 root to median-fm leaf '2058097' (F = 5,179)



Methanobacteriati (E114) · Set 1 · arc269 root to median-fm leaf '679901' (F = 7,335)



Euryarchaeota (ED194) · Set 1 · arc269 root to median-fm leaf '1961135' (F = 8,855)

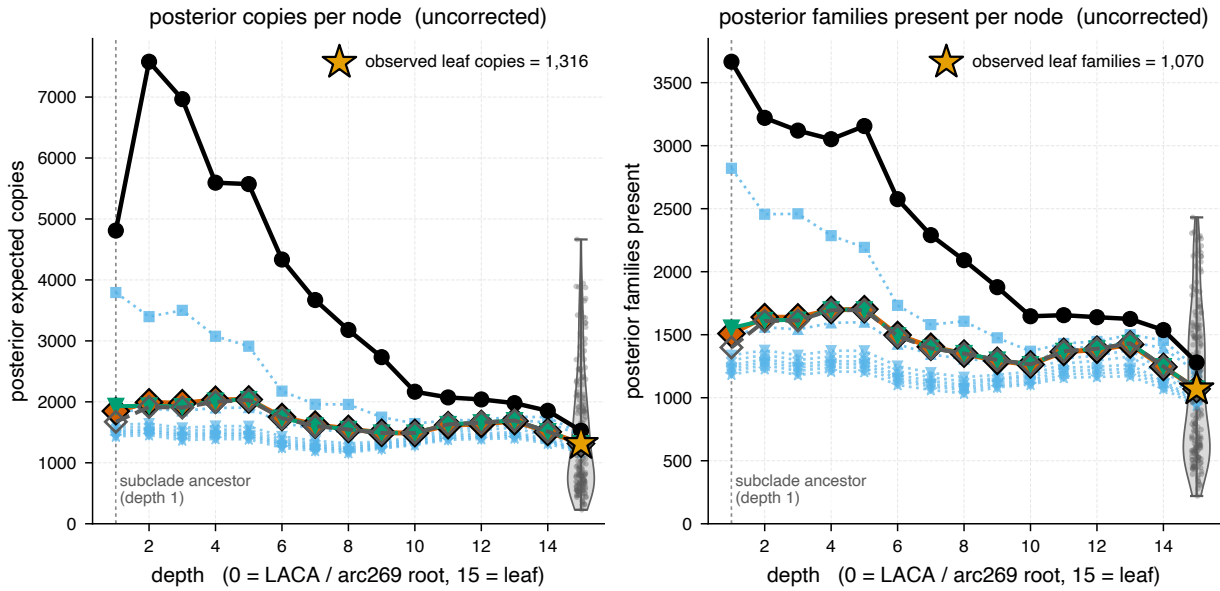


Figure 11. Set 1 — the same sum 1..10 sweep run per subclade, each clade fitted on its own genomes, traced along its root-to-leaf path.

Three things, read straight off Figures 10 and 11 and Table 11:

- **The threshold has a large effect.** From sum 1 to sum 10 the inferred root family count changes by $1.5\times$ to $3.5\times$, depending on the dataset: arc269 $3,847 \rightarrow 1,506$ (factor 2.6); DPANN $1,798 \rightarrow 518$ and Euryarchaeota $3,666 \rightarrow 1,184$ (both $3\times$); Proteoarchaea $2,629 \rightarrow 1,662$ and Methanobacteriati $1,732 \rightarrow 1,135$ ($1.5\times$). The choice of threshold sets the answer to within a factor of two to three.
- **Removing the singletons is not enough.** The single largest step in the sweep is sum 1 $\rightarrow$ sum 2 — discarding the singletons — but it does not settle the estimate: every later threshold moves it again. The arc269 sweep is monotone and still descending at sum 10 (3% per step, no plateau); after the singletons are gone the arc269 root still falls a further 39% ($2,460 \rightarrow 1,506$). There is no N at which the inferred ancestor stops moving within the range tested.
- **The effect is clade-dependent.** The four subclades do not shrink by the same factor — DPANN and Euryarchaeota lose $3\times$ of their root content across the sweep, Proteoarchaea and Methanobacteriati $1.5\times$

— so a threshold that discards one fraction of the small-family stratum in one clade discards a different fraction in another. (The per-subclade curves dip then rise slightly at $N = 4$: that point alone is read from Csurős’ canonical $\min = 4$ fit, run under the $\Omega_{\min} = 4$ observation-bias setting rather than the $mc = 1 +$ pre-filter used at the other thresholds; the endpoints, and the arc269 sweep, are unaffected.)

A further pattern emerges when the two sweeps are read against each other at the **subclade ancestor** — the deepest node at which both a full-tree fit (Set 1.5) and a subset-only fit (Set 1) report a value. The full-tree fit’s response to the threshold is large and roughly uniform across the four clades; the subset-only fits respond far more unevenly. Each cell below gives the inferred family count at that clade’s ancestor at $\text{sum} = 1$ (no filter) and at $\text{sum} = 10$, with the sweep ratio (uncorrected posterior counts):

Table 12. Inferred family count at each subclade ancestor at $\text{sum} = 1$ vs $\text{sum} = 10$, full-tree fit vs subset-only fit.

subclade ancestor	full-tree fit (Set 1.5)	subset-only fit (Set 1)
DPANN	2,822 $\rightarrow$ 1,245 ($\times 2.3$)	1,798 $\rightarrow$ 518 ($\times 3.5$)
Proteoarchaea	3,565 $\rightarrow$ 1,515 ($\times 2.4$)	2,629 $\rightarrow$ 1,662 ($\times 1.6$)
Methanobacteriati	3,777 $\rightarrow$ 1,421 ($\times 2.7$)	1,732 $\rightarrow$ 1,135 ($\times 1.5$)
Euryarchaeota	3,777 $\rightarrow$ 1,421 ($\times 2.7$)	3,666 $\rightarrow$ 1,184 ($\times 3.1$)

Methanobacteriati and Euryarchaeota resolve to the same ancestral node in the arc269 tree, so the full-tree fit reports one value for both. Across the sweep the full-tree fit moves its estimate of every clade ancestor by $\times 2.3$ – 2.7 on families ($\times 3.1$ – 3.6 on copies). The subset-only fits split in two: the DPANN and Euryarchaeota ancestors move $3\times$ — as much as the full-tree fit — but the Proteoarchaea and Methanobacteriati ancestors move only $1.5\times$. For those two clades the family-size filter is a much heavier knob in the full-tree fit than in the clade-alone fit: the threshold change that barely shifts the Proteoarchaea or Methanobacteriati ancestor when the clade is fitted on its own moves the full-tree fit’s estimate of the *same node* by 1.5 – $1.7\times$ as much (2.0 – $2.4\times$ on copies). The threshold’s influence on the inferred ancestor is contingent twice over — on the clade, and on whether that clade was fitted alone or as part of the whole tree.

The data do not single out a threshold: the estimate is a continuous, unplateaued function of N , and the per-clade effect differs, so no one N is simultaneously the natural cut for all four clades. Csurős’ reasons for $N = 4$ (next subsection) come from outside this sweep.

Problem 1 — the threshold $\Omega_{\min} = 4$ is not a property of GLD Why four? Csurős 2026 PNAS (p. 2) gives the reason: comparability with gene-tree reconciliation methods, which need at least four copies of a family before its unrooted gene tree carries any topological signal. That is a genuine rationale — but it is a property of *reconciliation*, not of GLD. The GLD likelihood places no lower bound on family size; it is defined for a family of any size, and the $\Omega_{\min}$ sampling-bias correction is defined for any $\Omega_{\min} \geq 1$, including $\Omega_{\min} = 1$ (no filter). Nothing inside GLD selects four, or selects any threshold at all.

And the inferred ancestor depends strongly on the choice. The sweep in Table 11 is smooth and monotone — root family count $3,847 \rightarrow 2,460 \rightarrow 2,064 \rightarrow 1,843 \rightarrow \dots \rightarrow 1,506$ as N runs $1 \rightarrow 10$ — with no elbow, no plateau, no value the data singles out. Csurős’ own SI reports fits at $\Omega_{\min} = 4, 2$ and 1 and notes that the inferred history shifts with the threshold. The ancestral genome size GLD reports is therefore set, to better than a factor of two, by a knob whose value is imported from the requirements of a different method.

Problem 2 — filtering makes the fit to the family-size data worse Return to the goodness-of-fit tables (Tables 7–9) in section 2. The unfiltered $mc = 1$ fit reproduces the empirical family-size histogram (ratios 0.94–1.29). Every filtered fit fails it, and fails it worse the stronger the filter:

- at Csurős’ $\Omega_{\min} = 4$ the predicted/empirical ratios run from 0.01 at $k = 1$ to 1.35 at $k = 8$ — the filtered rate process predicts far too few small families and too many large ones;
- the conditional tests rule out “it is only the singletons”: re-normalised to drop singletons ($\Omega = 2$) the $\text{sum} = 4$ ratios still span 0.17–2.56, and dropping 1- and 2-copy families ($\Omega = 3$) they span 0.24–1.70. The *shape* of the predicted distribution is wrong on the families the filter keeps, not only on the families it discards;
- the misfit grows monotonically with the threshold: at $\text{sum} = 10$ the $\Omega = 2$ ratios span 0.03–3.27.

So filtering does not solve the problem; it moves it. The unfiltered fit reproduces the family-size data, but its ancestor is inflated past any living genome and — by section 1 — transfer- confounded. The filtered fit returns a

smaller, modern-sized ancestor, but it no longer reproduces the family-size data, the threshold that set its size is arbitrary, and the ancestor is still transfer-confounded: dropping the small families lowers the count but does not give the likelihood the gene-tree signal it would need to separate inheritance from transfer. No filter setting yields an ancestral estimate that is at once consistent with the family-size data and free of the transfer ambiguity — the second is not something a copy-number filter can buy.

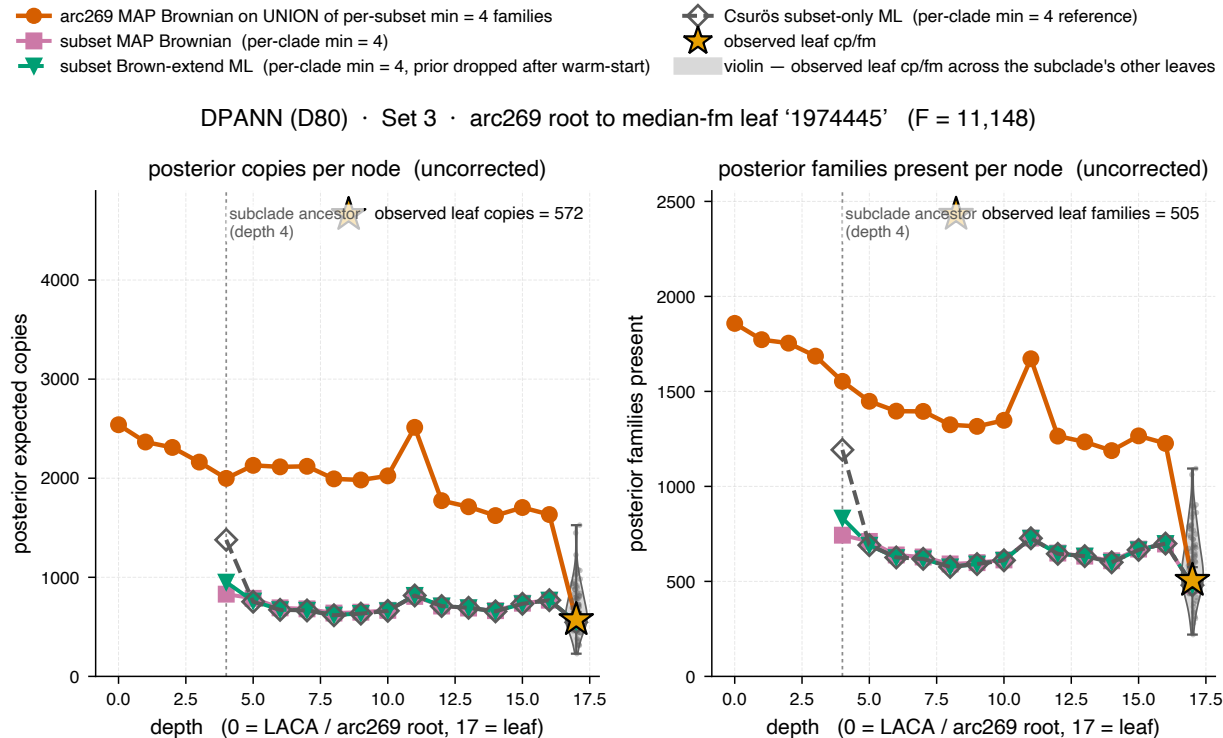
The fits analysed here are Csurős’ fits The critique above is of the published analysis, not of a re-fit that happens to behave differently. Evaluated at the four subclade ancestors, the recount fit on the union of the per-subclade sum 4 family sets matches Csurős’ published per-clade fits to within 0.88–1.28 on ancestral family count:

Table 13. Ancestral family count at each subclade ancestor — Csurős’ published per-clade sum 4 fit vs the recount union-sum 4 fit.

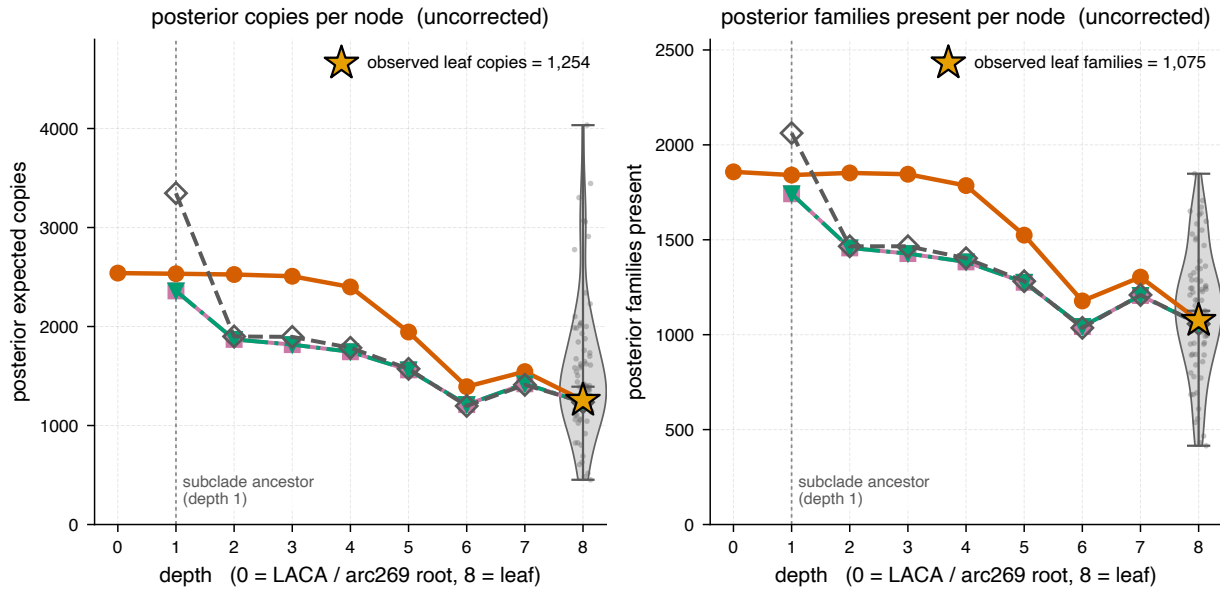
subclade ancestor	Csurős per-clade sum 4 fm	recount union-sum 4 fm	ratio
DPANN	1,210	1,553	1.28
Proteorarchaea	2,096	1,841	0.88
Methanobacteriati	1,453	1,772	1.22
Euryarchaeota	1,419	1,772	1.25

The per-branch rates agree to within 10–30 % on the gain axis for three of the four subclades (median ratio 0.71–1.11; DPANN is the exception — long-branch artefacts are a plausible cause) and to within 25 % on branch length. The [reproduction tables](#) (Tables 2–5) make the same point on log-likelihood: recount’s recommended fits land within a few nat of Csurős’ published rates on every dataset. The behaviour analysed here is the behaviour of the published GLD analysis.

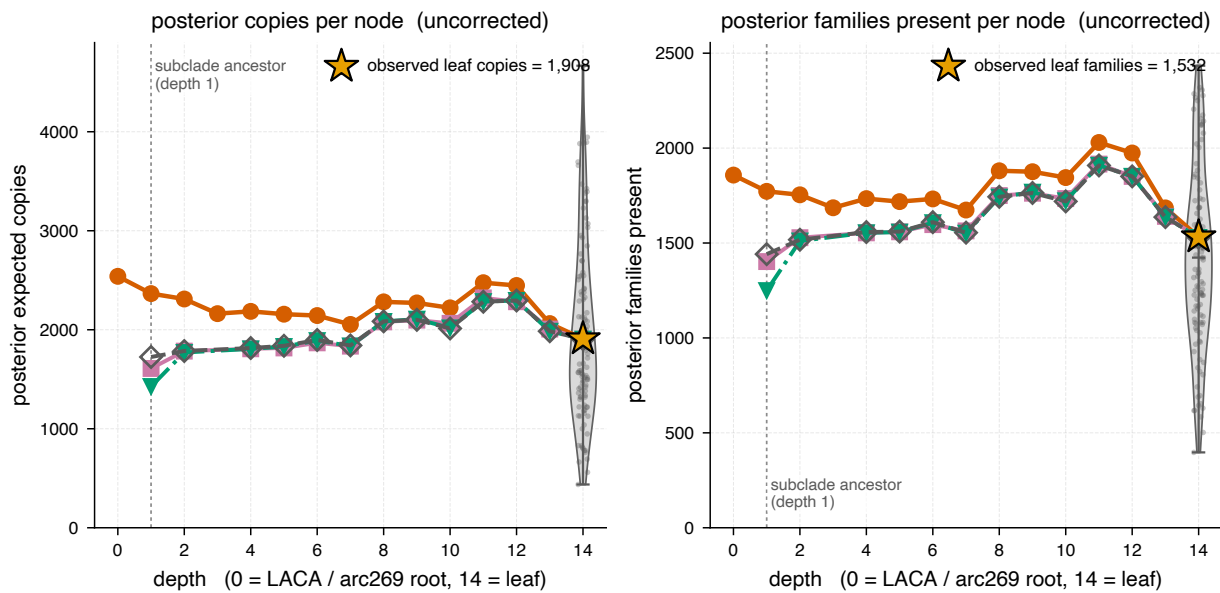
The Set 3 path plots (Figure 12) show this agreement node by node. Each panel traces the single arc269 full-tree fit on the union-sum 4 universe (seagreen) alongside Csurős’ published per-clade subset-only ML (gray) and recount’s two per-clade fits (MAP Brownian, Brownian-extend ML), from the arc269 root to a subclade’s median-fm leaf. Over the stretch where both a full-tree and a per-clade fit are defined — from the subclade ancestor leafward — the seagreen and gray lines run close together: the node-by-node form of the 0.88–1.28 ancestral agreement of Table 13.



Proteoarchaea (P75) · Set 3 · arc269 root to median-fm leaf '2058097' (F = 11,148)



Methanobacteriati (E114) · Set 3 · arc269 root to median-fm leaf '679901' (F = 11,148)



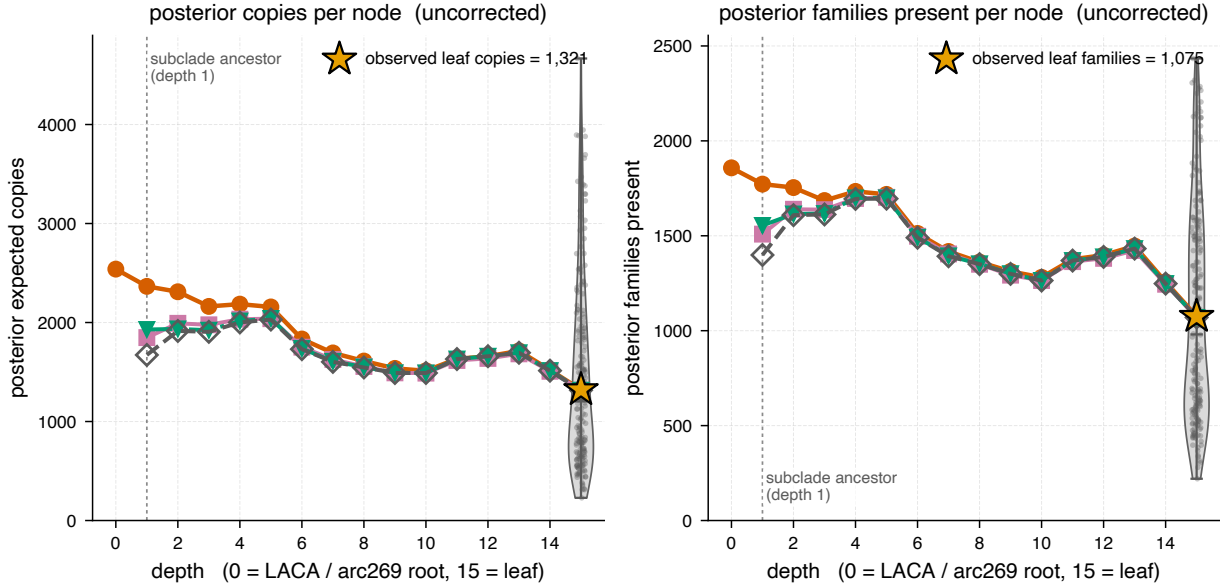


Figure 12. Set 3 — the arc269 full-tree fit on the union-sum 4 universe traced against Csurós’ per-clade reference and recount’s two per-clade fits, root to median-fm leaf.

The L(0)-amplified counterparts are in [L\(0\)-corrected trajectories](#) at the end of this README.

Characterising the filtered ancestor: the LACA CORE Take the filtered fit at face value for a moment. From the per-family posteriors of the arc269 fit on the union of the per-subclade sum 4 families (F = 11,148 — the fit that reproduces Csurós’ published per-clade numbers, above), define the inferred ancestral core as the families the model places at the root with posterior probability at least one-half:

LACA CORE = { families with $P(\text{present at the arc269 root}) \geq 0.5$ }

— 1,817 families. The natural check on a putative ancestral core is how it is distributed across the modern genomes that descend from it:

Table 14. Distribution of the 1,817-family LACA CORE across each subclade’s modern genomes.

subclade	LACA-CORE families in 90 % of the subclade’s genomes	...in every genome	median per-genome retention
DPANN	1.7 %	1 family	20 %
Proteoarchaea	4.1 %	1 family	45 %
Methanobacteriati	13.5 %	1 family	52 %
Euryarchaeota	3.2 %	0 families	41 %

Not one of the 1,817 inferred-ancestral families is present in every genome of even a single subclade, and the median modern genome carries only 20–52 % of the inferred core. That is a strikingly patchy object for a putative ancestral genome. But patchiness is consistent with two readings — heavy lineage-specific *loss* of a genuinely ancestral core, or a core that is partly *transfer-spread* families that were never ancestral — and, by section 1, copy numbers cannot tell the two apart. The characterization is reported for completeness; it does not, on its own, decide the question. Full per-threshold tables are in the [supporting tables](#) below; the figure and tables are regenerated by [validation/laca_core_analysis.py](#).

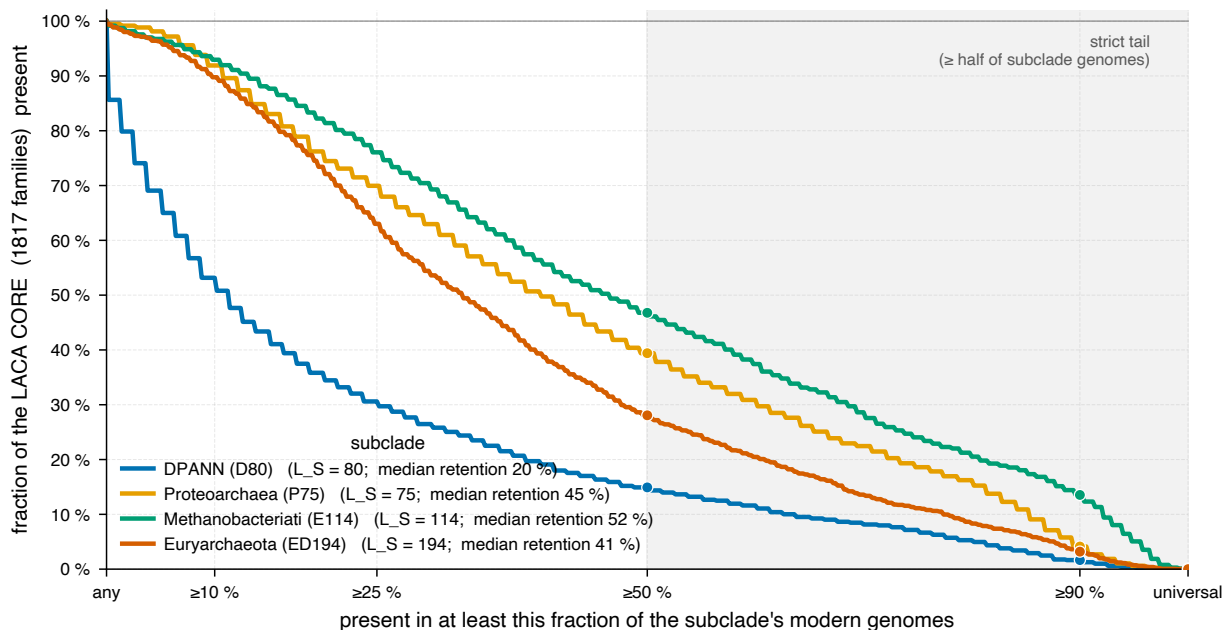


Figure 13. Distribution of the LACA CORE families across the genomes of each subclade.

4 — Conclusion

The [reproduction section](#) confirms that the GLD likelihood, its gradient and the $\Omega_{\min}$ sampling-bias correction are correctly implemented, and that recount’s fits recover Csürös’ published rates to within a few nat. As a likelihood of gene-copy-number data, the GLD model is exactly what the paper says it is.

The claim tested in this section is the narrower one in the abstract’s closing sentence — that the framework is “*a statistically sound foundation for hypothesizing about gene content evolution across the diversity of entire kingdoms.*” On the arc269 archaeal data it is not, for three reasons that compound:

- **Copy numbers do not identify ancestry when transfer is present.** A transferred family and a genuinely inherited one can carry the same copy-number profile, and GLD — whose model has no transfer edge — gives the two the same ancestral posterior; it cannot tell them apart (section 1). Because a transferred family’s true ancestral contribution is zero, every cross-clade transfer biases the inferred ancestor upward. Archaea are a transfer-rich kingdom, so this is the typical case, not an edge case.
- **Unfiltered, GLD fits the family-size data and infers an ancestor larger than any living archaeon.** The no-filter fit reproduces the empirical family-size distribution to within 10–30 % per bin, yet places 3,847 families at LACA — above the richest modern archaeal genome — and its own extinction correction raises that to 18,600 (section 2). The inflation is not an optimisation failure — the fit is faithful to the family-size data — and copy numbers cannot say what share of the inferred ancestor is genuine ancestral content and what share is transfer-spread families read as ancestral.
- **The filter that suppresses the inflation is unprincipled and degrades the fit.** $\Omega_{\min} = 4$ is imported from the requirements of gene-tree reconciliation, not from GLD; the inferred ancestor is a smooth function of the threshold with no value the data selects; and every filtered fit describes the family-size distribution worse than the unfiltered fit, whether or not the singleton stratum is excluded (section 3).

No setting of the copy-number-only GLD framework delivers, at the same time, a rate process consistent with the observed family-size distribution and an ancestral genome estimate not confounded by transfer. Unfiltered, the fit is faithful to the data and the ancestor is not credible. Filtered, the ancestor is modern-sized but the fit is no longer faithful to the data, the threshold is arbitrary, and — because the family-size filter does not change what the likelihood can see — the identifiability gap of section 1 remains: the smaller ancestor is still a mixture of inherited and transfer-spread families that copy numbers cannot separate.

This is a limitation of the *input*, not of the algorithm. Gene copy numbers alone do not carry the signal that separates vertical inheritance from horizontal transfer; the within-family signal that does — gene-tree topology, sampled from sequence alignments — is what reconciliation methods consume and GLD does not. A copy-number likelihood,

however correctly implemented, is therefore not a sufficient foundation for ancestral gene-content inference in a kingdom shaped by transfer. That is the specific sense in which the analysis here does not support the closing claim of Csurös 2026.

Supporting data

Filter-sensitivity tables: per-leaf retention of M_N across thresholds $N = 1..10$

A model-free companion to the LACA CORE characterization above. For subset S with leaves L_S , threshold N , and filter family universe $M_N = \{ f : \Sigma \text{ over } v \in L_S \text{ of } \text{profile}[f, v] \geq N \}$, define

$$r_v(N) = |\{ f \in M_N : \text{profile}[f, v] = 1 \}| / |M_N|$$

— the fraction of the filter family universe present at extant genome v . The tables below (Tables 15–24) report this distribution per threshold N alongside the model’s posterior fm at the subset root (Csurös subset-only ML at $N = 4$; per-subset Brownian MAP at the other N) and the “universal” / “90 % of leaves” counts.

$N = 1$ (no filter applied) **Table 15.** Per-leaf retention of the filter family universe M_N at threshold $N = 1$ (no filter), per subset.

subset	L_S	F_N	root_fm	r_{p10}	r_{p50}	r_{p90}	r_{max}	universal	90 % lvs	%univ	% 90 %
DPANN	80	28,576	1,798	0.020	0.031	0.045	0.061	1	30	0.00	0.10
Proteo	75	33,547	2,629	0.031	0.042	0.065	0.102	1	75	0.00	0.22
Methano	114	37,609	1,732	0.031	0.044	0.067	0.080	1	246	0.00	0.65
Eury	194	62,435	3,666	0.012	0.021	0.038	0.048	0	58	0.00	0.09

$N = 2$ **Table 16.** Per-leaf retention of M_N at threshold $N = 2$, per subset.

subset	L_S	F_N	root_fm	r_{p10}	r_{p50}	r_{p90}	r_{max}	universal	90 % lvs	%univ	% 90 %
DPANN	80	6,632	820	0.067	0.096	0.135	0.178	1	30	0.02	0.45
Proteo	75	10,746	1,985	0.073	0.111	0.160	0.213	1	75	0.01	0.70
Methano	114	13,257	1,432	0.067	0.114	0.168	0.195	1	246	0.01	1.86
Eury	194	17,637	2,821	0.030	0.062	0.111	0.147	0	58	0.00	0.33

$N = 3$ **Table 17.** Per-leaf retention of M_N at threshold $N = 3$, per subset.

subset	L_S	F_N	root_fm	r_{p10}	r_{p50}	r_{p90}	r_{max}	universal	90 % lvs	%univ	% 90 %
DPANN	80	4,084	683	0.101	0.144	0.201	0.253	1	30	0.02	0.73
Proteo	75	6,694	1,888	0.105	0.169	0.241	0.305	1	75	0.01	1.12
Methano	114	9,202	1,376	0.092	0.156	0.236	0.271	1	246	0.01	2.67
Eury	194	11,515	1,492	0.042	0.088	0.165	0.217	0	58	0.00	0.50

$N = 4$ (Csurös’ canonical threshold) **Table 18.** Per-leaf retention of M_N at the canonical threshold $N = 4$, per subset.

subset	L_S	F_N	root_fm	r_{p10}	r_{p50}	r_{p90}	r_{max}	universal	90 % lvs	%univ	% 90 %
DPANN	80	3,034	1,192	0.124	0.180	0.255	0.315	1	30	0.03	0.99
Proteo	75	5,179	2,062	0.131	0.214	0.294	0.348	1	75	0.02	1.45
Methano	114	7,335	1,441	0.108	0.193	0.289	0.331	1	246	0.01	3.35
Eury	194	8,855	1,399	0.052	0.110	0.212	0.275	0	58	0.00	0.65

N = 5 Table 19. Per-leaf retention of M_N at threshold N = 5, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	2,481	575	0.148	0.212	0.296	0.363	1	30	0.04	1.21
Proteo	75	4,285	1,853	0.150	0.254	0.338	0.378	1	75	0.02	1.75
Methano	114	6,308	1,276	0.114	0.219	0.331	0.375	1	246	0.02	3.90
Eury	194	7,454	1,337	0.060	0.129	0.247	0.318	0	58	0.00	0.78

N = 6 Table 20. Per-leaf retention of M_N at threshold N = 6, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	2,166	555	0.164	0.236	0.324	0.393	1	30	0.05	1.39
Proteo	75	3,659	1,810	0.170	0.289	0.372	0.415	1	75	0.03	2.05
Methano	114	5,709	1,241	0.123	0.237	0.360	0.406	1	246	0.02	4.31
Eury	194	6,691	1,297	0.066	0.142	0.272	0.348	0	58	0.00	0.87

N = 7 Table 21. Per-leaf retention of M_N at threshold N = 7, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	1,893	539	0.184	0.258	0.357	0.418	1	30	0.05	1.58
Proteo	75	3,268	1,767	0.185	0.318	0.407	0.442	1	75	0.03	2.29
Methano	114	5,268	1,210	0.129	0.255	0.384	0.433	1	246	0.02	4.67
Eury	194	6,089	1,258	0.071	0.155	0.295	0.375	0	58	0.00	0.95

N = 8 Table 22. Per-leaf retention of M_N at threshold N = 8, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	1,718	527	0.198	0.274	0.378	0.435	1	30	0.06	1.75
Proteo	75	2,962	1,728	0.202	0.341	0.437	0.470	1	75	0.03	2.53
Methano	114	4,916	1,186	0.136	0.269	0.406	0.455	1	246	0.02	5.00
Eury	194	5,674	1,235	0.075	0.165	0.314	0.396	0	58	0.00	1.02

N = 9 Table 23. Per-leaf retention of M_N at threshold N = 9, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	1,573	516	0.215	0.294	0.400	0.454	1	30	0.06	1.91
Proteo	75	2,727	1,696	0.217	0.367	0.462	0.498	1	75	0.04	2.75
Methano	114	4,610	1,163	0.143	0.283	0.425	0.474	1	246	0.02	5.34
Eury	194	5,296	1,211	0.079	0.175	0.333	0.414	0	58	0.00	1.10

N = 10 Table 24. Per-leaf retention of M_N at threshold N = 10, per subset.

subset	L_S	F_N	root_fm	r_p10	r_p50	r_p90	r_max	universal	90 % lvs	%univ	% 90 %
DPANN	80	1,458	518	0.228	0.312	0.417	0.467	1	30	0.07	2.06
Proteo	75	2,514	1,662	0.230	0.385	0.489	0.529	1	75	0.04	2.98
Methano	114	4,319	1,135	0.151	0.299	0.443	0.492	1	246	0.02	5.70
Eury	194	4,940	1,184	0.084	0.186	0.351	0.433	0	58	0.00	1.17

Variability and universal core. At every (subset, N) the p10–p90 spread in r_v is a factor of 2–3, and tightening the filter shifts the whole distribution upward but does not narrow the spread. Within the model-free M_N family universe, the strict universal-presence core (in every leaf of the subclade) is at most one family at any threshold (ED194 has zero); the near-universal core (90 % of leaves) is at most 246 families (Methanobacteriati at $N=7$) and typically 30–80 — that is 5 % of M_N across all 28 (subclade $\times$ threshold) combinations. The same pattern is reported in the LACA CORE section above using the model’s per-family posteriors rather than the filter universe.

Profile of families dropped by stricter filters. As the filter is loosened from $N = 4$ toward $N = 1$, the subset’s family count grows roughly nine-fold (DPANN 3,034 $\rightarrow$ 28,576; Eury 8,855 $\rightarrow$ 62,435), dominated by very small profiles:

Table 25. Counts of the small-profile families (sum = 1, 2, 3) that stricter filters drop, per subset.

subset	sum = 1 (singletons)	sum = 2	sum = 3
DPANN	21,944	2,548	1,050
Proteo	22,801	4,052	1,515
Methano	24,352	4,055	1,867
Eury	44,798	6,122	2,660

Among sum = 1 families every member is, by construction, restricted to a single subset leaf. Among sum = 2 families, the majority (80–84 % across subsets) are spread across two distinct subset leaves rather than two copies on a single leaf; among sum = 3 families, 97 % span at least two leaves and 80 % span at least three. So “sum < 4 families are simply transient annotations on a single genome” is empirically true only for sum = 1. From sum = 2 onward the families are distributed across multiple lineages and the “transient” label is itself a model judgement — they could be ancient orthologs surviving only in a few lineages, recent transfers deposited in a few recipients, or annotation noise spanning a few genomes. The GLD likelihood cannot distinguish these on copy counts alone.

Diagnostic plots

Per-subset arc269 root $\rightarrow$ median-fm leaf path plots, organised into four sets that isolate orthogonal biological questions. Each set shares a single y-axis cap across its four subset panels so the plots are directly comparable side-by-side. The gold-star OBSERVED leaf value and the khaki violin distribution are computed against each set’s family universe (see each plot’s subtitle for F).

Each set is shown first in the uncorrected form (raw posterior over observed families). The $L(0)$ -amplified counterparts of every plot are collected in the [L\(0\)-corrected trajectories](#) section at the end of this README. Plots containing both a full-tree and a per-clade fit also carry a vertical dotted line at the depth where the subclade’s LCA sits on the arc269 path. Above this line only the arc269 line is defined (the subset tree is rooted at the subclade LCA); below it both fits are defined and can be compared node-for-node. Every Set plot in this section, and its $L(0)$ -corrected counterpart, is generated by [validation/brownian_split_path_plots.py](#).

Set 1 (per-clade sum-filter sweep, Figure 11) and **Set 1.5 (arc269 full-tree sum-filter sweep, Figure 10)** are shown and discussed in §3, [The sweep traced along the root-to-leaf path](#). **Set 2 (min = 1, no filter — subset full-complement vs arc269 canonical, Figure 8)** is shown in §2, [...and infers an ancestor larger than any living archaeon](#). **Set 3 (arc269 full-tree fit on the union of per-clade min = 4 families, against the Csurós per-clade reference, Figure 12)** is shown in §3, [The fits analysed here are Csurós’ fits](#). The $L(0)$ -amplified counterparts of all four sets are in [L\(0\)-corrected trajectories](#) below.

Per-branch GLD rates (matched-min comparison: arc269 full-tree vs subset)

For each filter min $N \in \{1..7\}$, two views: a 2×2 rate-path plot per subset (/ / / t along the same root $\rightarrow$ median-fm-leaf path), and a 1×3 pooled scatter (/ / t — omitted, always fixed at 1) of arc269 vs subset rate at every LCA-matched internal node across all 4 subsets. **At matched min N the two fits use the same filter threshold but different leaf sets** (arc269 sums over all 269 leaves; each subset sums over its own leaves), so it’s the closest like-for-like comparison without re-fitting arc269 once per subset (28 extra fits). The rate-path and pooled-scatter plots are generated by [validation/matched_min_rate_plots.py](#).

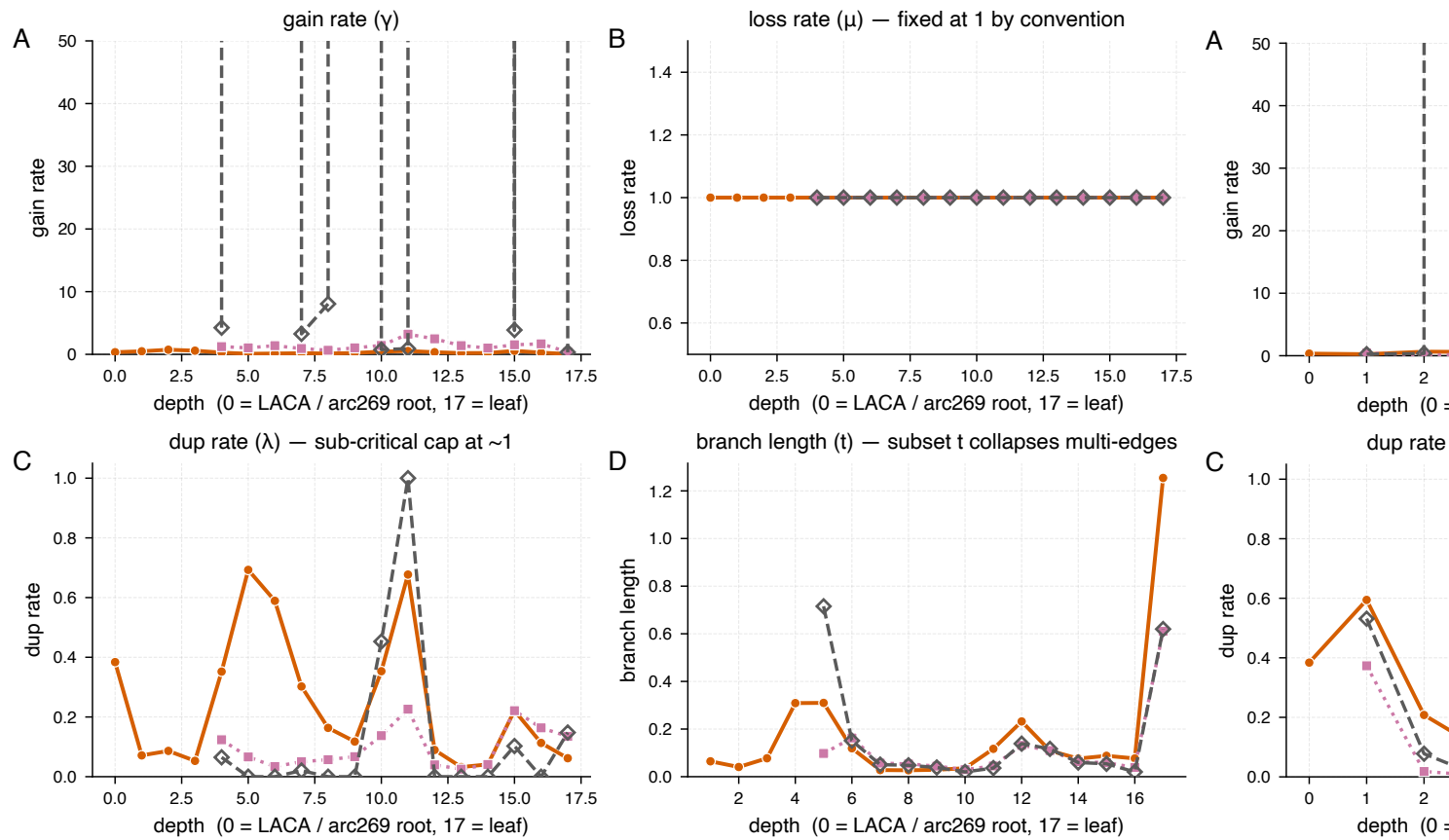
Shared legend (same colours across all min levels and pooled scatters):

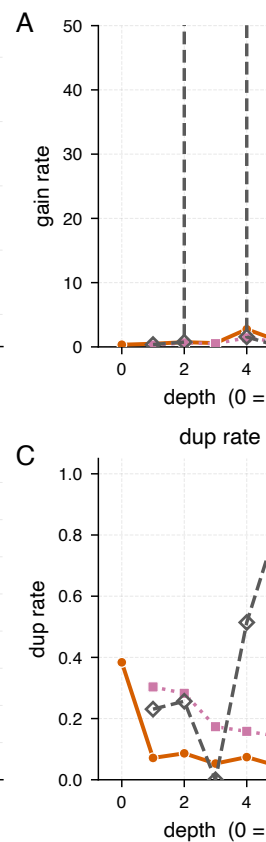
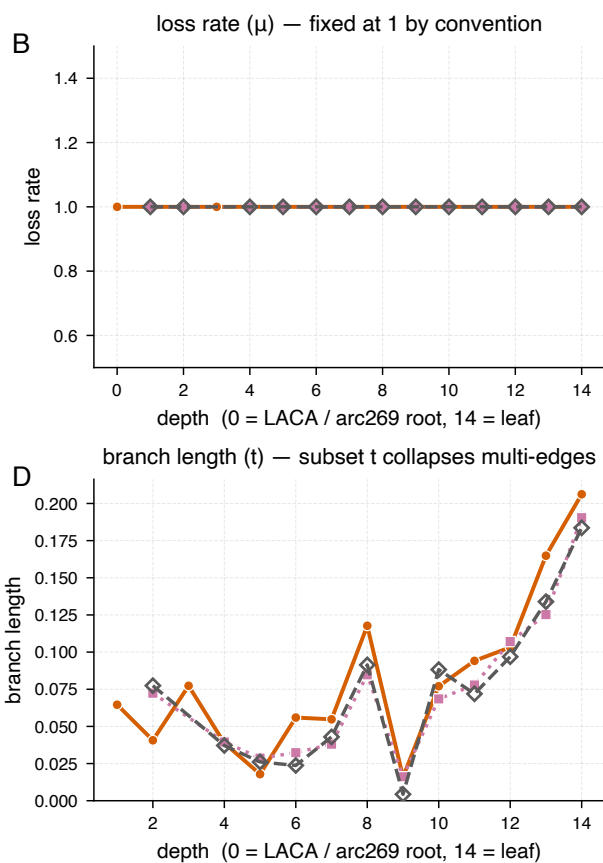
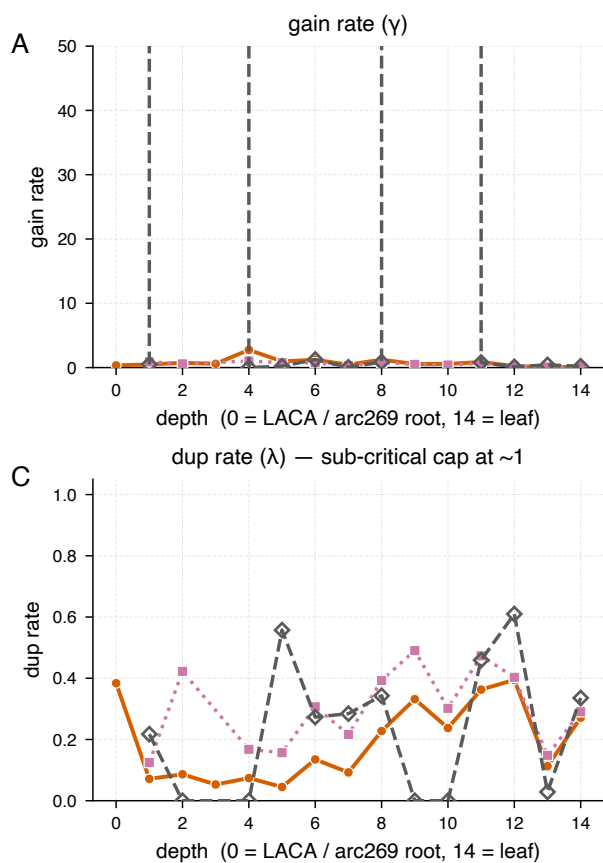
- arc269 MAP Brownian, $\text{sum} \geq N$ (full-tree, $F=90,243$ -filtered)
- subset MAP Brownian, $\text{sum} \geq N$ (per-clade, subset-leaves-filtered)
- ◇— Csurös subset-only ML (reference baseline, shown only at $\text{sum} \geq 4$)

The Csurös subset-only ML reference (gray dashed) is shown only at $\text{min}=4$ since that's the only filter Csurös fits at.

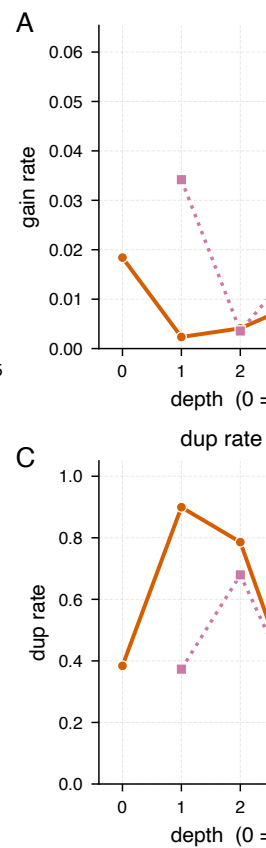
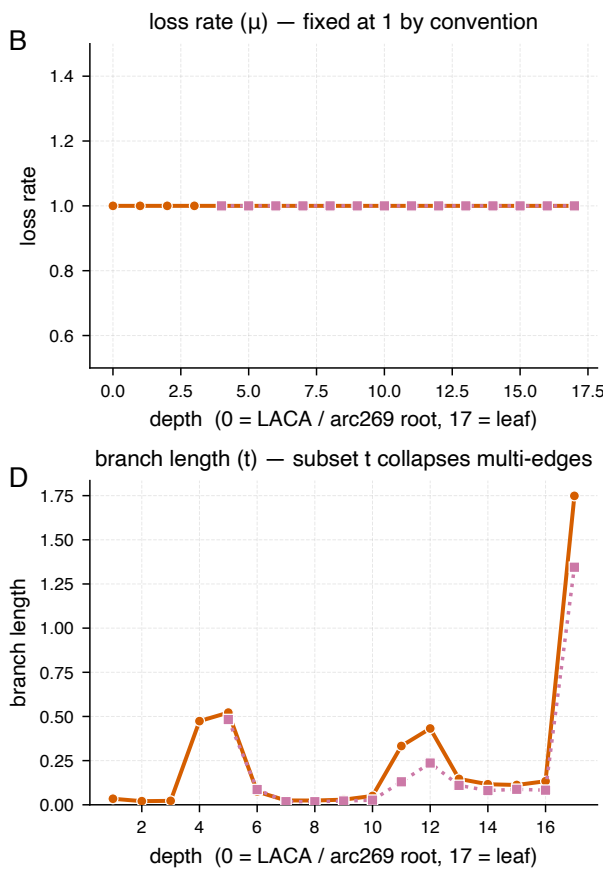
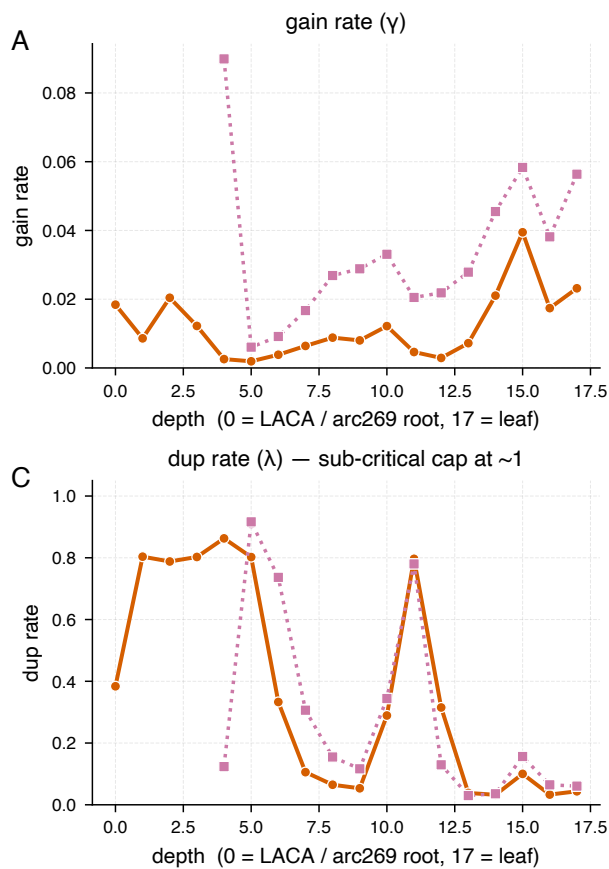
Per-min rate paths (4 subset panels each) $\text{min}=4$ is the canonical Csurös-matched comparison — shown first. Other min levels follow as filter-sensitivity diagnostics.

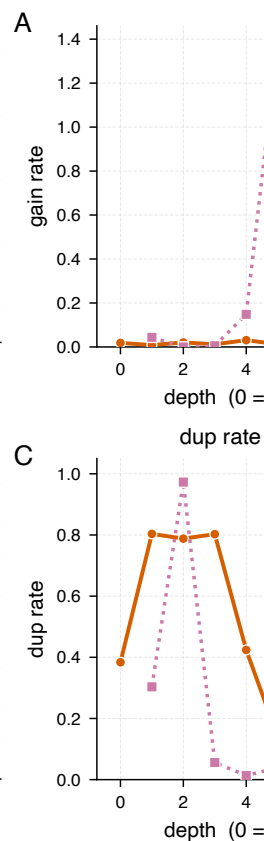
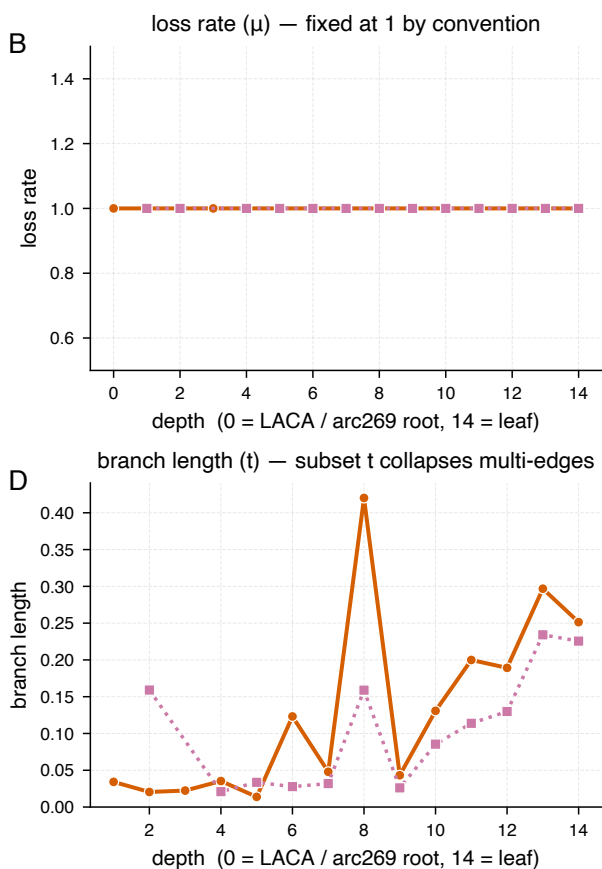
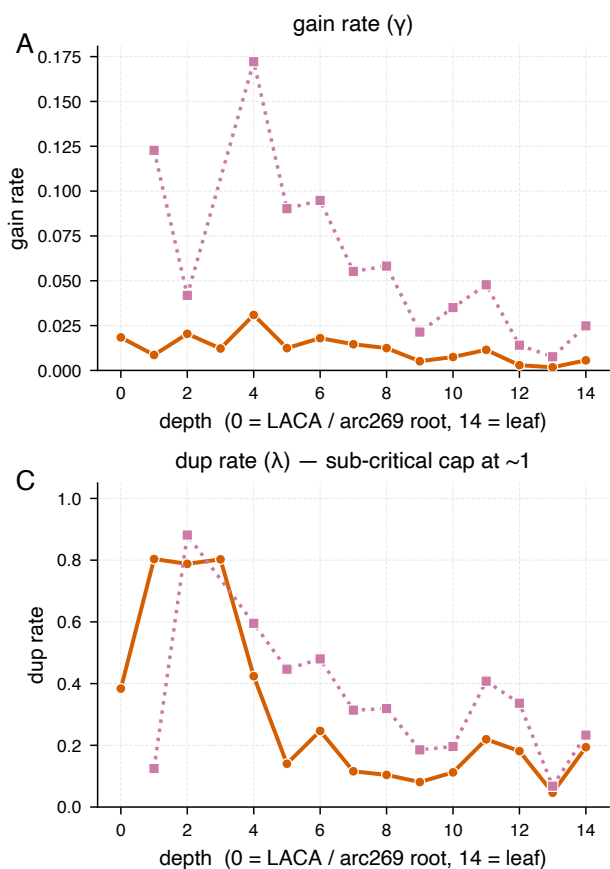
$\text{min}=4$ — canonical Csurös-matched (click to expand other mins)



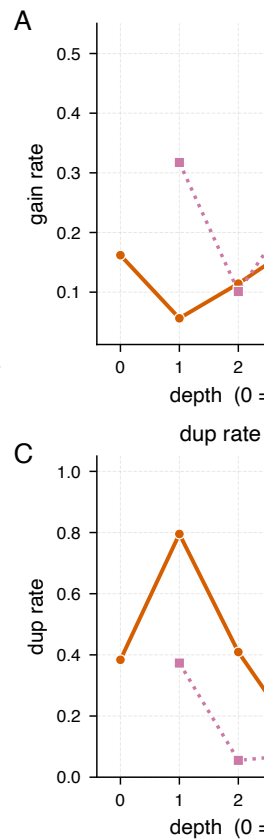
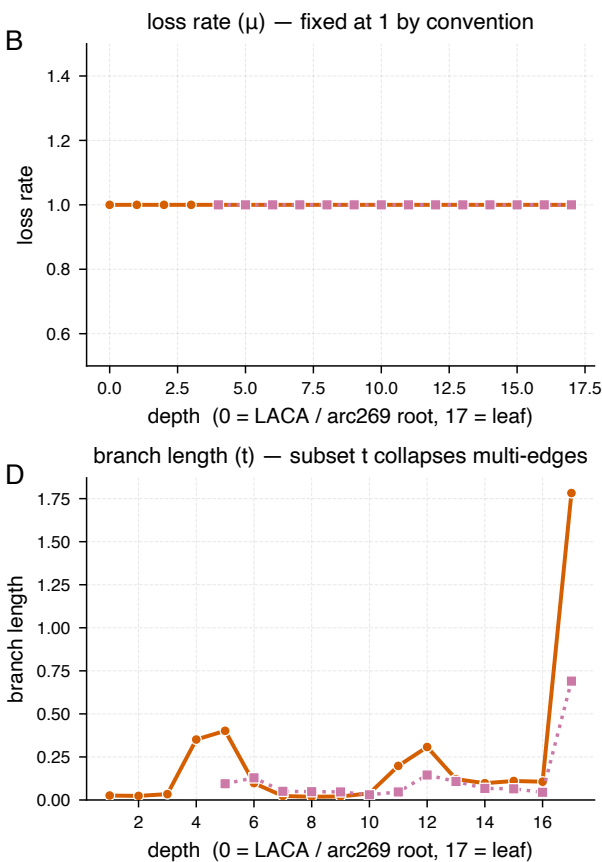
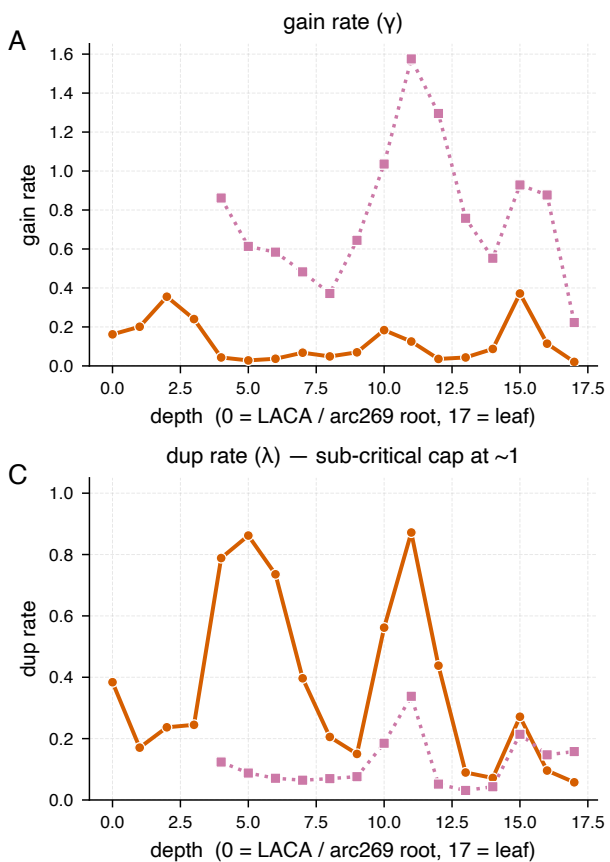


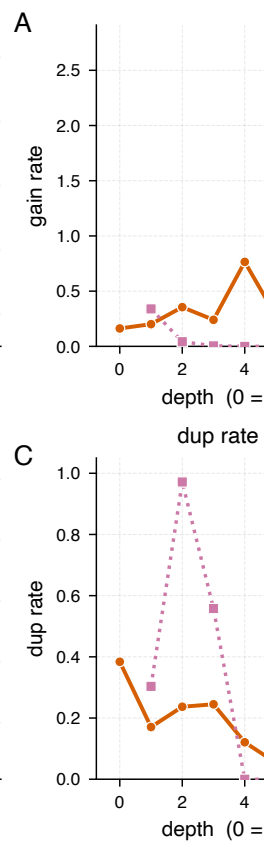
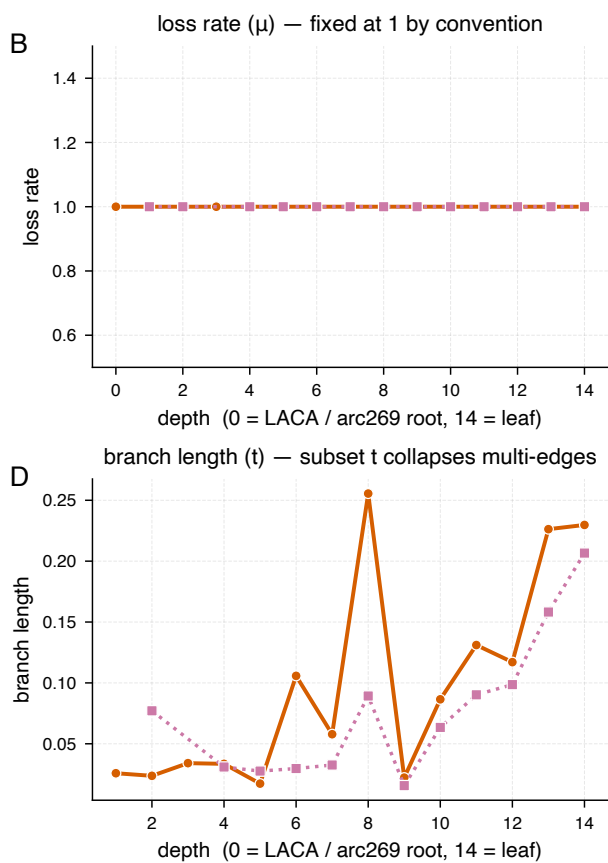
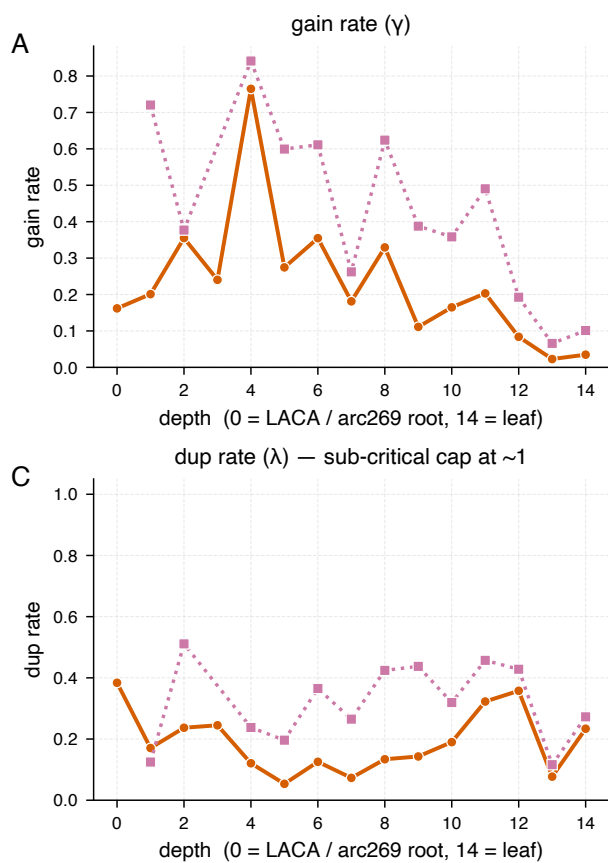
min=1 (no filter)



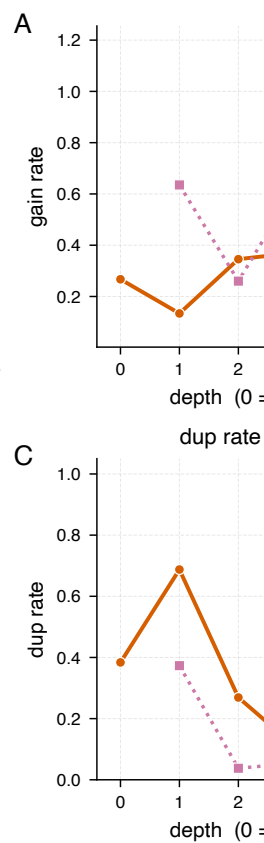
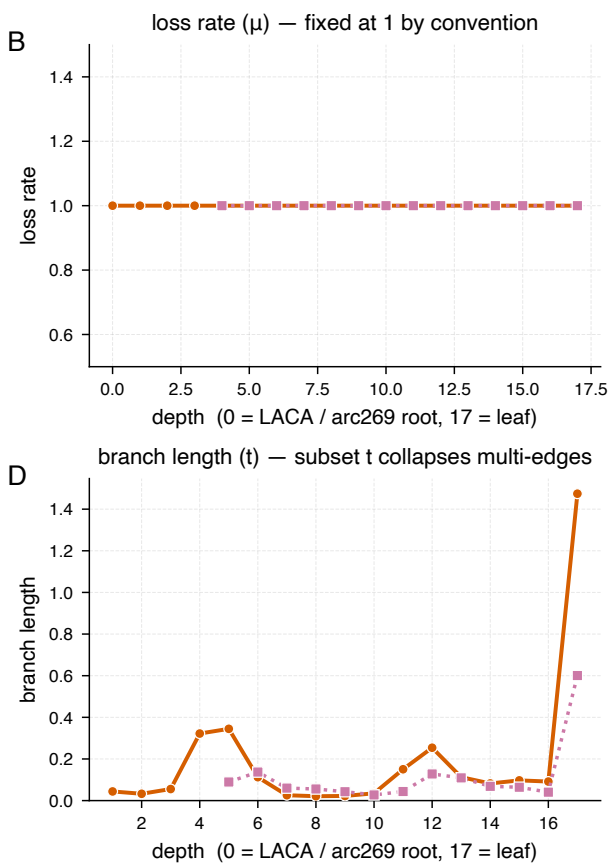
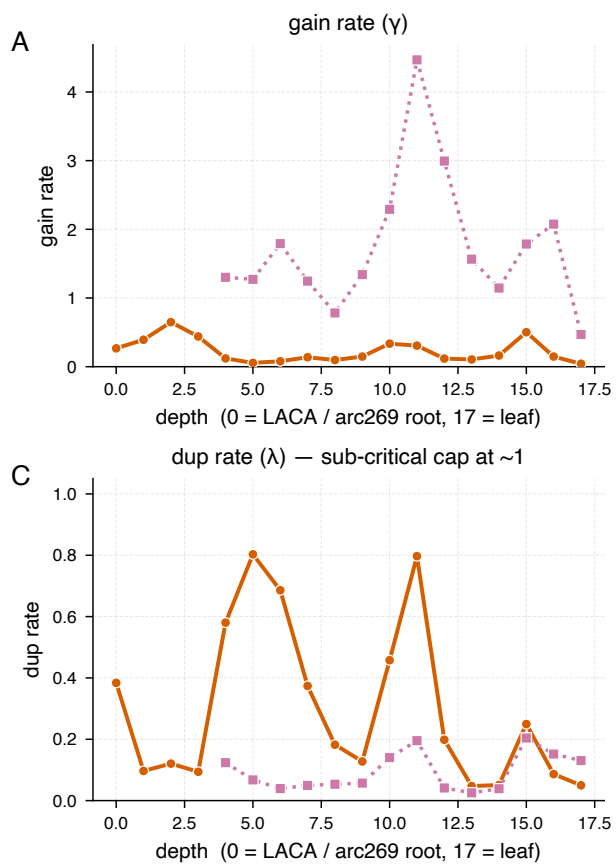


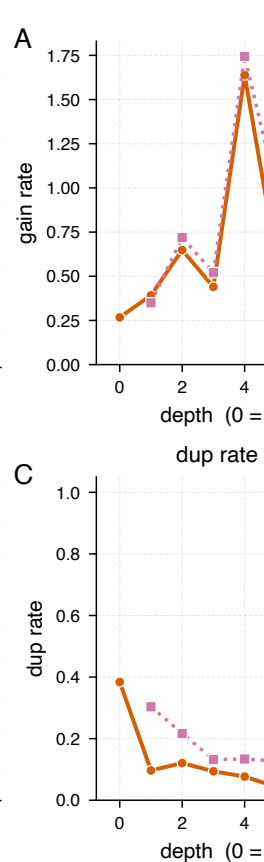
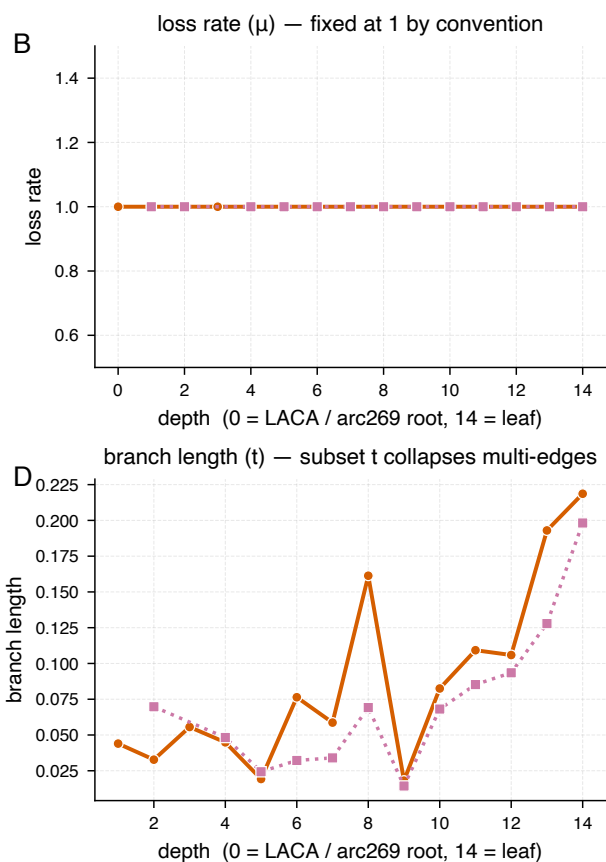
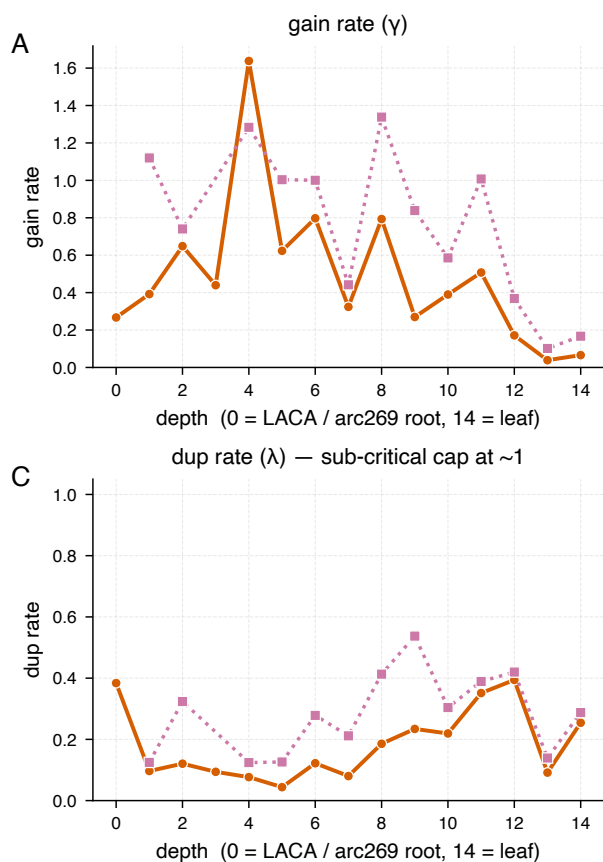
min=2



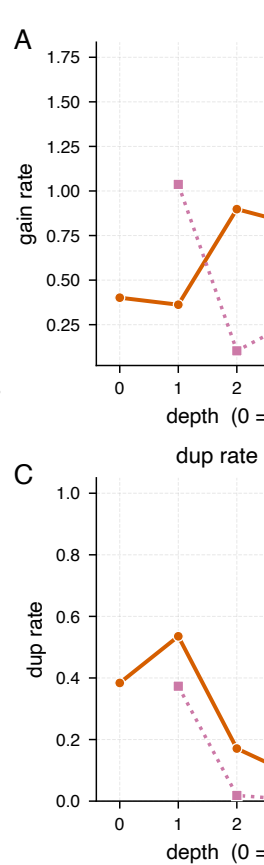
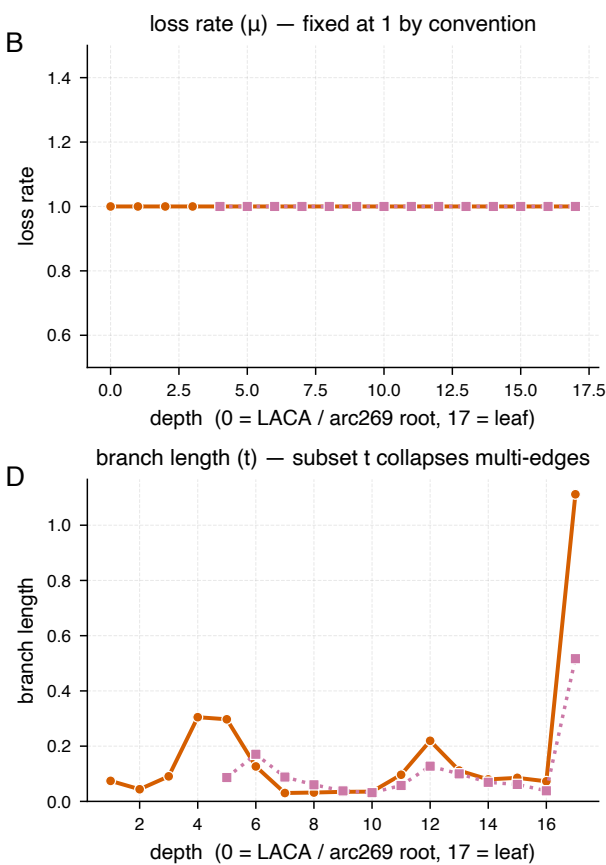
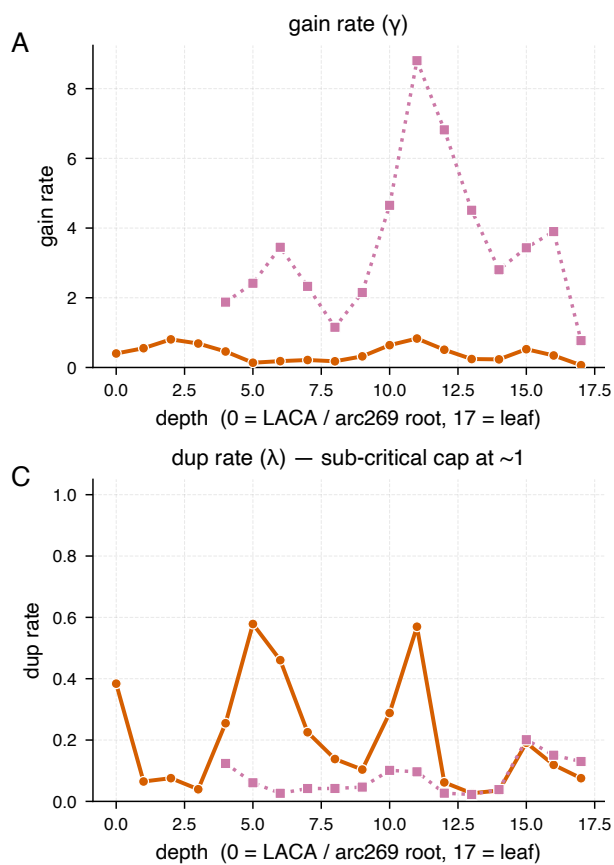


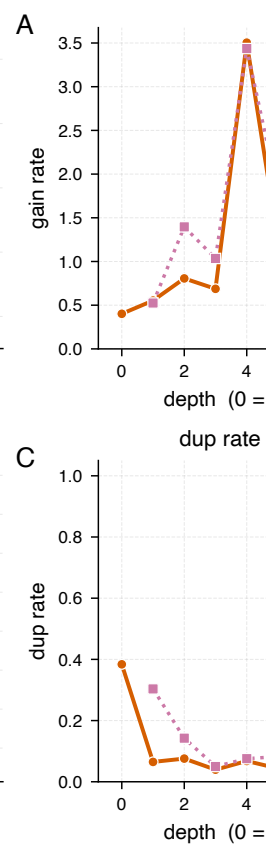
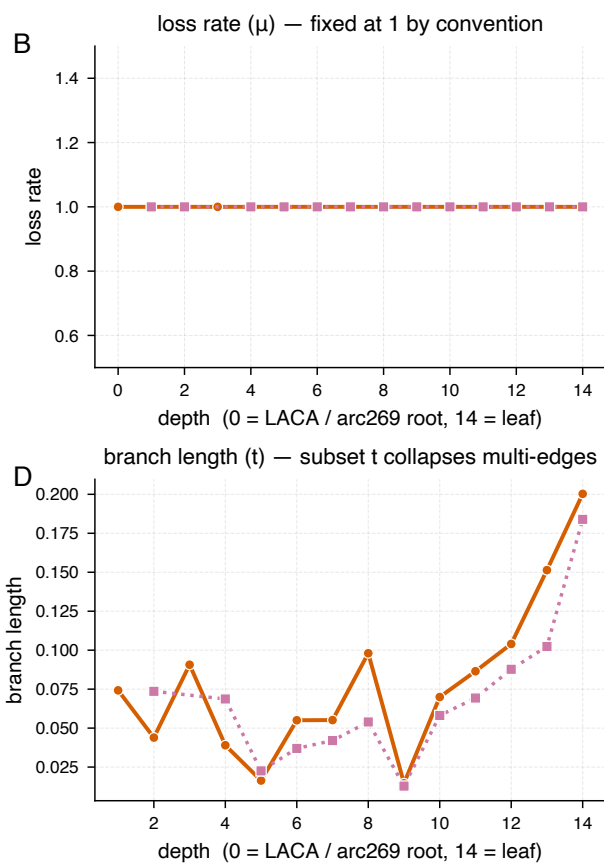
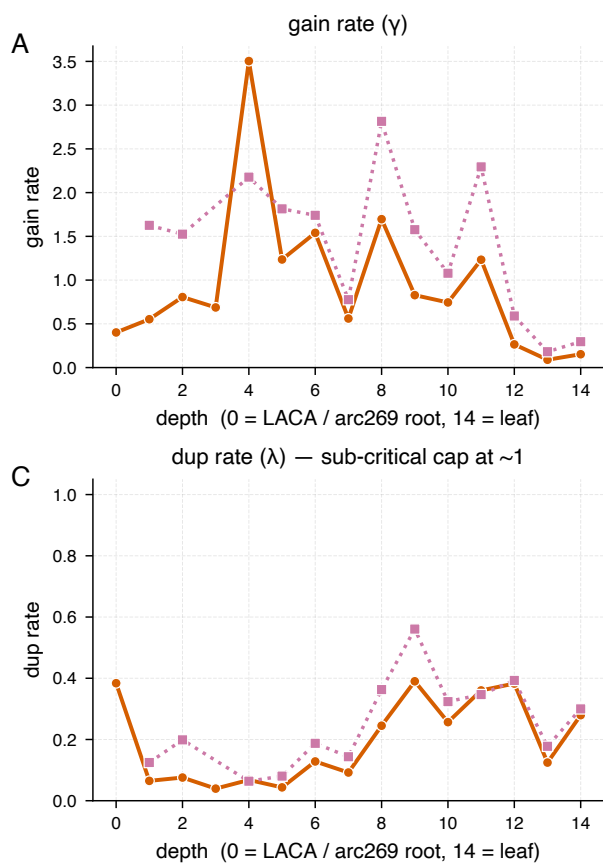
min=3



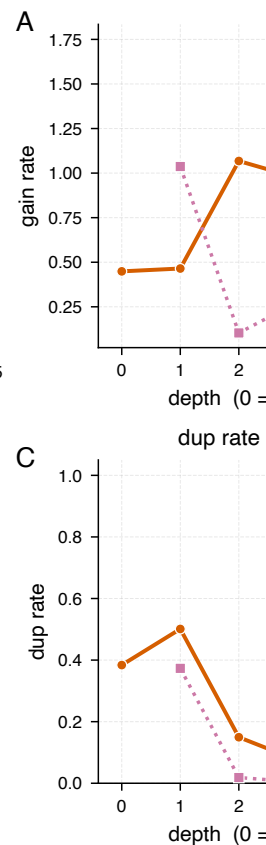
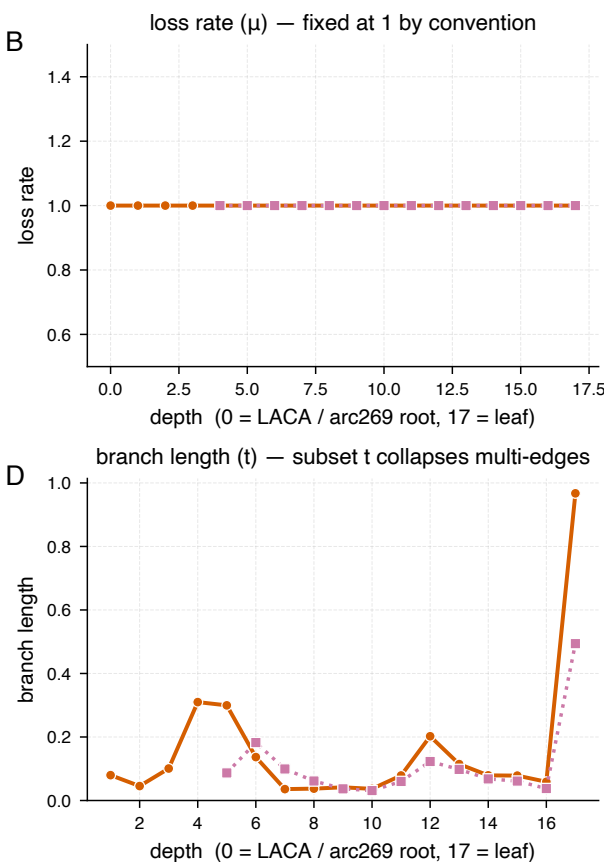
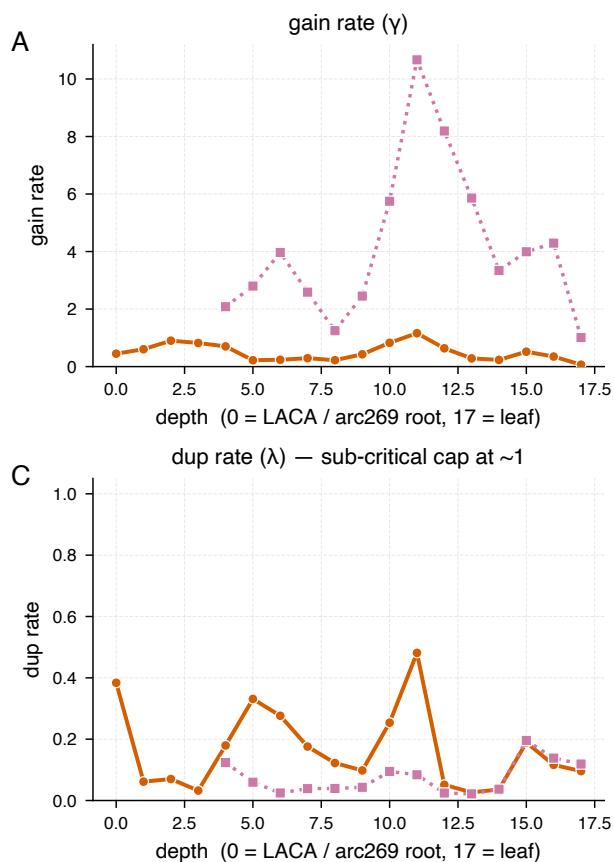


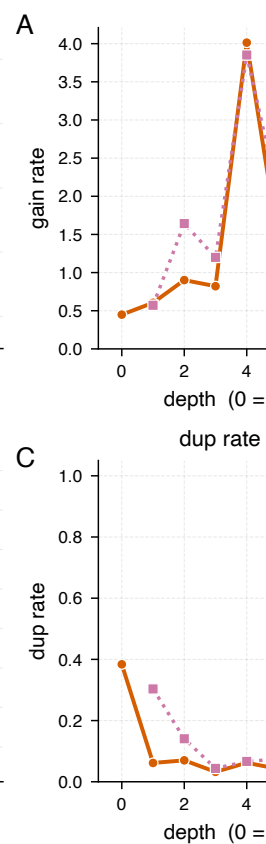
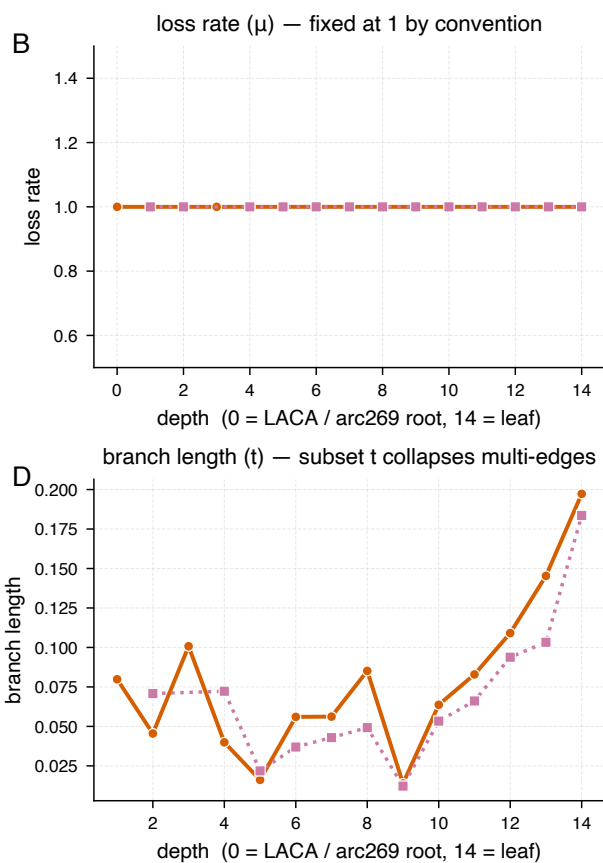
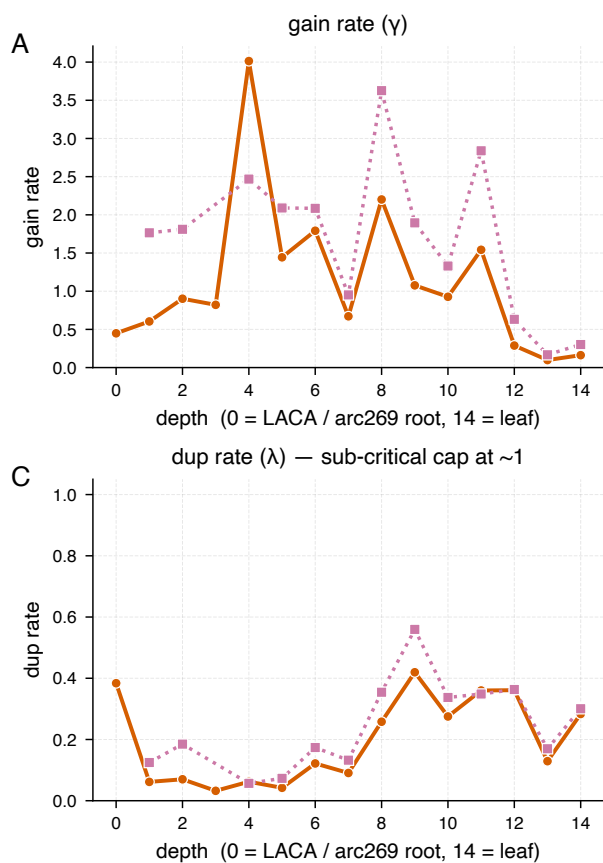
min=5



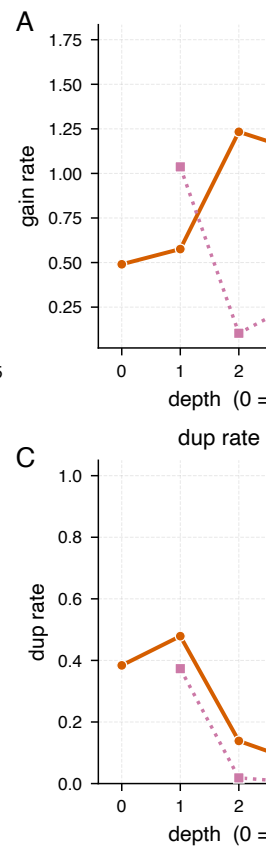
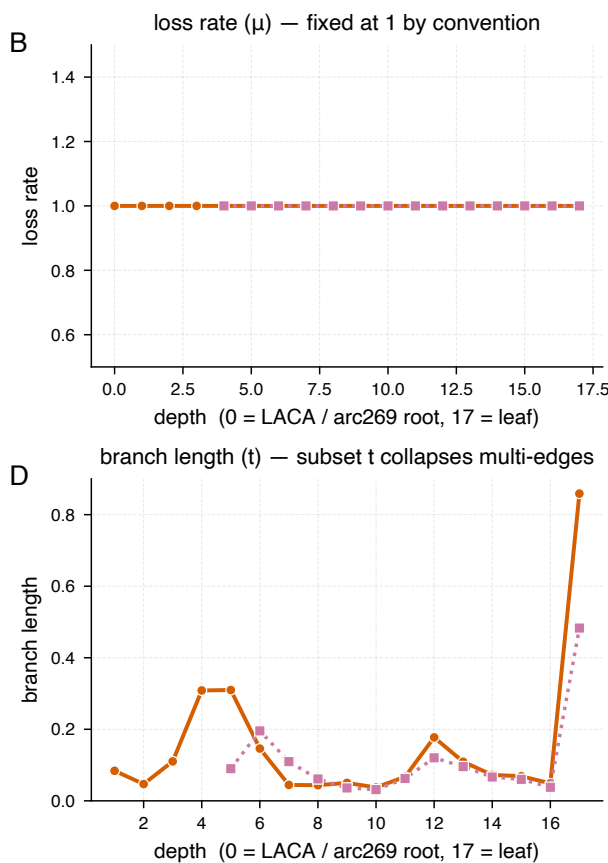
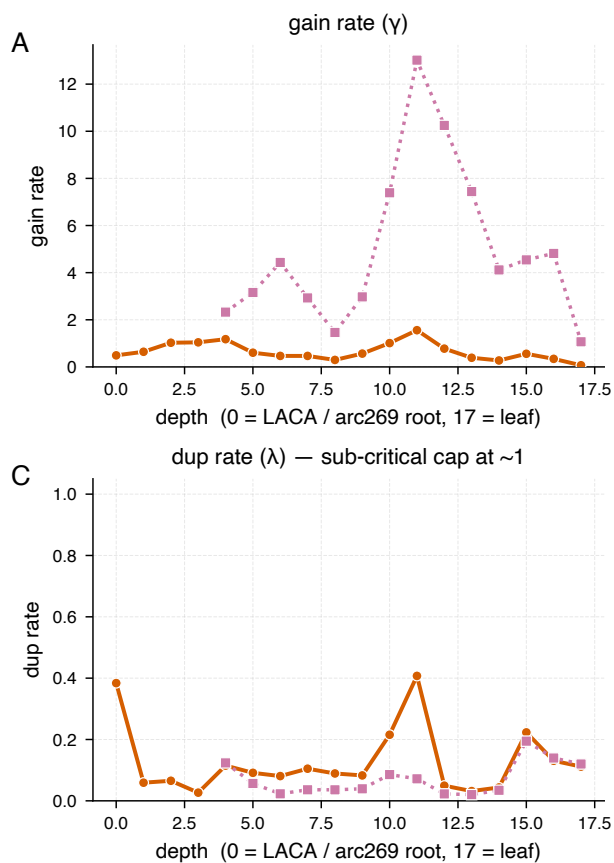


min=6





min=7 (most stringent)



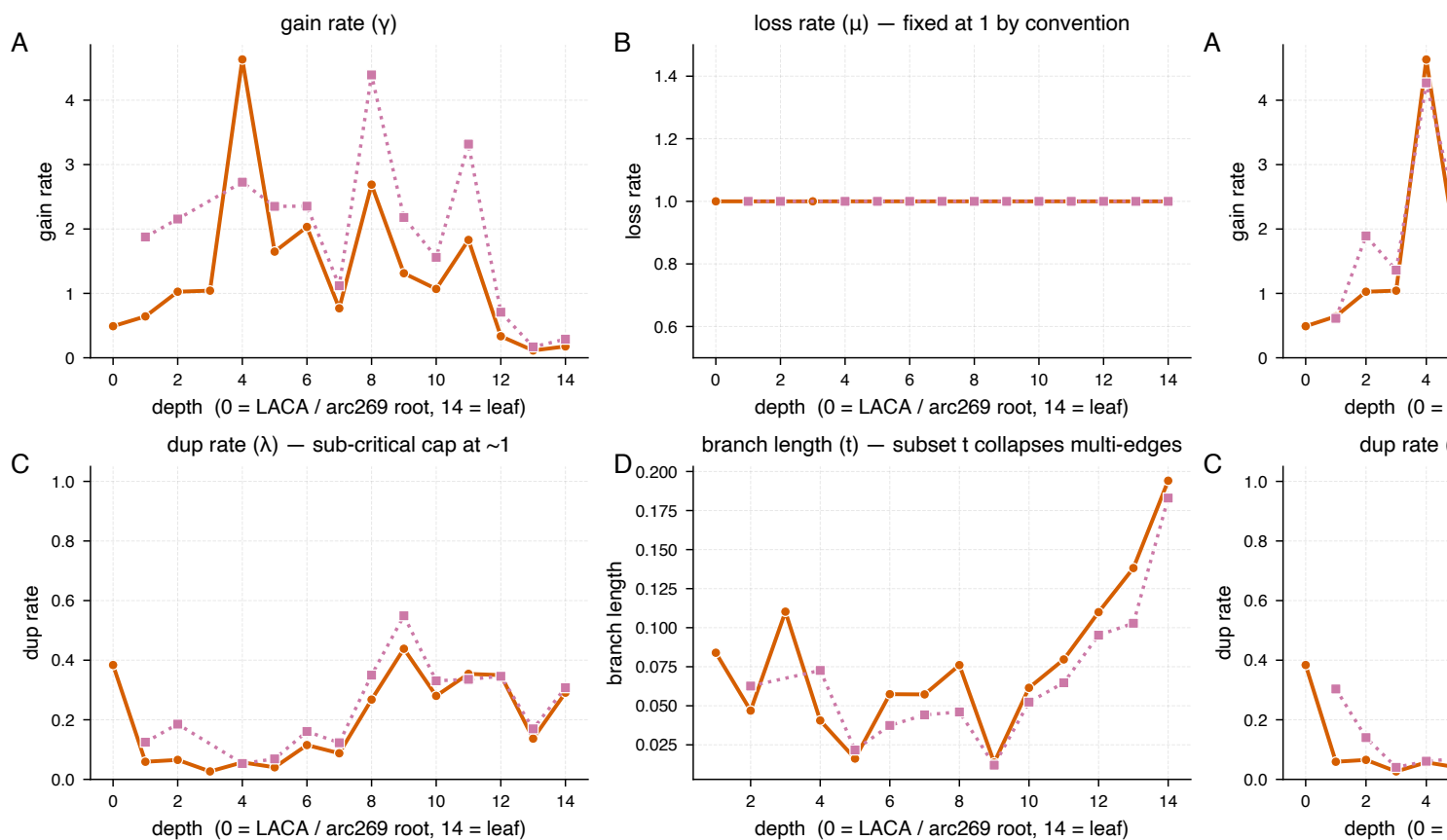
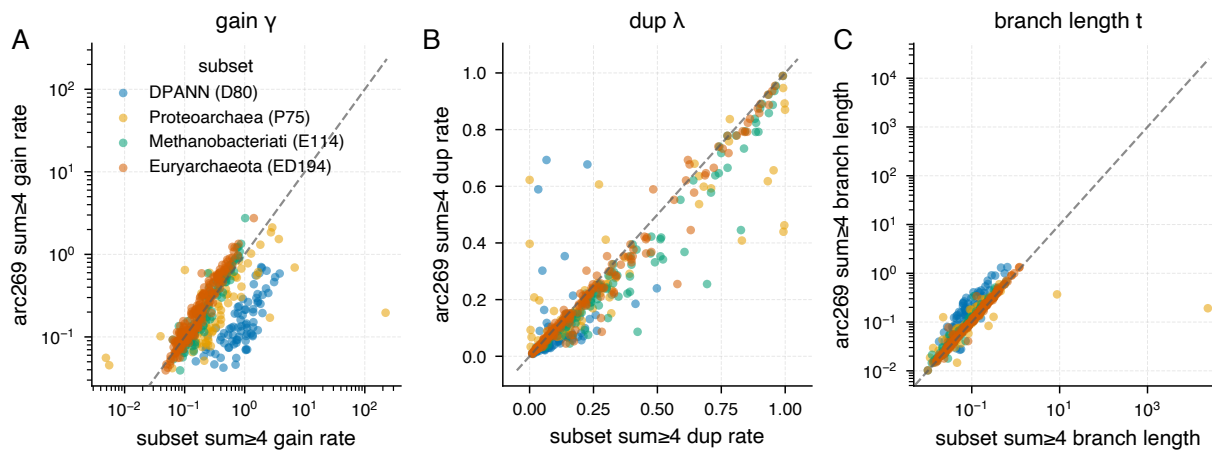


Figure 14. Per-branch GLD rate paths (γ / μ / λ / t) along each subset's root-to-leaf path, arc269 full-tree vs subset fit, at every filter min N {1..7}.

Pooled per-branch rate scatter (across all 4 subsets, per min) Each marker is one LCA-matched internal-node triple; colour codes the source subset (blue=dpann80, orange=proteo75, green=eury114, red=ed194). Diagonal $y=x$ is dashed black. Log axes on γ and t (heavy tails); linear on λ (bounded sub-critical cap).



Other min levels

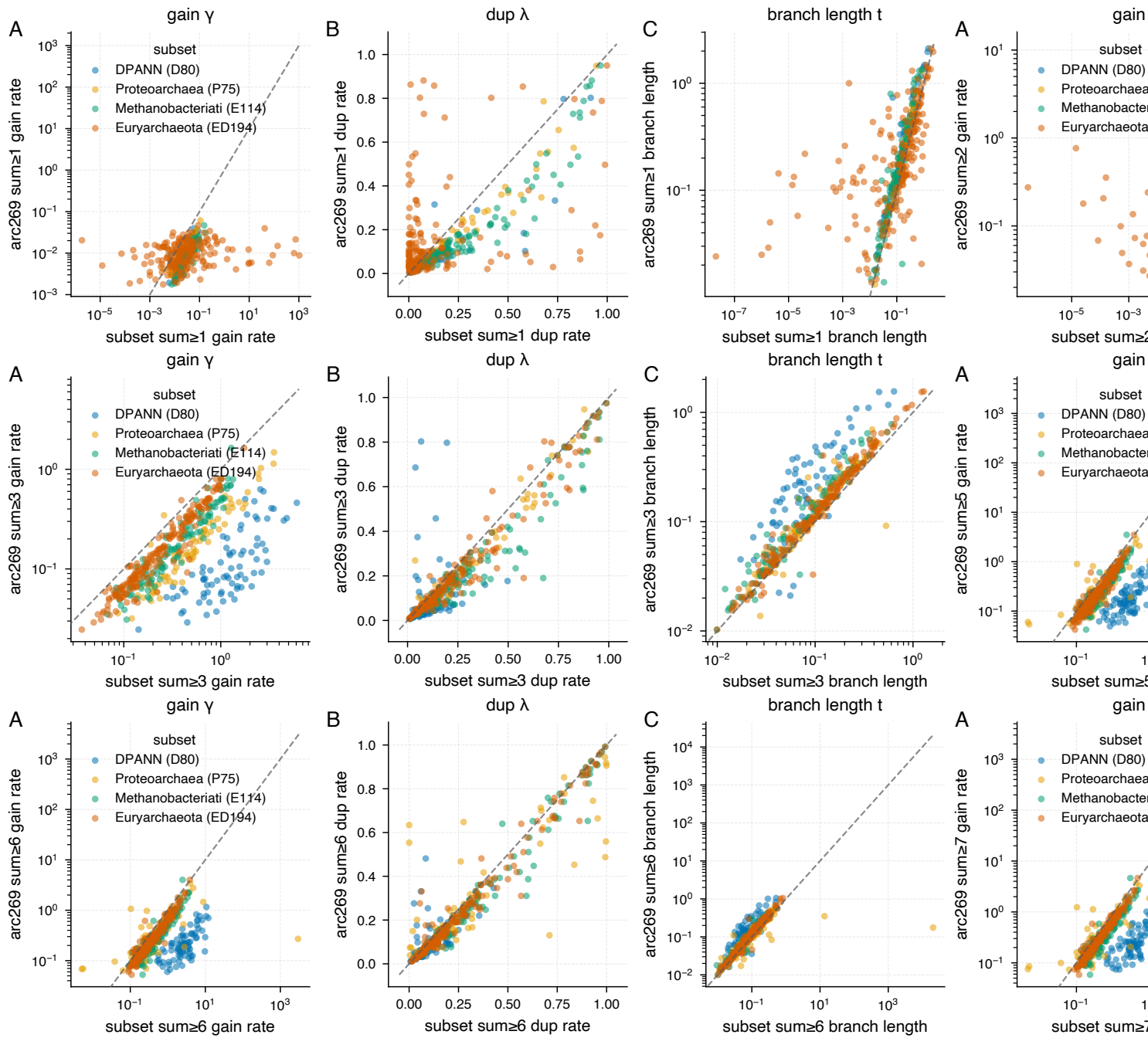


Figure 15. Pooled per-branch rate scatter (γ / λ / t) of arc269 vs subset rate at every LCA-matched internal node across all four subsets, at each filter min $N = \{1..7\}$.

Performance

Single-core vs multi-thread on M4 Max (12 P + 4 E = 16 cores)

Direct measurement on arc269 full-tree ($F=90,243$, $N=537$, $mc=1$ — the production case) at canonical rates ($\gamma=0.5$, $\lambda=1$, $t=1$) on the post-2026-05-19 fast kernel (commit dc7653c):

Table 26. Single-core vs multi-thread timings on arc269 full-tree (M4 Max, 16 cores).

num_threads	Forward LL	Gradient	Branch posteriors	Wall-time vs 1 thread
1	2,257 ms	5,523 ms	5,529 ms	1.00×
4	588 ms	1,429 ms	1,426 ms	3.84×
16 (P + E)	205 ms	504 ms	505 ms	11.0×

Near-linear up to 12 P-cores; the 4 E-cores add ~15 % on top. Versus the pre-2026-05-19 kernel (commit 68e1bbe, eury114 @ 8 threads): LL 890 ms → 205 ms, gradient 2,073 ms → 504 ms — the inside/outside refactor gives a 1.4–1.6× kernel-level speedup that compounds with the parallelism.

Speedup vs Csurös’ Java reference (M4 Max, same hardware)

Direct measurement on Williams2017 (F=5,378, N=119, mc=4 — Csurös’ canonical reference) plus two arc269 subsets, at canonical rates:

Table 27. Native (16-thread) vs Csurös’ Java timings per workload, with speedup, on the same M4 Max hardware.

Workload	Native (16-thread)	Java	Speedup
Forward LL (Williams2017, F=5,378)	9.5 ms	227 ms	24×
Branch posteriors (Williams2017)	18.4 ms	934 ms	51×
Forward LL (arc269 halo51, F=256)	1.2 ms	36 ms	30×
Forward LL (arc269 thermoplasmatota28)	0.3 ms	16 ms	47×
Branch posteriors (arc269 halo51)	2.4 ms	197 ms	81×
End-to-end MAP Brownian =1 (arc269, F=90,243)	~27 min	<i>Java cannot fit at -Xmx32g</i>	new mode

Native runs eury114 (N=227, F=7,335, max W=712) and ed194 (N=387, F=8,855, max W=712) in well under 100 ms per gradient call; Java cannot complete either at `-Xmx32g`. arc269-subset comparisons reach 47× (LL) and 81× (branch posteriors) on sparser trees because libdispatch’s per-family parallelism scales better when per-family work is smaller. Like-for-like comparison: same algorithm (Csurös 2021 [arXiv v1](#) Theorem 11 — Csurös 2022 TPB Theorem 12 + Csurös 2026 SI Thms 3–5), same hardware (M4 Max), same numeric tolerance (5×10^{-11} per-node).

Reproducible: [validation/benchmark_arc269_threads.py](#) and [validation/benchmark_arc269_threads.json](#).

Threading scaling on Apple silicon

`recount.native_backend` parallelises per-family work via libdispatch (Grand Central Dispatch). Default `num_threads=0` uses all online cores (P + E). Inspect the runtime topology:

```
>>> from recount.native_backend import cpu_topology
>>> cpu_topology()
{'ncpu': 16, 'physicalcpu': 16, 'perf_cores': 12, 'eff_cores': 4,
 'brand': 'Apple M4 Max'}
```

The 4 E-cores add ~10–15 % on top of the 12 P-cores; near-linear scaling up to 12 threads, slight diminishing returns past that.

Validation against Csurös’ Java

Every numerical capability that the native C backend exposes is validated against the bundled Java reference (`CountXXV.jar`):

Table 28. Each native C capability, its test against the Java reference, and the agreement reached.

Capability	Test	Agreement
Forward log-likelihood	<code>corrected_log_likelihood_native</code> vs Java on Williams2017 + 4 arc269 subsets	bit-for-bit, FP-sum floor 5×10^{-1}
Family conditioning at arbitrary $\Omega_{\min}$ (SI Thms 3–5)	<code>unobserved_logL0_native</code> vs Java for $\Omega_{\min} = 1, 2, 4$	bit-for-bit, 1 ULP
Per-branch posterior copies & families (observed)	<code>per_branch_stats_native</code> vs Java for all 4 focal subset clades	5×10^{-11} at every per-node value
L(0)-corrected root copies & families	end-to-end including the $F \cdot L(0)/(1-L(0))$ amplification at $\Omega_{\min} = 2$	5×10^{-11}
Analytical survival-rate gradient	<code>gradient_native</code> vs central finite-difference, Williams + all 4 subsets	median 1×10^{-10} relative, FP-sum floor 1×10^{-10}
Analytical L(0) gradient (Phase B)	<code>compute_L0_gradient_analytical</code> vs finite-difference for $\Omega_{\min} = 2, 4$	median 1×10^{-10} relative
LogisticShift K = 1, zero shifts	mixture LL identity collapses to base-GLD LL	exact, diff = 0.0
Brownian-prior gradient	<code>brownian_prior_native</code> vs central finite-difference	max rel 10^{-10}

See [VALIDATION.md](#) and [VALIDATION_FOCAL.md](#) §3 for per-node diff dumps. The pytest suite (pytest tests/) re-runs every check above in ~2 s.

Reproduction accuracy on the focal subset clades

Loading the bundled `.countxml.gz` rates from Csurös 2026 and recomputing per-branch posteriors via the C backend matches the Java output from [miklosc/Count](#) to 5×10^{-11} at every per-node copy/family/event count:

Table 29. Per-node reproduction accuracy vs Java for each focal subset clade and Williams2017.

Dataset	F families	leaves	nodes	per-node max diff vs Java
DPANN (D80)	3,034	80	159	5×10^{-11}
Proteoarchaea (P75)	5,179	75	149	5×10^{-11}
Methanobacteriati (E114)	7,335	114	227	5×10^{-11}
Euryarchaeota (ED194)	8,855	194	387	5×10^{-11}
Williams2017 (LACA)	5,378	60	119	5×10^{-11}

Repository tour

```

recount/                # Python package
  gld.py                 # NumPy reference (forward + outside + gradient)
  torch_backend.py       # PyTorch reference (forward + autograd grad)
  native_backend.py      # production: ctypes wrappers for the native C dylib
  ml.py / logistic_shift.py / brownian_prior_*.py
native/                  # C backend (Apple Accelerate vForce + libdispatch)
  src/                   # forward, outside, gradient, brownian_prior, ...
  include/               # public + internal headers
  test/                  # standalone C test (test_4leaf)
  Makefile               # auto-detects Apple Silicon / Intel Mac / Linux
validation/              # one-shot drivers + canonical queue + plot pipeline
  ml.py / map.py / brownian_extend_ml.py
  queue_full_subset_run.sh + QUEUE_INSTRUCTIONS.md
  reproduce.py           # Csurös' published rates → bit-perfect per-node check
  subclade_ancestor_comparison.py / brownian_split_path_plots.py / ...
  outputs/<fit-dir>/<ds>_{branches.csv, countxml.gz, summary.json, final_rates.npz}
docs/                    # specifications, proofs, and the PDF build
  brownian_prior.tex     # TKP autocorrelated-rate prior + gradient derivation
  boundary_identifiability.tex
  PER_FAMILY_PRESENCE.md
  build_readme_pdf.py    # regenerates README.{tex,pdf} from README.md
  README.tex / README.pdf # LaTeX / PDF rendering of this README (generated)
tests/                   # pytest (~30 checks)

```

A typeset LaTeX/PDF version of this README is kept at [docs/README.pdf](#). It is generated, not hand-written: after editing `README.md`, run `python3 docs/build_readme_pdf.py` to rebuild `docs/README.tex` and `docs/README.pdf` (the build uses the vector PDF version of every plot). That keeps the two in sync — the Markdown README is always the source of truth.

Per-family per-node presence tables

For any saved fit, you can produce two $F \times N$ matrices on demand:

Table 30. The two per-family per-node $F \times N$ matrices a saved fit can produce, with each element’s meaning.

matrix	element	meaning
<code>present[f, v]</code>	$P\{ _v = 1 \text{ given data_f, fitted rates } \}$	posterior probability family f is present at node v
<code>copies[f, v]</code>	$E[_v \text{ given data_f, fitted rates }]$	posterior expected copy number for f at v

These are intentionally **not** written by the standard fit pipelines ($F \times N$ is heavy on disk — arc269 is ~21 MB compressed NPZ per matrix). Generate explicitly:

```
# Compute for every dataset whose rates are present in a fit directory:
PYTHONPATH=. python3 validation/per_family_presence.py --all \
    --fit-dir validation/outputs/brownian_arc269
```

Recipe + query patterns: [docs/PER_FAMILY_PRESENCE.md](#).

Low-level Python API

For research / scripting, build a tree + rates by hand:

```
import numpy as np
from recount import Tree, GLDRates, corrected_log_likelihood, gradient_survival

# Tree: ((A:0.5,B:0.3):0.2,(C:0.4,D:0.6):0.1);
# Indexing: leaves first, parent index > child index, root has parent -1.
tree = Tree(parent=np.array([4, 4, 5, 5, 6, 6, -1]),
            leaf_names=["A", "B", "C", "D"])

rates = GLDRates(
    tree=tree,
    gain    = np.array([0.10, 0.20, 0.15, 0.25, 0.30, 0.18, 0.50]),
    loss    = np.array([1.00] * 7),
    dup     = np.array([0.40, 0.30, 0.50, 0.20, 0.45, 0.35, 0.00]),
    length  = np.array([0.50, 0.30, 0.40, 0.60, 0.20, 0.10, np.inf]),
)
profiles = np.array([
    [1, 0, 1, 1],
    [2, 0, 0, 1],
    [1, 1, 1, 1],
    [3, 2, 1, 0],
], dtype=np.int64)

ll = corrected_log_likelihood(tree, rates, profiles, min_copies=1)
g = gradient_survival(tree, rates, profiles, min_copies=1)
# g has shape (3 * num_nodes,), indexed as 3*v + {GAIN=0, LOSS=1, DUP=2}

PyTorch path (autograd through compute_survival_params_t):

import torch
from recount.torch_backend import corrected_log_likelihood_t, gradient_autograd

dt = torch.float64
gain    = torch.tensor([0.10, 0.20, 0.15, 0.25, 0.30, 0.18, 0.50], dtype=dt)
loss    = torch.tensor([1.0] * 7, dtype=dt)
dup     = torch.tensor([0.40, 0.30, 0.50, 0.20, 0.45, 0.35, 0.00], dtype=dt)
length  = torch.tensor([0.50, 0.30, 0.40, 0.60, 0.20, 0.10, float("inf")], dtype=dt)
profiles_t = torch.tensor(profiles, dtype=torch.long)

ll      = corrected_log_likelihood_t(tree, gain, loss, dup, length,
                                   profiles_t, min_copies=1)
grads = gradient_autograd(tree, gain, loss, dup, length,
                          profiles_t, min_copies=1)
# grads is a dict {'gain', 'loss', 'dup', 'length'} of per-node (ln L*)/rate
```


Model

The GLD process on each edge of the rooted tree is a linear birth-death process with constant per-copy duplication rate λ , per-copy loss rate μ , and copy-independent gain rate γ . Closed-form transition probabilities on an edge of length t give per-edge probability parameters p (loss prob), q (duplication parameter), and either r (Poisson gain rate, if $\gamma=0$) or ρ (Pólya / NegBinomial shape, if $\gamma>0$).

The “survival” parameterization $(p, q, r / \rho)$ implicitly marginalizes out copies whose entire lineage dies before reaching any leaf, which keeps the recursion stable. The observation-bias correction (`min_copies={0,1,2}`) conditions the likelihood on observing at least that many copies per family — the typical setting for gene-family-count data is `min_copies=1` (no empty families) or `min_copies=2` (no empty and no singleton families).

Root prior: the root edge has length $t = \infty$ (by convention), which forces $p_{\text{root}} = 1$ (any hypothetical “pre-root” copy goes extinct before the root with probability one). So the root marginal of $_R$ collapses to the *gain PMF on the root edge* — $\text{Poisson}(_R)$ when $_R = 0$ (the Poisson branch) or $\text{Pólya/NegBinomial}(_R, q_R)$ when $_R > 0$. This matches Csűrös’ Java (`count.model.TreeWithLogisticParameters`) and the documented convention in Csűrös 2026 SI §A. The “root copies” quoted across this README is the $L(0)$ -corrected posterior expectation $E[_R \mid \text{profile sum } \Omega_{\min}]$ under that root prior — equivalently, the root entry of `count.model.AncestorPosteriors.Profile.getNodePosteriors(root)` in the Java reference, averaged over families with the $L(0)$ re-normalisation baked in.

Gradient

The analytical gradient is computed in the survival parameterization $(\ln L^*) / (p, q, r / \rho)$ via Csűrös (2021) [arXiv v1 Theorem 11](#) (Csűrös 2022 TPB Theorem 12), applied to the corrected log-likelihood of Corollary 10 (TPB Corollary 11), in native C. For `min_copies = 2` (arbitrary- $\Omega_{\min}$ unobserved-profile correction), the analytical $L(0)$ gradient is added via `recount.unobserved_outside.compute_L0_gradient_analytical` (Phase B port of Java `LogGradient.PosteriorStatistics.getLogSurvivalGradient`, validated to FD precision floor; see [VALIDATION.md](#)). The PyTorch backend’s `gradient_autograd` provides the same gradient by reverse-mode AD for forward-only diagnostic use.

Indexing convention

Trees are indexed with **leaves first** and **parent index > child index**:

- `0..num_leaves-1` are leaves
- `num_leaves..num_nodes-1` are internal nodes
- The root is `num_nodes - 1`
- `parent[v] > v` for every non-root v , and `parent[root] == -1`

This convention (inherited from `count.ds.IndexedTree` in the Java source) makes the bottom-up recursion `for v in range(N): ...` a valid post-order traversal, and `for v in range(N-1, -1, -1): ...` a valid pre-order.

Citing

If you use this package in academic work, please cite the paper that derives the algorithms:

M. Csűrös. “Gain-loss-duplication models on a phylogeny: exact algorithms for computing the likelihood and its gradient.” 2021. [arXiv:2107.11440](#).

The original Count Java implementation and the focal datasets used in this repository are described in:

M. Csűrös. “Reconstructing ancient genomes from gene counts: A robust likelihood framework with sampling bias correction.” *PNAS* 123(19): e2537812123 (2026). [doi:10.1073/pnas.2537812123](#)

The Brownian-prior mode (default for new fits in this repo) implements the **Thorne–Kishino–Painter** autocorrelated-rate model:

J. L. Thorne, H. Kishino, I. S. Painter. “Estimating the rate of evolution of the rate of molecular evolution.” *Molecular Biology and Evolution* 15(12):1647–1657 (1998).

Section 1, **Copy numbers cannot separate transfer from inheritance**, reproduces Figure 2 (panels A–C) under fair use from:

T. A. Williams, A. A. Davin, L. L. Szánthó, et al. “Phylogenetic reconciliation: making the most of genomes to understand microbial ecology and evolution.” *The ISME Journal* 18(1):wrae129 (2024). [doi:10.1093/ismejo/wrae129](https://doi.org/10.1093/ismejo/wrae129).

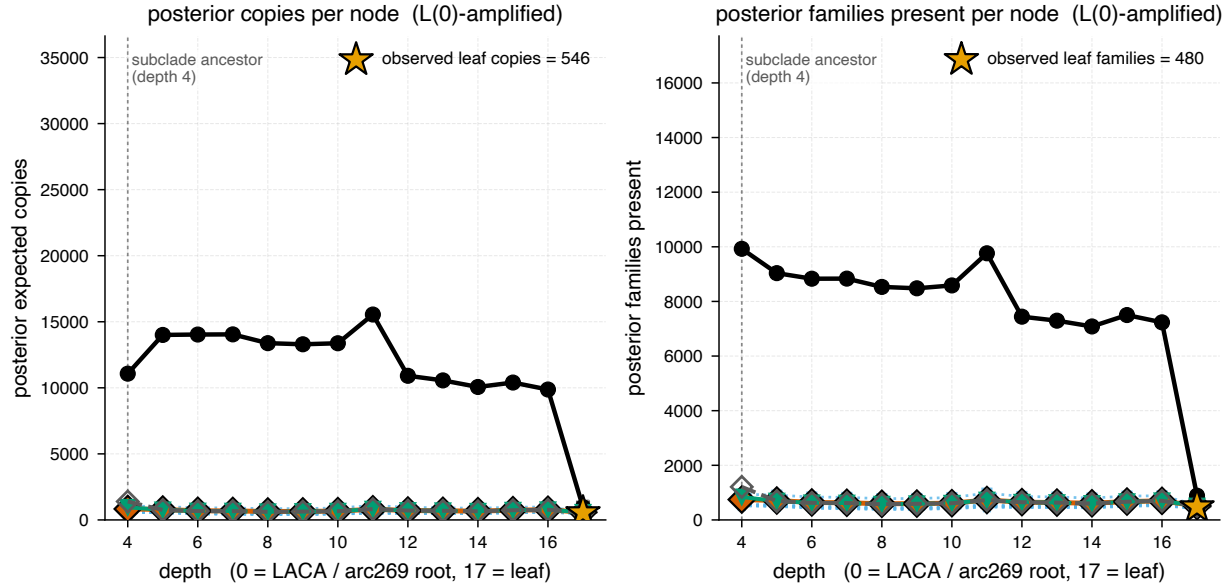
L(0)-corrected trajectories

This section reproduces every plot in **Diagnostic plots** (Sets 1, 1.5, 2, 3) with the $F^* = F/(1-L(0))$ amplification applied per line. The amplification accounts for ancestral families that, under the fitted rate process and the survival conditioning, are expected to have produced sub- $\Omega_{\min}$ profiles and would not appear in the dataset. For fits with $\Omega_{\min} > 1$ (Csurös subset-only ML; per-subset Brownian and Brownian-extend at min=4) the amplification is taken from the ***_corrected** columns in branches.csv, which implement Csurös’ SI Theorems 3–5 (Pn-tensor recursion) bit-for-bit. For mc=1 fits (arc269 canonical, arc269 sum N filtered, subset full-complement) the same Pn-tensor pipeline is not wired up; the multiplicative approximation observed $\times 1/(1-L(0))$ is applied at internal nodes, with leaf values left as observed.

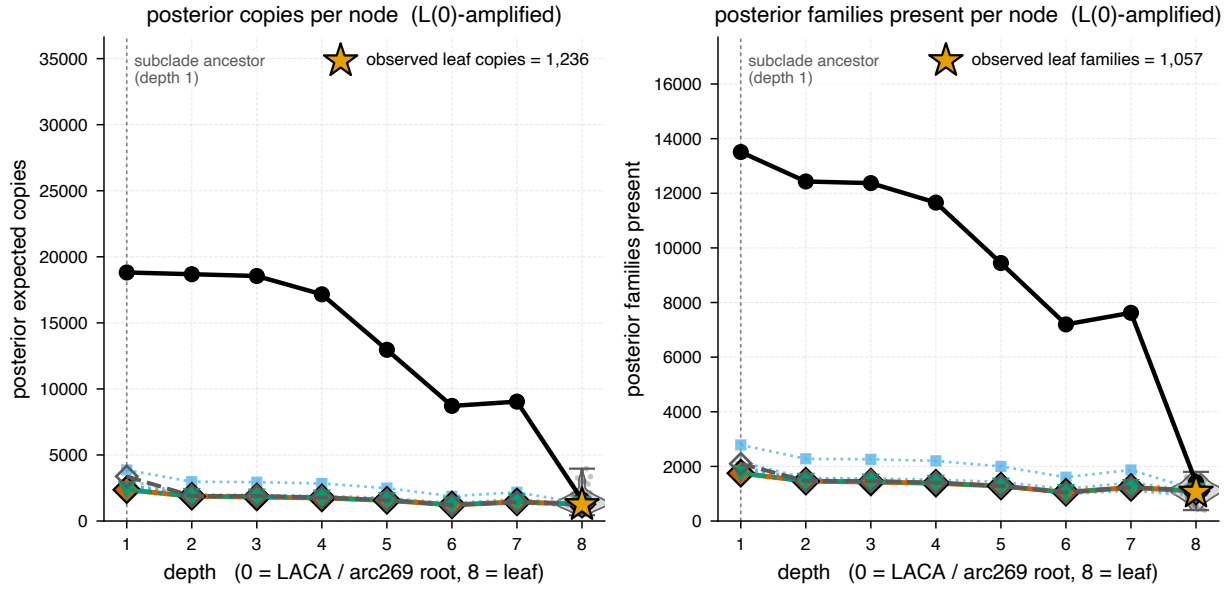
L(0) values across these fits range from 0.01 (sum 6 / sum 7) up to 0.794 (arc269 canonical, mc=1, no filter), giving amplification factors from 1.01 to 4.85.

Set 1 — L(0)-corrected

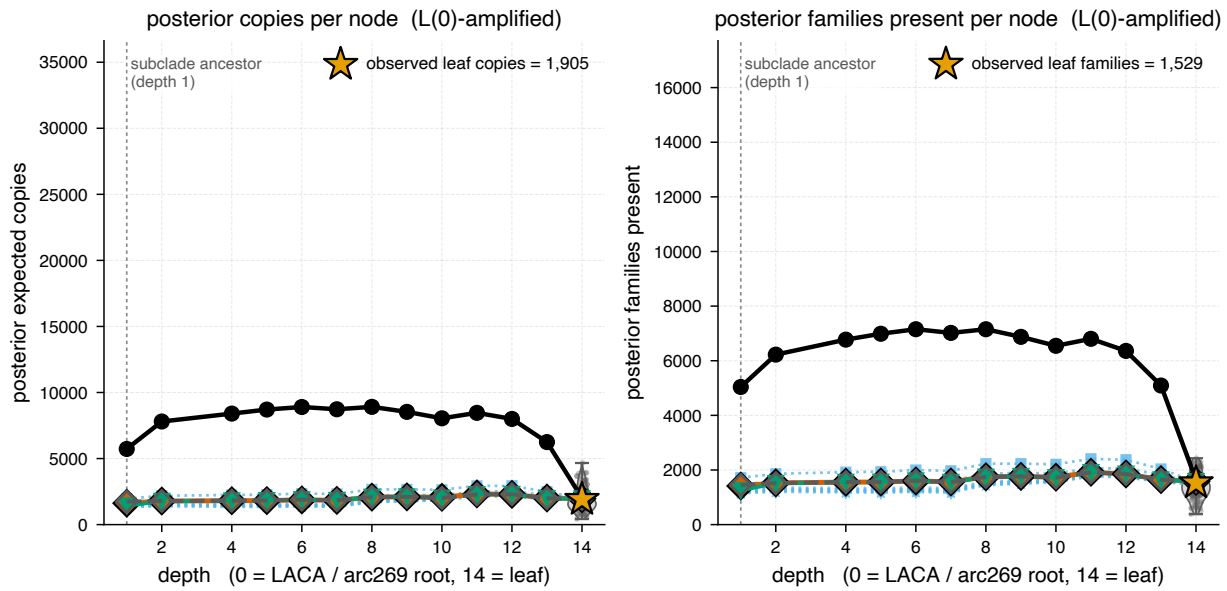
DPANN (D80) · Set 1 · arc269 root to median-fm leaf ‘1974445’ (F = 3,034)



Proteoarchaea (P75) · Set 1 · arc269 root to median-fm leaf '2058097' (F = 5,179)



Methanobacteriati (E114) · Set 1 · arc269 root to median-fm leaf '679901' (F = 7,335)



Euryarchaeota (ED194) · Set 1 · arc269 root to median-fm leaf '1961135' (F = 8,855)

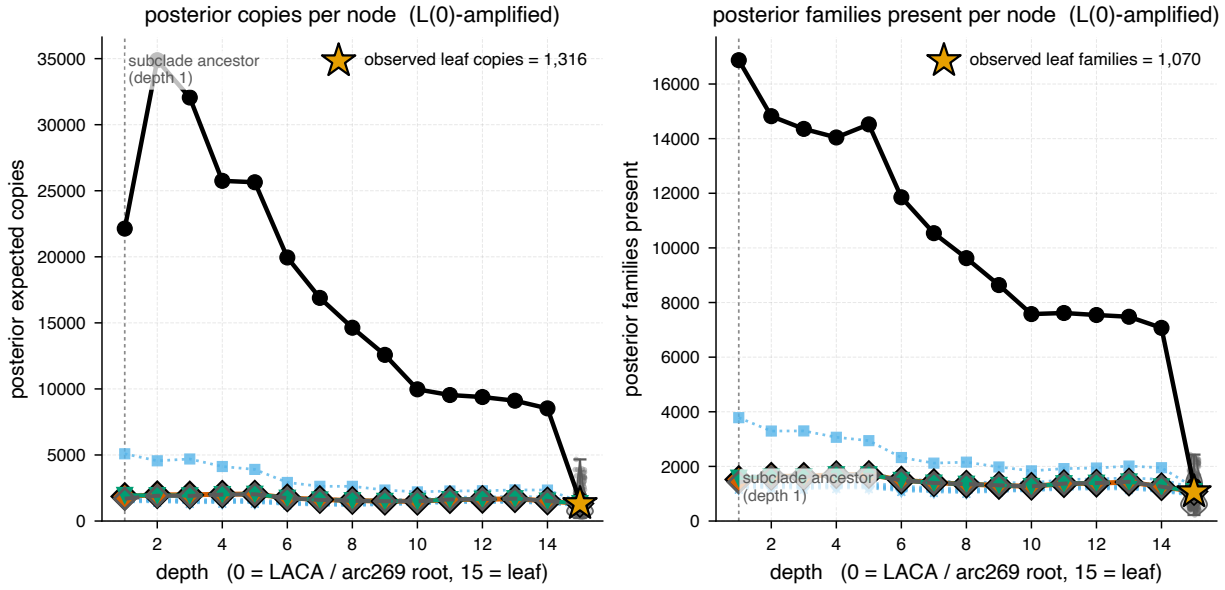
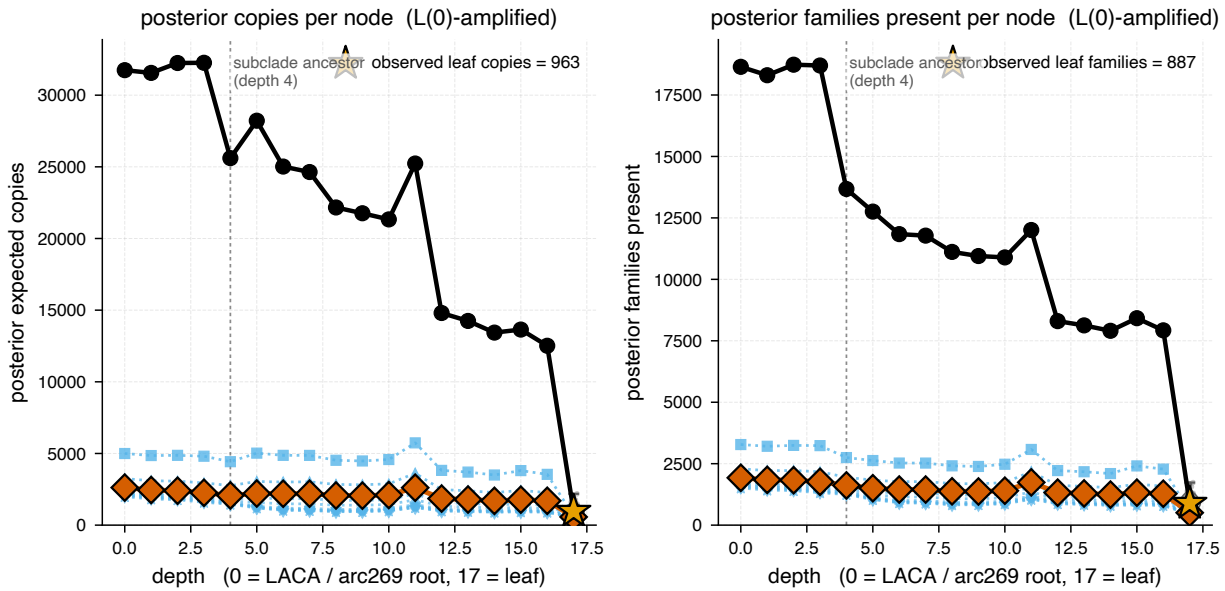


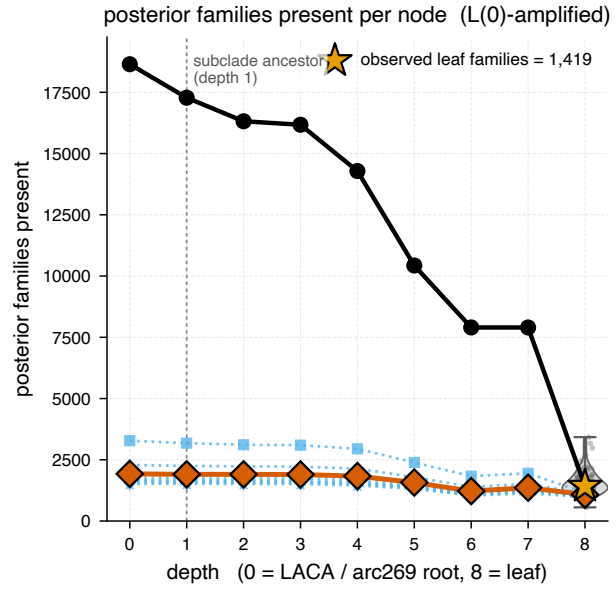
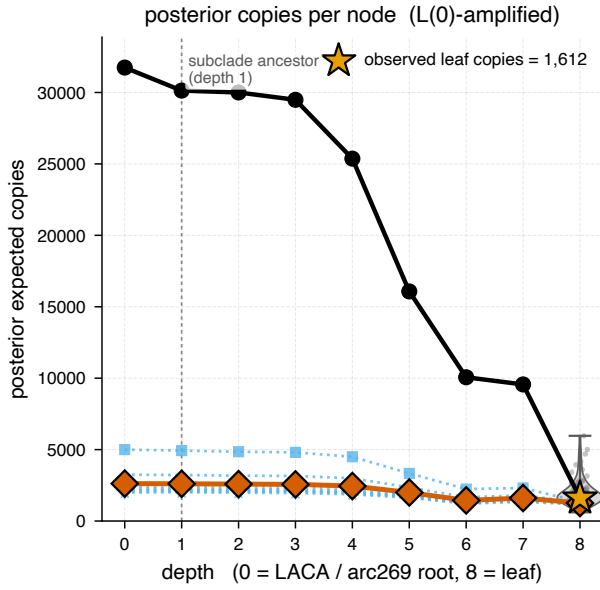
Figure 16. Set 1 (per-clade sum-filter sweep) root-to-leaf trajectories with the $F^* = F/(1-L(0))$ amplification applied per line.

Set 1.5 — L(0)-corrected

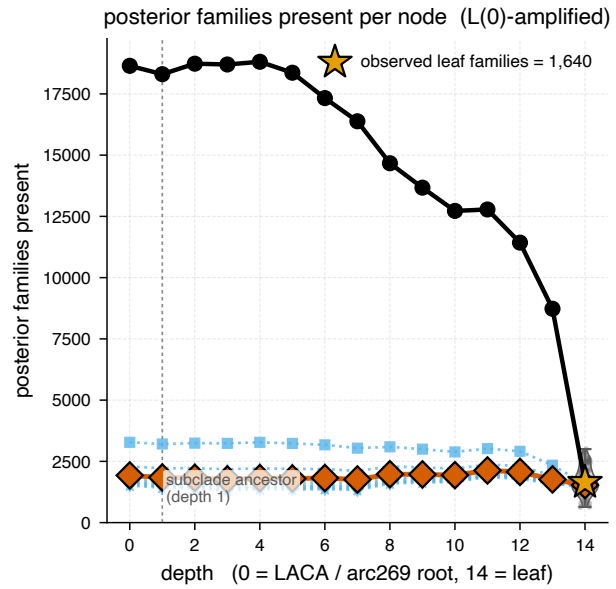
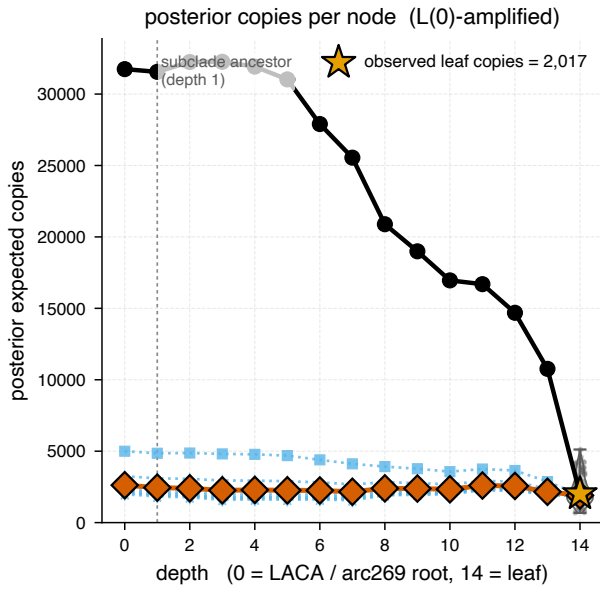
DPANN (D80) · Set 1.5 · arc269 root to median-fm leaf '1974445'



Proteoarchaea (P75) · Set 1.5 · arc269 root to median-fm leaf '2058097'



Methanobacteriati (E114) · Set 1.5 · arc269 root to median-fm leaf '679901'



Euryarchaeota (ED194) · Set 1.5 · arc269 root to median-fm leaf '1961135'

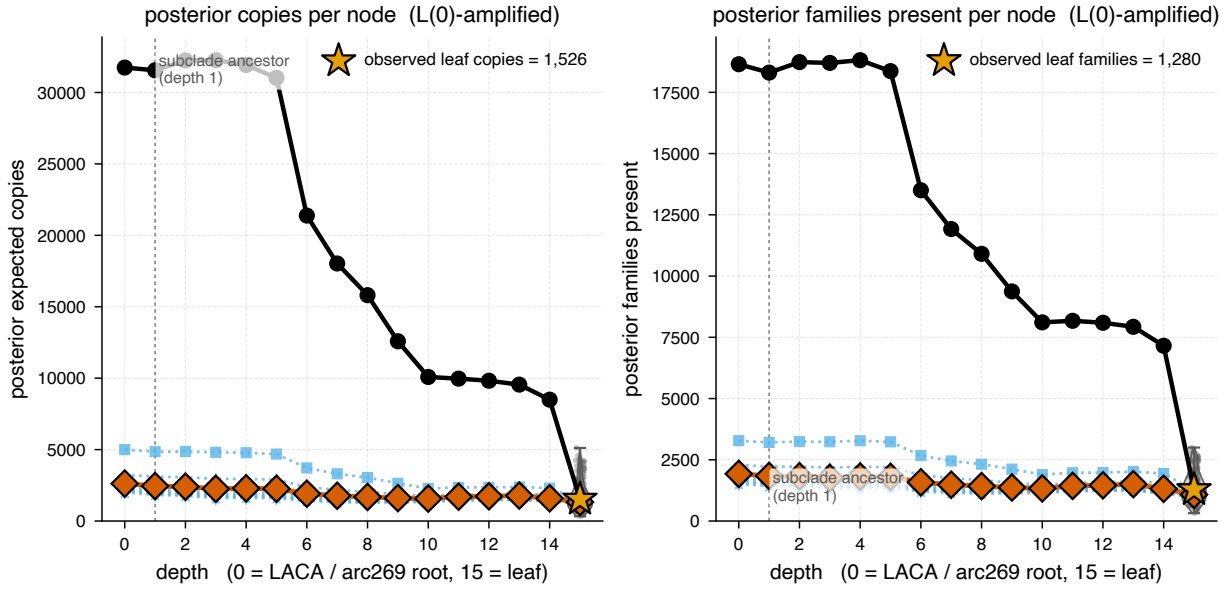
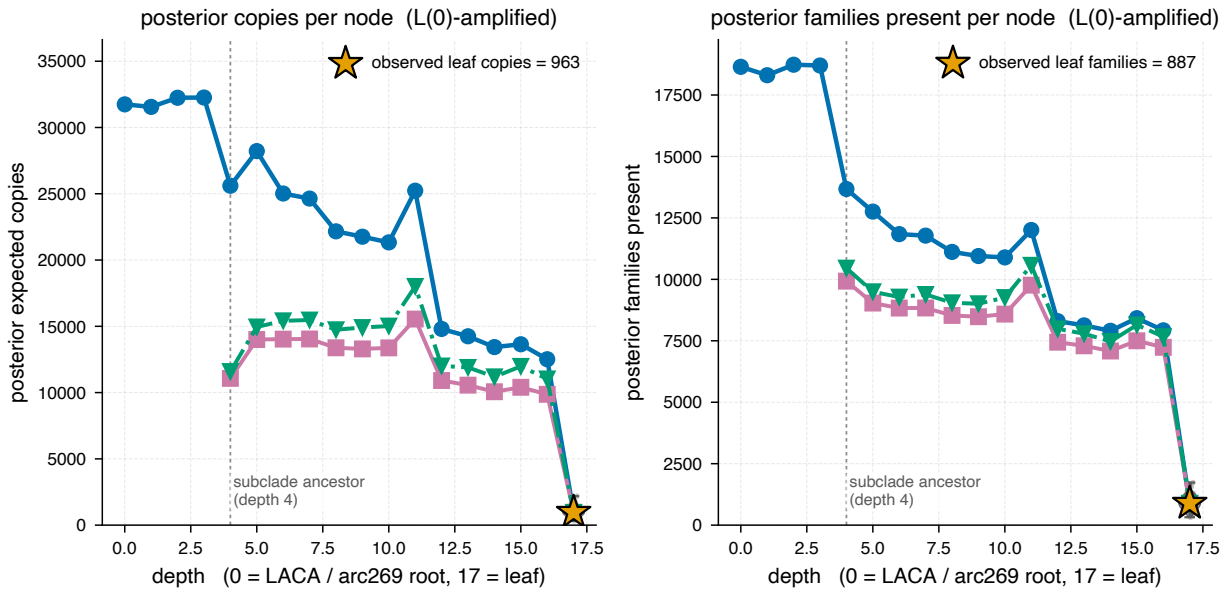


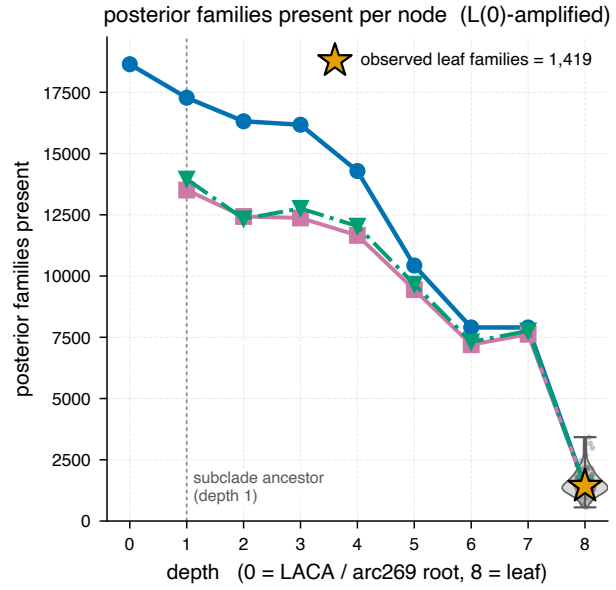
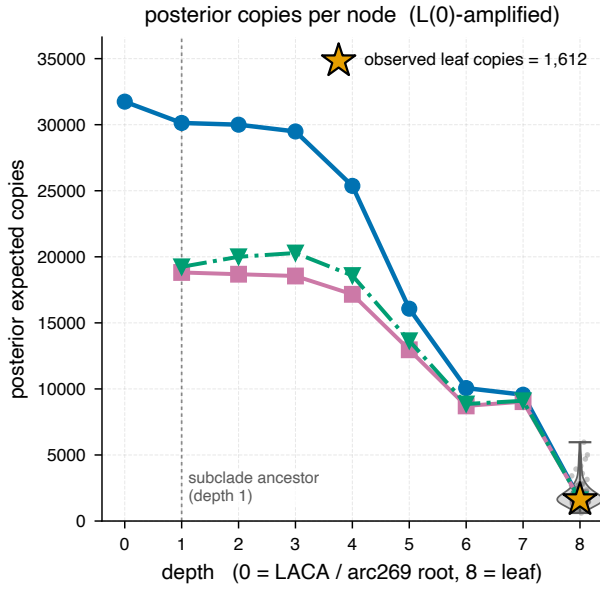
Figure 17. Set 1.5 (arc269 full-tree sum-filter sweep) root-to-leaf trajectories with the $F^* = F/(1-L(0))$ amplification applied per line.

Set 2 — L(0)-corrected

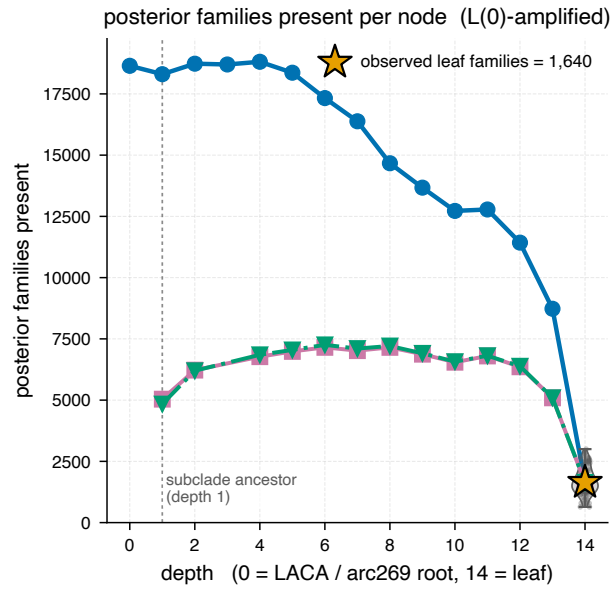
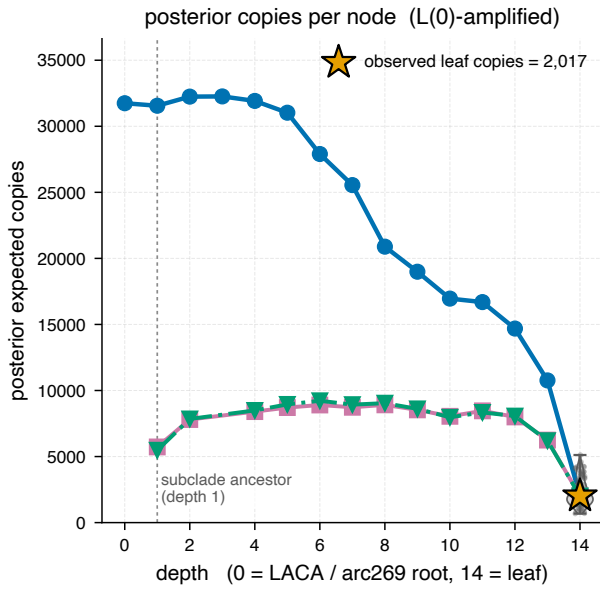
DPANN (D80) · Set 2 · arc269 root to median-fm leaf '1974445'



Proteoarchaea (P75) · Set 2 · arc269 root to median-fm leaf '2058097'



Methanobacteriati (E114) · Set 2 · arc269 root to median-fm leaf '679901'



Euryarchaeota (ED194) · Set 2 · arc269 root to median-fm leaf '1961135'

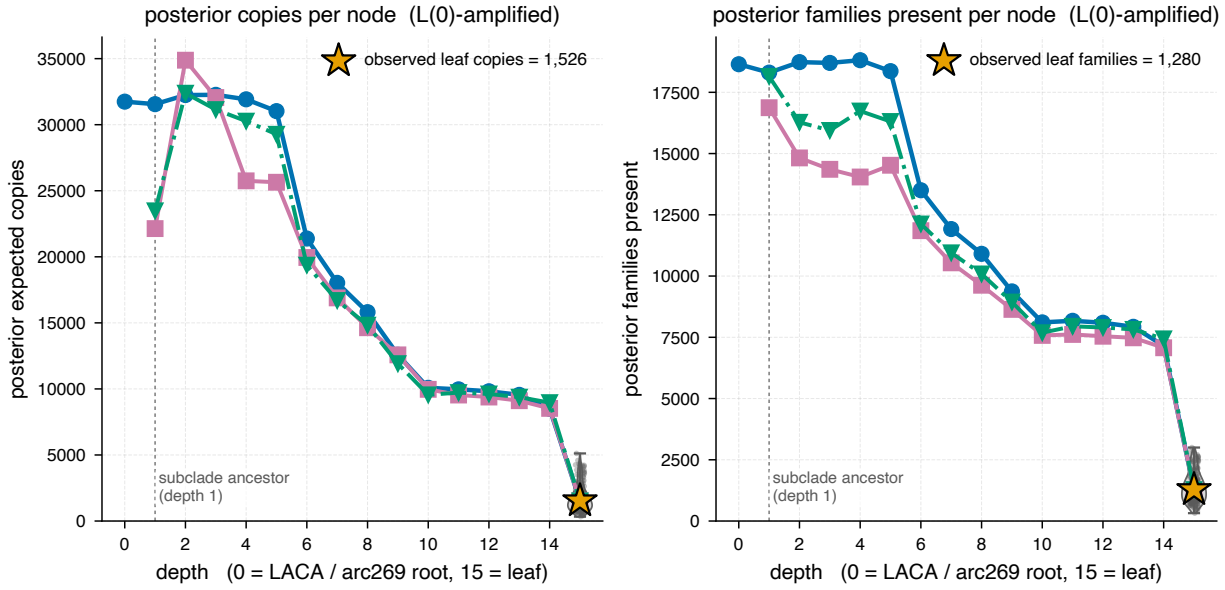
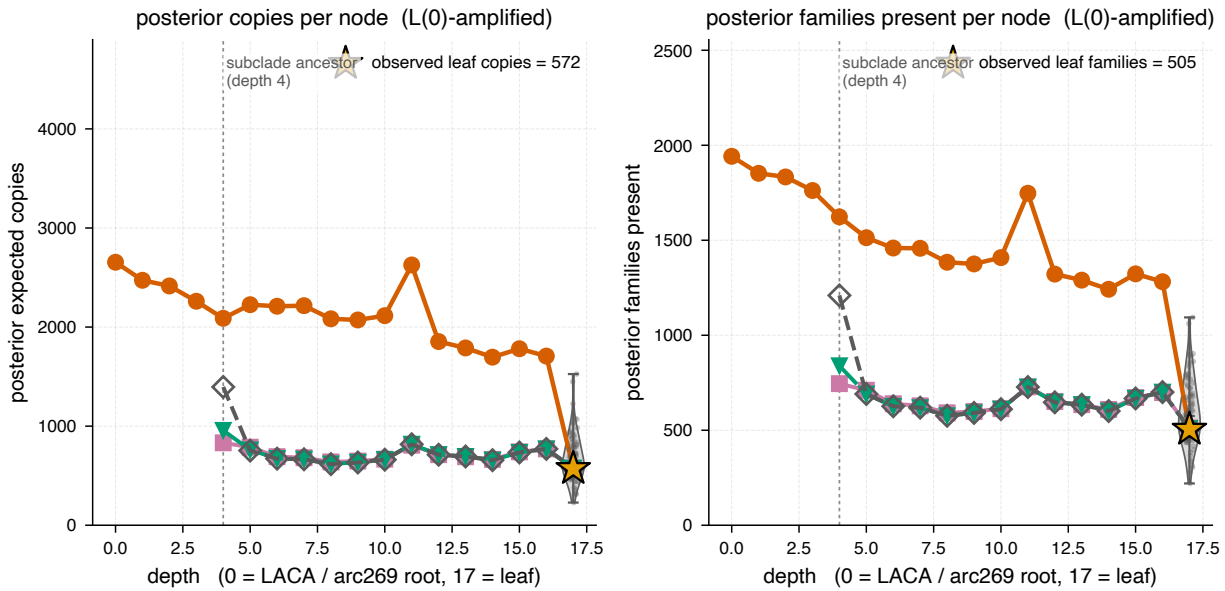


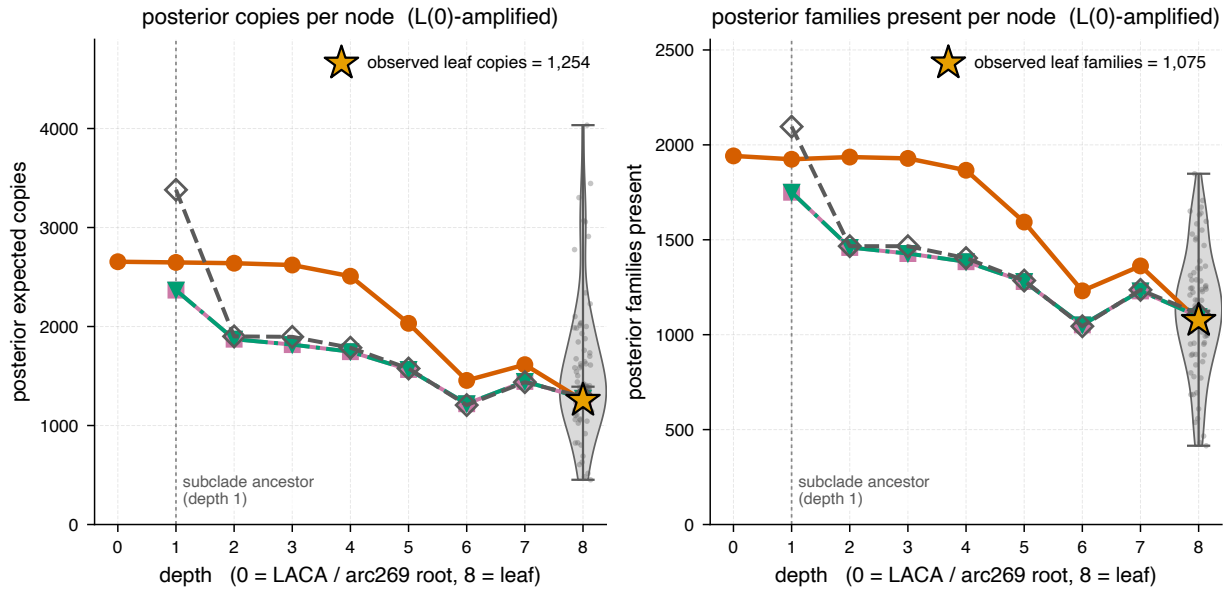
Figure 18. Set 2 (min = 1, no filter) root-to-leaf trajectories with the $F^* = F/(1-L(0))$ amplification applied per line.

Set 3 — L(0)-corrected

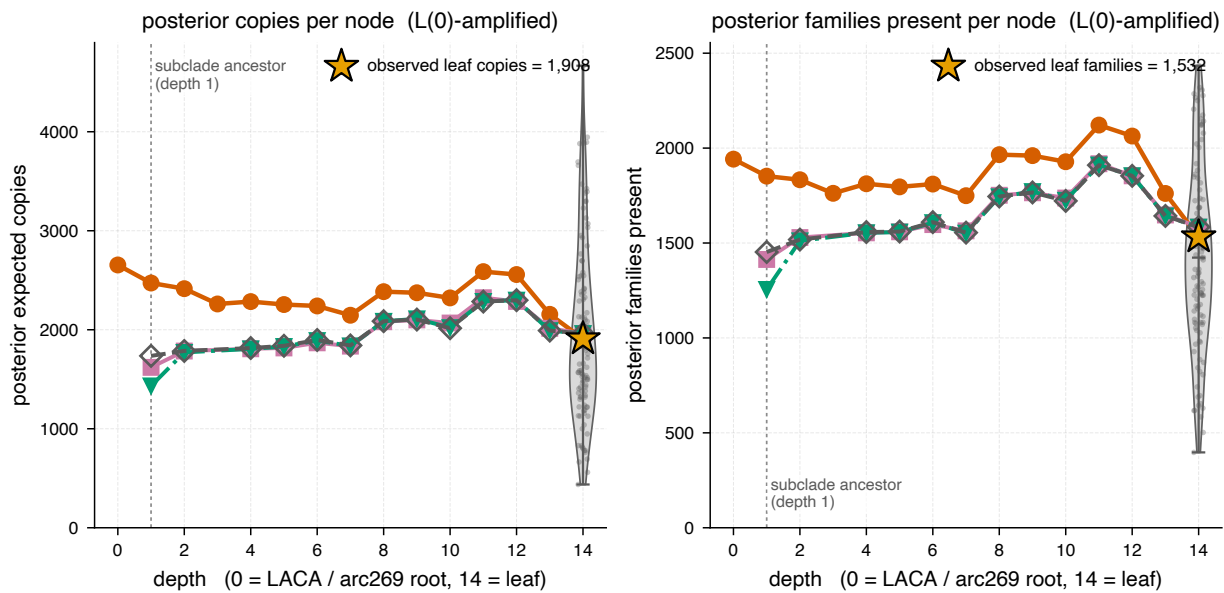
DPANN (D80) · Set 3 · arc269 root to median-fm leaf '1974445' (F = 11,148)



Proteoarchaea (P75) · Set 3 · arc269 root to median-fm leaf '2058097' (F = 11,148)



Methanobacteriati (E114) · Set 3 · arc269 root to median-fm leaf '679901' (F = 11,148)



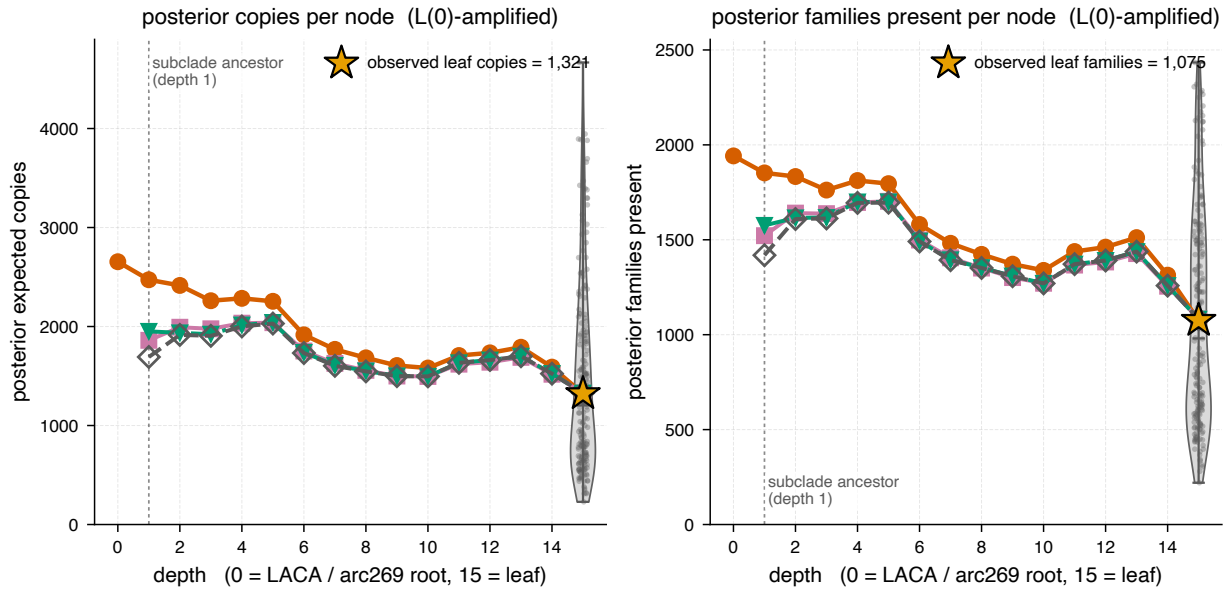


Figure 19. Set 3 (arc269 full-tree fit on the union of per-clade min = 4 families) root-to-leaf trajectories with the $F^* = F/(1-L(0))$ amplification applied per line.

License

Apache License 2.0 — see [LICENSE](#). Same license as the original Count Java package.